CORE COMPUTER SCIENCE · VOLUME 2

Everything About Hash Tables

A printed handnote: 91 topics, worked in C++, with practice sets, answer keys and revision cards.

Chaining and open addressing · probe sequences and tombstones · Swiss tables and cache lines · the standard-library layer in six languages · fifteen interview patterns · hash flooding, key mutation and the p99 resize spike

Contents

PART I

Mechanism

What the machine actually does. Sections 1–5: the array-index trick, the hash function itself, chaining, open addressing, and why the layout in memory decides the constant factor.

SECTION 1 — FOUNDATION

1. The array-index trick: why O(1) is possible at all

An array subscript is not a search. `a[i]` compiles to one address computation, `base + i * sizeof(T)` — a shift and an add — followed by one load. The cost does not depend on how many elements the array holds. A hash table is the observation that this is the only genuinely constant time lookup a machine offers, plus a function that turns a key you actually have into a subscript you can use.

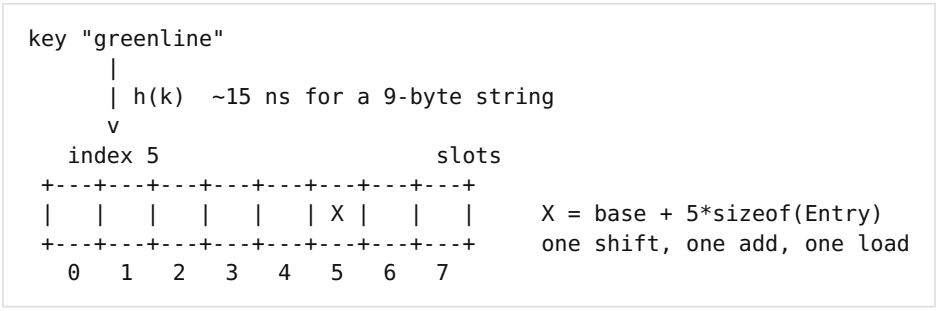
So a hash table lookup is exactly three things: compute `h(key)`, index the array once, and compare the key you found against the key you wanted. Everything else in this book — chaining, probing, tombstones, Swiss groups — exists because step three sometimes says "no".

```
#include <string>
#include <vector>

// The whole idea, minus collision handling.
struct Entry { std::string key; int val; bool used = false; };

size_t h(const std::string& k, size_t m) {
    size_t x = 14695981039346656037ULL;          // FNV-1a offset basis
    for (unsigned char c : k) { x ^= c; x *= 1099511628211ULL; }
    return x & (m - 1);                           // m is a power of two
}

int get(const std::vector<Entry>& t, const std::string& k) {
    const Entry& e = t[h(k, t.size())];           // ONE indexed load
    return (e.used && e.key == k) ? e.val : -1;   // ONE comparison
}
```



THE KEY INSIGHT

A hash table does not search for the key. It *computes where the key would have to be*, looks there once, and verifies. Search is what happens only when that guess collides.

THE TRAP

The code above silently loses data: two keys landing on slot 5 overwrite each other and `get` returns `-1` for a key you definitely inserted. "Key I just put in reads back as missing" is the signature of a table with no collision strategy — see Topics 16 and 26.

```
lookup = hash(k) + 1 indexed load + 1 key compare · Time O(1) expected · Space O(m) slots for n keys
```

2. The dictionary ADT and its four implementations

"Hash table" is an *implementation*. The interface it implements is the dictionary (also: map, associative array, symbol table): a set of key-value pairs supporting `insert`, `find`, `erase`. Four data structures implement it, with genuinely different guarantees, and choosing badly is a much bigger mistake than tuning a load factor badly.

Implementation	find	Ordered?	Worst case	Bytes/entry, 1M int->int
Hash table	O(1) expected	No	O(n)	~32–48
Balanced BST (red-black)	O(log n) — 20 compares at n=1M	Yes	O(log n)	~48–64
Sorted array + binary search	O(log n), no pointer chasing	Yes	O(log n)	8–16, insert is O(n)
Trie	O(len(key))	Yes (lexicographic)	O(len)	high; one node per prefix

```
#include <map>             // red-black tree: ordered, O(log n)
#include <unordered_map>   // hash table:    unordered, O(1) expected

std::map<std::string,int>           tree;   // begin() is the smallest key
std::unordered_map<std::string,int> hash;   // begin() is arbitrary

// Both answer this identically:
tree["a"] = 1; hash["a"] = 1;
// Only one answers this at all:
// for (auto it = tree.lower_bound("m"); it != tree.end(); ++it) ...
```

THE KEY INSIGHT

The hash table trades *order* for *speed*. Every capability you lose — sorted iteration, range queries, predecessor/successor, "the smallest key >= x" — is a consequence of that single trade, not a separate limitation.

```
dictionary ADT: insert · find · erase. Ordered variants add: min, max, range, predecessor,
successor — none of which a hash table can do without a full O(n + m) scan.
```

3. Direct-address tables, and when a plain array still wins

If your keys are integers in `[0, U)` and `U` is small, you do not need a hash function at all. The key *is* the index. This is a direct-address table, and it is strictly faster than any hash table: no hash to compute, no collisions to resolve, no equality comparison, no load factor.

```
// Character frequency over ASCII: U = 128, dense, tiny.
int freq[128] = {0};
for (unsigned char c : text) ++freq[c];        // 1 load, 1 add, 1 store

// The hash-table version does the same work plus a hash and a compare:
std::unordered_map<char,int> f;
for (char c : text) ++f[c];                    // ~4-8x slower, 40x memory
```

```
DIRECT ADDRESS (U = 8)              HASH TABLE (m = 8)
key 5 --------------> slot 5       key 5 -- h(5) --> slot 3 -- compare --> hit
no hash, no compare                 hash cost + compare cost + collisions
memory: U slots always              memory: m slots, m > n
```

The break-even is about *density*, not size. Keys 0–1,000,000 with 900,000 present: the direct array costs 4 MB and never misses a beat. The same range with 500 keys present costs the same 4 MB to hold 2 KB of data — a 2000:1 waste, and now the hash table wins. The rule of thumb is a load of roughly 1/8: below that, hash; above it, direct-address.

THE KEY INSIGHT

A hash table is a direct-address table for a key universe too large to allocate. If the universe fits in memory, you have been sold a solution to a problem you do not have.

THE TRAP

Reaching for `unordered_map<int,int>` to count 26 letters or 12 months. Symptom: a profiler showing 30–40% of run time inside hashing and allocation on a loop that should be three instructions. See Topic 71.

```
direct address — Time O(1) worst case (not expected) · Space O(U) regardless of n · zero hash cost, zero collisions
```

4. Vocabulary: slot, bucket, load factor, probe, fingerprint

The literature uses these words inconsistently; this book uses them as follows, and so does most modern source code.

Term	Meaning in this book
Slot	One addressable cell of the backing array.
Bucket	Everything that hashes to one index. Under chaining, a bucket is a list and may hold many entries; under open addressing, bucket and slot are the same thing.
Capacity `m`	Number of slots. Almost always a power of two or a prime.
Size `n`	Number of live entries.
Load factor α	`n / m`. The single number that predicts performance.
Collision	Two distinct keys with the same index. Not a bug; a certainty.
Probe	One inspection of a slot during a lookup. Probe count, not "O(1)", is the honest cost metric.
Fingerprint (h2)	A few extra bits of the hash stored alongside the entry so a mismatch can be rejected without touching the key. Topic 38.
Tombstone	A slot marked "was occupied, now deleted" so probe sequences do not break. Topic 30.
Rehash	Reinserting every entry into a new, larger array. Topic 22.

```
              m = 8 slots, n = 5 entries, alpha = 5/8 = 0.625
     idx: 0     1     2     3     4     5     6     7
         [ ]  [K1]  [K2]  [ ]  [K3]  [K4]  [K5]  [T]
                           ^                       ^
                        free slot                tombstone
  lookup("K5") -> h=4 -> probe 4 (K3, miss), 5 (K4, miss), 6 (K5, HIT)
                = 3 probes
```

THE KEY INSIGHT

α = n/m is the control variable of the entire subject. Every formula in Sections 3 and 4 is a function of α alone — never of n.

```
notation used throughout: n entries · m slots · α = n/m · h(k) index · h2(k) fingerprint
```

5. Expected O(1) vs worst-case O(n): who picks the input

"O(1) average" is a claim about a probability distribution, and the distribution is over *hash outcomes*, not over inputs. The standard assumption — simple uniform hashing — is that each key is equally likely to land in each of the m slots, independently. Under it, the expected chain length is α and lookup is O(1 + α). If that assumption fails, so does the bound: every key can land in one bucket and the table degrades to a linked list with O(n) lookup.

Two different things can break it. An unlucky hash function for your particular key set (all keys are multiples of 16 and m is a power of two, Topic 10), or an *adversary* who chooses keys after learning your hash function (Topic 79).

```
// Collisions are not rare events; they are the normal case.
// Expected colliding PAIRS with n keys in m slots = C(n,2)/m.
//   n = 1,000   m = 1,000,000  ->  999*1000/2/1e6   = 0.50 pairs
//   n = 10,000  m = 1,000,000  ->  ~50 pairs
//   n = 23      m = 365        ->  0.69  (the birthday paradox: P ~ 50%)
// P(no collision at all) ~ exp(-n^2 / 2m)
```

```
alpha:      0.5      0.75      0.9      0.95     1.0(chaining)
chained     1.25     1.38      1.45     1.48     1.50   avg probes, hit
linear      1.50     2.50      5.50    10.50      inf   avg probes, hit
linear      2.50     8.50     50.50   200.50      inf   avg probes, MISS
                                      ^^^^^ the cliff is not gradual
```

THE KEY INSIGHT

The average case is an average over hash randomness, which you control by choosing the hash function. It is not an average over inputs, which you do not control at all.

THE TRAP

Quoting "O(1) amortized" in a latency budget. Amortized means the *total* over many operations is bounded; one individual insert that triggers a rehash of 4M entries can take 40 ms. If you have a p99 SLO, you care about the individual operation. Topic 88.

```
lookup — expected O(1 + α) · worst case O(n) · the worst case is reachable by an attacker in one
HTTP request
```

6. Hash table vs sorted array vs BST vs trie

The honest comparison, and the point in the book where it is worth saying plainly: the hash table is often the wrong choice.

You need	Use	Why not a hash table
Exact-key lookup, any order	**Hash table**	—
Range query, "all keys in [a,b]"	B-tree / sorted array	Hashing destroys ordering by design; you would scan all m slots
Predecessor / successor / min / max	BST, skip list	Same reason; O(n + m) per query
Prefix search, autocomplete	Trie	A hash of "gre" tells you nothing about "green"
Sorted output at the end	Sorted container, or hash + one sort	Hash + sort is O(n log n) anyway — often still the right call
Hard worst-case bound (real time, avionics)	Balanced BST	O(log n) always beats O(1) expected / O(n) worst
n < ~16	Flat array + linear scan	16 sequential compares are one cache line; the hash alone costs more
Keys are dense small integers	Direct-address array (Topic 3)	Hash is pure overhead
Memory is the binding constraint	Sorted array	A hash table is deliberately ≥15–33% empty

```
// Small-n reality check, measured shapes not adjectives:
//   n = 8   linear scan over vector<pair<int,int>>:  ~5 ns  (1 cache line)
//   n = 8   unordered_map<int,int>::find:            ~20 ns (hash + chase)
//   n = 1M  linear scan:                             ~500 us
//   n = 1M  unordered_map::find:                     ~60-100 ns
// The crossover is around n = 30-100 for trivial keys.
```

THE KEY INSIGHT

Ask "will I ever need these keys in order?" before "how do I make lookup fast?". Retrofitting order onto a hash table is a rewrite; retrofitting speed onto a tree is a load factor.

THE TRAP

Using a hash map as a cache keyed on time, then discovering you need "all entries older than T". Symptom: a full scan on every eviction pass, O(n) per tick. You needed a heap or an ordered structure beside the map — Topics 72 and 73.

```
decision order: (1) do I need order? (2) is the universe dense? (3) is n tiny? (4) is the input adversarial? Only then reach for a hash table.
```

PRACTICE SET — Topics 1–6

A. Conceptual

1. In one sentence, what does a hash function let you avoid doing?
2. Why is `a[i]` constant time while `list.at(i)` on a linked list is not, given both end in a load?
3. A colleague says "our map is O(1), so latency is bounded." Give the two distinct reasons that is false.
4. Define α without using the words "how full".
5. Name three operations a `std::map` supports that no hash table can support in better than O(n + m).
6. You have keys 0–255, all present. Argue for the direct-address array over a hash table using memory numbers, not adjectives.
7. The simple uniform hashing assumption is an assumption about what, exactly — the keys, the hash function, or the pairing of the two?
8. Why does a fingerprint (h2) reduce work even though the full key is stored right next to it?
9. Give a workload where a sorted `vector` beats both `map` and `unordered_map`.

B. Tracing and analysis

1. The ten keys below were hashed with FNV-1a and masked with `& 7` (m = 8). Given the slots `apple=1, banana=2, cherry=2, date=7, elder=5, fig=7, grape=2, kiwi=7, lemon=2, mango=3`: how many slots are occupied, how many are empty, what is the longest bucket, and what is α?
2. For the same ten keys, compute the expected number of colliding pairs under simple uniform hashing with m = 8, and compare it to the actual count.
3. A table holds 3,000,000 entries at α = 0.75. How many slots does it have? If each slot is 16 bytes, how much memory is the slot array, and how much of it is empty?
4. Using P(no collision) ≈ $\exp(-n^2/2m)$: with m = 65,536, at what n does the probability of at least one collision first exceed 50%?
5. An interviewer says lookup costs "one hash plus one memory access". A miss in L3 costs 80 ns and the hash costs 15 ns. What is the lookup cost, and what does the same table cost if the entry is in L1 (1 ns)?
6. You must store 500 keys drawn from the range 0–1,000,000. Give the memory cost of a direct-address array of `int` and of a hash table at α = 0.5 with 16-byte slots. Which wins, and by what factor?

C. Code tasks

1. **Build.** Type the eight-slot `Entry` table from Topic 1 verbatim, with `m = 8`. Insert the ten fruit keys from B1.
2. **Break it and observe.** Now call `get("banana")`, `get("cherry")`, `get("grape")` and `get("lemon")`. Record which succeed. Before running it, predict the answer from the slot list. Do not fix it yet — write down what a user of this map would report as the bug.
3. **Break it and observe.** Write a byte-frequency counter as `int freq[128]` indexed by a plain `char`, and feed it the string `"café"` (UTF-8). Compile with `g++ -fsanitize=address -g` and run. Record the sanitizer's first line and the reported index. Then fix it two ways — `unsigned char`, and `freq[256]` — and say which is the real fix.

4. **Measure.** Time 10^7 increments of `int freq[128]` against `std::unordered_map<char,int>` over the same text. Report the ratio. Then repeat with keys spread over 0–1,000,000 with only 500 distinct values present, and report the memory of each.
5. **Find the crossover.** For n = 4, 8, 16, 32, 64, 128, 1024, compare lookup time in a `std::vector<std::pair<int,int>>` scanned linearly against `unordered_map`. Report the n at which the hash table first wins on your machine.
6. **Observe the missing guarantee.** Insert the integers 1–20 into an `unordered_map<int,int>` and print them by iterating. Do the same for `std::map`. Then insert the same 20 integers in reverse order into a fresh `unordered_map` and print again. Explain the two orders you saw, and why neither is a bug.

ANSWER KEY — Topics 1–6

A. Conceptual

1. **Searching.** It converts a key into an array subscript, so you compute a location instead of scanning for it.
2. Array indexing is **address arithmetic on a contiguous block**: `base + i*sizeof(T)`, computable without touching memory. A linked list must load node i−1 to learn where node i is, so it performs i dependent loads.
3. **(a)** O(1) is expected, not worst case — a bad hash or an adversary makes it O(n) (Topics 5, 79). **(b)** Even when the average holds, a single insert can trigger a full rehash: amortized O(1) hides a 40 ms outlier at n = 4M (Topic 88).
4. α = **n/m**: live entries divided by slots. It is a ratio, not a percentage of memory, and it can exceed 1 under chaining.
5. Any three of: **min/max, predecessor/successor, range query [a,b], ordered iteration, lower_bound**.
6. Direct-address: **256 × 4 B = 1 KB**, zero hashing, zero collisions, O(1) *worst case*. A hash table at α = 0.5 needs 512 slots × ~16 B = **8 KB** plus a hash per access — 8× the memory to do strictly less.
7. About **the pairing**. No hash function is uniform for all key sets, and no key set is uniform under all hash functions. That is exactly why the fix for adversarial input is to randomize the function (Topic 80).
8. Because comparing 7 bits is **one register operation on a byte you already loaded**, while comparing the key may mean dereferencing a pointer to a heap string — a potential cache miss costing 80 ns to reject a candidate that a 1-cycle test could have rejected.
9. **Build once, then read-only, memory-tight, needs ordering** — e.g. a static lookup table shipped in a binary. 8 B/entry, binary search is 20 cache-friendly compares at n = 1M, and it sorts for free.

B. Tracing and analysis

1. Occupied slots are 1, 2, 3, 5, 7 → **5 occupied, 3 empty** (0, 4, 6). Longest bucket is slot 2 with **4 keys** (banana, cherry, grape, lemon); slot 7 holds 3. α = 10/8 = **1.25** — legal under chaining, impossible under open addressing.
2. Expected pairs = C(10,2)/m = 45/8 = **5.6**. Actual: slot 2 contributes C(4,2)=6, slot 7 contributes C(3,2)=3, total **9**. Higher than expected, which is ordinary variance at n = 10 — the formula is an expectation, not a bound.
3. m = n/α = 3,000,000/0.75 = **4,000,000 slots**, rounded up to a power of two that is **4,194,304**. At 16 B: 4,194,304 × 16 = **67.1 MB**, of which 1,194,304 slots × 16 B = **19.1 MB (28%) is empty**. That emptiness is the price of $\alpha \leq 0.75$.
4. Solve $\exp(-n^2/131{,}072) = 0.5 \rightarrow n^2 = 131{,}072 \times 0.693 = 90{,}876 \rightarrow n \approx$ **302 keys**. Three hundred keys in sixty-five thousand slots already collide half the time.
5. **95 ns** cold (15 + 80) and **16 ns** hot (15 + 1). The hash is 16% of the cold cost and 94% of the hot one — which is why Section 5 is about cache lines and Section 2 is about cheap hashes.
6. Direct address: 1,000,001 × 4 B = **4.0 MB**. Hash table: 1000 slots × 16 B = **16 KB**. The hash table wins by **250×**. Density here is 500/1,000,000 = 0.0005, far below the ~1/8 rule of thumb from Topic 3.

C. Code tasks — what to verify

1. Compiles and runs; ten inserts into eight slots must lose data.
2. Expect `get("banana")`, `get("cherry")` **and** `get("grape")` **to return −1** and only `get("lemon")` — the last writer to slot 2 — to return its value. Same on slot 7: only `kiwi` survives. The bug report a user files is **"the key I just inserted is missing"**, and it is intermittent, data-dependent and unreproducible on small test sets. That is the phenomenon Sections 3 and 4 exist to prevent.
3. Expected first line: `ERROR: AddressSanitizer: global-buffer-underflow` (or `stack-buffer-underflow` if `freq` is a local) **on address … which is 244 bytes to the left** of the array, because `'é'` in UTF-8 is the two bytes 0xC3 0xA9, which as `signed char` are **−61 and −87**. `unsigned char` is the **real fix**: it makes the index provably 0–255. Enlarging to `freq[256]` alone does not help — the index is still negative.
4. Expect the array to be **4–8×** faster; the gap is hashing plus a dependent load. On the sparse key set expect roughly **4 MB (array) vs ~16–32 KB (map)** — the direction reverses completely.
5. Typical crossover is **n ≈ 30–100**. Below 16, the vector often wins by 3–4× because the whole scan is one or two 64-byte cache lines.
6. `std::map` prints **1..20 ascending, always**. `unordered_map` prints an order determined by `hash(k) % bucket_count`; libstdc++ hashes integers to themselves and inserts at the *head* of each bucket, so you will typically see a descending or block-shuffled order, and the **reverse-insertion run may differ from the first**. Neither is a bug: the standard guarantees only that iteration visits each element once. Depending on it is Topic 81.

HANDNOTE SUMMARY CARD — Foundation

```
FOUNDATION (1-6)                                          alpha = n/m
================================================================================
THE ONE IDEA
  lookup = h(key) -> index -> compare.  Not a search: a computed guess + a check.
  Everything else in the book handles the case where the check says "no".

VOCAB
  slot   one array cell            m  capacity (slots)     alpha  n/m
  bucket all keys at one index     n  live entries         probe  one slot visit
  h1     which slot                h2 fingerprint (7 bits, rejects without deref)
  tombstone  deleted-but-not-empty marker   rehash  reinsert all into new array

COST MODEL (what to say in an interview)
  hash a short string        ~10-20 ns
  L1 hit                     ~1 ns        L3 hit   ~15 ns       DRAM  ~80-100 ns
  lookup, hot                ~16 ns       lookup, cold          ~95 ns
  => the hash is 94% of a hot lookup and 16% of a cold one.

AVG PROBES vs ALPHA
  alpha        0.50   0.75   0.90   0.95
  chaining     1.25   1.38   1.45   1.48      (hit)   = 1 + alpha/2
  linear hit   1.50   2.50   5.50  10.50      = (1 + 1/(1-a))/2
  linear MISS  2.50   8.50  50.50 200.50      = (1 + 1/(1-a)^2)/2   <-- the cliff

COLLISION MATH
  expected colliding pairs  = C(n,2)/m
  P(no collision)          ~= exp(-n^2 / 2m)
  m = 65536 -> 50% chance of a collision at just n = 302 keys.

FOUR IMPLEMENTATIONS OF THE DICTIONARY ADT
  hash table    O(1) exp    unordered   worst O(n)     ~32-48 B/entry
  red-black     O(log n)    ordered     worst O(log n) ~48-64 B/entry
  sorted array  O(log n)    ordered     insert O(n)    8-16  B/entry
  trie          O(len)      prefix-ord  worst O(len)   high

WHEN THE HASH TABLE IS THE WRONG ANSWER
  need range / min / max / predecessor  -> B-tree, skip list
  need prefix or autocomplete           -> trie
  keys are dense small ints             -> plain array (direct address)
  n < ~30                               -> vector + linear scan
  hard worst-case latency bound         -> balanced BST
  memory is the binding constraint      -> sorted array (hash is 15-33% empty)
  input chosen by a stranger            -> randomized hash, or a tree (T79)

DIRECT-ADDRESS RULE OF THUMB
  density = n / U.   density > 1/8  -> plain array.   density < 1/8 -> hash.
  keys 0..1e6, n=500 -> array 4.0 MB vs hash 16 KB  -> hash wins 250x
  keys 0..255, n=256 -> array 1 KB   vs hash 8 KB   -> array wins 8x

TRAPS
  "O(1) so latency is bounded"   -> amortized hides a 40 ms rehash (T88)
  no collision strategy          -> "the key I just inserted is missing"
  signed char as an index        -> ASAN buffer-underflow at index -61 on UTF-8
  relying on iteration order     -> differs by insertion order and by run (T81)
```

SECTION 2 — HASH FUNCTIONS

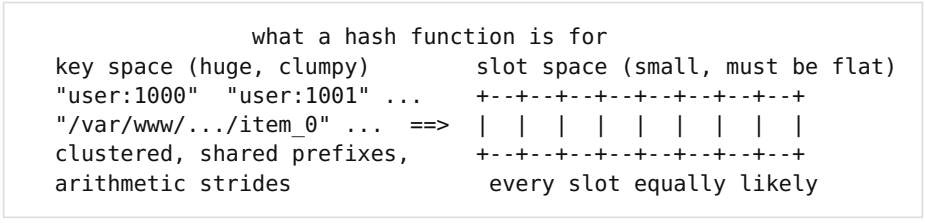
7. What a hash function must do

A hash function for a hash table has exactly three obligations, and one non-obligation that people wrongly add.

1. **Deterministic within a process.** The same key must give the same value every time, or lookups fail. Note the qualifier: *within a process*. Modern runtimes deliberately change the value between processes (Topic 80), so a hash value is never a valid disk or wire format.
2. **Uniform for your key set.** Every slot equally likely. This is a property of the pair (function, keys), not of the function alone.
3. **Cheap.** A hot lookup is ~16 ns and the hash is most of it (Topic 5). A 200 ns cryptographic hash makes the table 10× slower.

The non-obligation: **it need not be collision-resistant in the cryptographic sense**. m is small, so collisions are guaranteed by the pigeonhole principle regardless. What you sometimes need is *unpredictability* — an attacker unable to *construct* collisions — which is a keyed property, not a strength property.

```
// The same 10,000 keys, three hash functions, m = 16384, mask indexing.
// Ideal (perfectly random) empty-slot count = m*exp(-n/m) = 8898.
//
// key set        function   longest bucket   empty slots   variance
// random 8-char  djb2                    5          8915      0.617
// random 8-char  fnv1a                   6          8942      0.618
// random 8-char  sdbm                    5          8888      0.611
// "user:1000"..  djb2                    4         10433      0.882
// "user:1000"..  fnv1a                   5          9076      0.635
// "user:1000"..  sdbm                    9         12778      2.352   <-- 78% empty
```



THE KEY INSIGHT

Real keys are never random. They are sequential ids, shared-prefix paths, aligned pointers and timestamps. A hash function's job is to destroy exactly that structure — and the functions that fail do so only on structured keys, which is why a random-key benchmark proves nothing.

THE TRAP

Benchmarking a hash function on random strings. All three functions above are statistically indistinguishable there (max bucket 5–6, 8,900 empty). Switch to your production key format and sdbm's worst bucket goes to 9 with 78% of the table unused. Benchmark on *your* keys.

```
hash cost budget — short string 10–20 ns · int 1–3 ns · SipHash-1-3 ~1 ns/byte + 20 ns fixed ·
SHA-256 ~200+ ns. Anything above ~30 ns dominates the lookup.
```

8. The division method, and why m is prime

`h(k) = k mod m`. One instruction on paper; on hardware, a 64-bit integer division costs **20–40 cycles** versus 1 cycle for a bitwise AND, which is the entire reason Topic 10 exists.

The reason to pay it: `mod m` for prime m mixes *all* bits of k into the result, while `& (m-1)` uses only the low $\log_2 m$ bits. If your hash function has weak low bits — sdbm, or an identity hash on pointers — the prime modulus rescues you and the mask does not.

```
// Same sdbm hash, same 10,000 structured keys, two index derivations:
//   m = 16384 (power of two), h & (m-1):   longest bucket 9, 12778 empty
//   m = 16381 (prime),        h %  m   :   longest bucket 3,  7485 empty
// The hash function did not change. Only the last operation did.

size_t index_prime(uint64_t h) { return h % 16381; }   // ~20-40 cycles
size_t index_mask (uint64_t h) { return h &  16383; }   // 1 cycle
```

Choose a prime that is *not* close to a power of two and not of the form $2^i \pm 1$: near-power-of-two primes reintroduce the very bias they are meant to remove. libstdc++'s `unordered_map` ships a table of primes and grows through it (13, 29, 59, 127, 257, 541, ...) precisely so it can be safe with mediocre user-supplied `std::hash` specialisations.

THE KEY INSIGHT

Prime modulus and a strong hash are substitutes, not complements. Pay for one of them: a prime `%` with a weak hash, or a mask with a well-mixed hash. Paying for neither is the bug; paying for both is 30 wasted cycles.

THE TRAP

`k % m` where k is a signed `int` and negative: `-7 % 8` is `-7` in C++, not 1. Symptom: a segfault or an ASAN buffer-underflow at a negative index, exactly as in Practice C3. Cast to an unsigned type before the modulus, always.

```
division method — index cost 20–40 cycles · uses all bits of h · requires a prime table for
growth · immune to weak low bits
```

9. Multiplicative and Fibonacci hashing

Knuth's multiplicative method takes the *high* bits of a product instead of the low bits of a division. Multiply the key by a constant A, keep the fractional part, scale to m. In 64-bit integer form with m a power of two, it is one multiply and one shift — 3–4 cycles total, and it mixes low input bits upward into the result.

```
// Fibonacci hashing: A = 2^64 / phi = 11400714819323198485
// (phi = 1.618..., the golden ratio; it minimises clustering for
//  arithmetic key sequences, which is exactly what real ids are.)
constexpr uint64_t GOLDEN = 11400714819323198485ULL;

size_t fib_index(uint64_t key, int log2m) {
    return (key * GOLDEN) >> (64 - log2m);   // take the HIGH log2m bits
}
// key = 1,2,3,4 with log2m = 4 (m = 16) gives 0, 9, 3, 12 -- spread out,
// whereas key & 15 gives 1, 2, 3, 4 -- adjacent, and clusters under
// linear probing (Topic 27).
```

```
       key * GOLDEN  (64-bit product, low bits overflow away)
63                                                            0
+-------------------------------------------------------------+
|  keep these  |            discard these                      |
+-------------------------------------------------------------+
 ^ high bits depend on EVERY input bit, because multiplication
   propagates carries leftward -- the mirror image of masking
```

THE KEY INSIGHT

Multiplication propagates influence *upward*. Masking reads the *bottom*. Fibonacci hashing exists to reconcile the two: multiply first so the top bits are well mixed, then take from the top.

THE TRAP

Multiplying and then masking the low bits — `(key * GOLDEN) & (m-1)`. The low bits of a product are exactly as structured as the low bits of the input (the bottom bit of an odd multiplier times an even key is still 0). Symptom: an even-key workload uses only half the slots. Shift, do not mask.

```
multiplicative — 1 multiply + 1 shift, 3–4 cycles · m must be a power of two · quality depends on
A being odd and irrational-scaled
```

10. Power-of-two masking and the low-bits trap

Nearly every modern table (Go, Rust, absl, Java, PHP) uses `m = 2^k` and indexes with `h & (m-1)`. It costs 1 cycle, and it makes growth a doubling with a cheap re-index. The price is that **only the low k bits of the hash are ever consulted**. Every bit of quality above bit k is thrown away.

The classic failure is hashing addresses or aligned integers with an identity hash. Allocations are 16- or 64-byte aligned, so the low 4–6 bits are always zero.

```
// 1000 pointer-like values, 64 bytes apart, m = 1024, mask indexing:
//   identity hash:        longest bucket 63,  1008 of 1024 slots EMPTY
//   fmix64 then mask:     longest bucket  5,   366 of 1024 slots empty
// Same keys, same table, same load factor. The only change is 6
// instructions of bit mixing (Topic 13).
```

```
key = 0x7F0000001000        low 6 bits are structurally zero
                      |
       & 1023   ->   ...0000000000    always lands on a multiple of 64
                                       => 16 usable slots out of 1024
       fmix64   ->   pseudo-random 64 bits, all slots reachable
```

Java's `HashMap` patches this in the container rather than trusting the user: it computes `h ^ (h >>> 16)` before masking, folding the high half down. It is one XOR and one shift, and it is there because `Integer.hashCode()` is the identity function.

THE KEY INSIGHT

A mask is a *projection*, not a hash. If the bits you project are structured, the table is structured. Either mix before masking or divide by a prime — Topic 8's rule, stated from the other side.

THE TRAP

C++'s `std::hash<int>` is the identity function in libstdc++ and libc++. It is safe there only because libstdc++ uses prime moduli. Port that key type to a power-of-two table (absl, your own) without adding a mixer and throughput drops by 10×. Symptom: a profiler showing enormous time in probe loops with a low load factor.

masking — 1 cycle · consults only $\log_2 m$ bits of h · growth is a cheap doubling · demands a well-mixed hash as a precondition

11. String hashing I: DJB2, FNV-1a, sdbm

All three are byte-at-a-time loops of the same shape: a running accumulator, one multiply and one combine per byte. They differ only in constants and in the order of the XOR.

```
uint64_t djb2(const std::string& s) {          // h*33 + c
    uint64_t h = 5381;
    for (unsigned char c : s) h = h * 33 + c;   // = (h << 5) + h + c
    return h;
}
uint64_t fnv1a(const std::string& s) {         // XOR first, then multiply
    uint64_t h = 14695981039346656037ULL;        // offset basis
    for (unsigned char c : s) { h ^= c; h *= 1099511628211ULL; }
    return h;                                     // prime = 2^40 + 2^8 + 0xb3
}
uint64_t sdbm(const std::string& s) {          // h*65599 + c
    uint64_t h = 0;
    for (unsigned char c : s) h = c + (h << 6) + (h << 16) - h;
    return h;
}
```

Function	Speed	Low-bit quality	Verdict
djb2	~1 byte/ cycle	Weak for short keys: single chars 'a'–'e' give low nibbles 6,7,8,9,10 — consecutive	Fine with a prime m; mix before masking
FNV-1a	~1 byte/ cycle	Good — the XOR precedes the multiply, so each byte affects the whole word	The best of the three by default
sdbm	~1 byte/ cycle	Poor under masking: 12,778/16,384 slots empty on structured keys	Only with `% prime`

All three are *slow* by modern standards for long keys: one byte per iteration with a serial dependency. xxHash3 and wyhash process 8–32 bytes per iteration and reach 10–30 GB/s; FNV-1a manages roughly 1 GB/s. For keys under 16 bytes the difference is noise; for hashing 4 KB blobs it is 20×.

THE KEY INSIGHT

FNV-1a's advantage over FNV-1 is one line: XOR the byte in *before* multiplying, so the byte's influence is spread by the very next multiply rather than sitting in the low 8 bits until the loop ends.

THE TRAP

Assuming a fast hash makes lookups fast. If keys are 8-byte strings, hashing is ~10 ns of a ~95 ns cold lookup. Replacing FNV with xxHash saves 7 ns and changes nothing. Profile before optimising the hash; the cache miss is usually the bill.

```
byte-at-a-time hashes — ~1 GB/s · 8-byte key ~10 ns · no SIMD · adequate below ~64-byte keys, replaceable above
```

12. String hashing II: the polynomial rolling hash

The same `h*B + c` shape as djb2, but taken modulo a chosen M and used for a different purpose: because the contribution of each character is B^{position}, you can *remove* the leading character and append a new one in O(1). That turns "hash of every length-L window" from O(nL) into O(n), which is what makes Rabin–Karp and Topic 70 work.

```
// Rolling hash over a sliding window of length L.
const uint64_t B = 131, M = (1ULL << 61) - 1;   // Mersenne prime

uint64_t pow_BL = 1;
for (int i = 0; i < L; ++i) pow_BL = pow_BL * B % M;

uint64_t h = 0;
for (int i = 0; i < L; ++i) h = (h * B + s[i]) % M;     // first window

for (int i = L; i < (int)s.size(); ++i) {
    h = (h * B + s[i]) % M;                              // append s[i]
    h = (h + M * B - pow_BL * s[i - L] % M) % M;         // drop s[i-L]
}
```

Choose B larger than the alphabet (131 or 1000003) and M a large prime. M = $2^{61}-1$ keeps products inside 128-bit intermediates and gives a collision probability per pair of about L/M. Over 10^6 windows the expected number of colliding pairs is $C(10^6,2)/2^{61} \approx \mathbf{2\times10^{-7}}$ — safe. With M = 10^9+7 the same computation gives **500** expected collisions, which is not.

THE KEY INSIGHT

A rolling hash is the only hash in this book whose *algebra* matters. Every other function is a black box you judge by distribution; this one you judge by whether you can subtract the old character back out.

THE TRAP

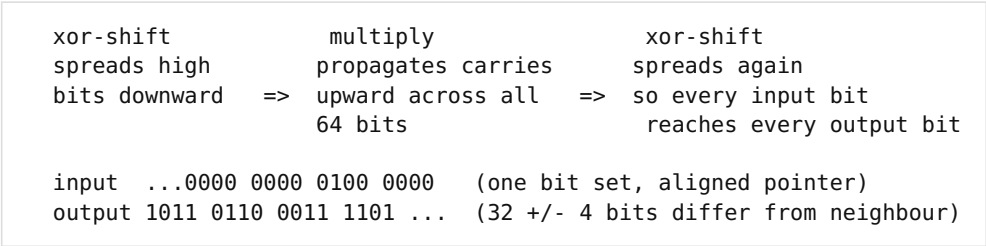
M = 2^{32} or a fixed public B on adversarial input. Thue–Morse strings break 32-bit polynomial hashes deterministically, and competitive programming judges have hacked thousands of submissions this way. Symptom: a solution that passes every test and fails one specific one at exactly the time limit. Randomize B at process start (Topic 80's discipline, applied here).

```
rolling hash — build O(n) · window update O(1), ~5 ops · collision probability per pair ~ L/M · needs modular arithmetic, not a mask
```

13. Avalanche and finalizers: fmix64, splitmix64

Avalanche is the measurable property behind "well mixed": flip one bit of the input and, on average, half the output bits should flip. For a 64-bit hash the ideal is **32.0 bits changed**. Measured over 20,000 single-bit flips: djb2 over decimal strings changes **22.3** bits; murmur3's `fmix64` changes **32.0**.

```
// murmur3 finalizer: 5 instructions, ~1 ns, turns any 64-bit value
// into a well-distributed one. This is the single most useful six
// lines in the whole subject.
uint64_t fmix64(uint64_t k) {
    k ^= k >> 33;
    k *= 0xff51afd7ed558ccdULL;
    k ^= k >> 33;
    k *= 0xc4ceb9fe1a85ec53ULL;
    k ^= k >> 33;
    return k;
}
// splitmix64 is the same shape with a counter added first; use it when
// you need a fast PRNG as well as a mixer.
```



THE KEY INSIGHT

You do not need a better hash function; you need a finalizer. Keep the cheap identity or djb2 hash for the key type and append `fmix64` before indexing. One nanosecond buys you the right to mask.

THE TRAP

Testing avalanche by eye. "It looks random" is not measurement. Count popcount of `h(x) ^ h(x ^ bit)` over thousands of trials; anything below ~28 or above ~36 bits average is biased. Practice B for this section walks the arithmetic.

```
fmix64 — 5 instructions, ~1 ns · avalanche 32.0/64 bits · turns a mask-indexed table from 16
usable slots into 658 (Topic 10's numbers)
```

14. Universal hashing

Every fixed hash function has a bad key set — a set of keys that all collide. For a fixed h, that set is discoverable, and Topic 79 is what happens when someone discovers it. Universal hashing removes the fixity: pick h at *random* from a family H at startup, such that for any two distinct keys x, y, `P[h(x) = h(y)] ≤ 1/m` over the choice of h.

```
// Carter-Wegman: p prime > universe size, a in [1,p-1], b in [0,p-1].
struct UniversalHash {
    uint64_t a, b, p = (1ULL << 61) - 1;
    size_t m;
    UniversalHash(size_t m_, std::mt19937_64& rng) : m(m_) {
        a = 1 + rng() % (p - 1);
        b = rng() % p;
    }
    size_t operator()(uint64_t k) const {
        return (unsigned)(((__uint128_t)a * k + b) % p) % m;
    }
};
// Multiply-shift variant (Dietzfelbinger), 2 ops instead of a modulus:
//   size_t h(uint64_t k) { return (a * k) >> (64 - log2m); }  // a odd, random
```

The guarantee is not "no collisions". It is that the expected chain length stays 1 + α *no matter which keys are inserted,* because the adversary must commit to keys before learning a and b. That is exactly the property Topic 80 buys with a per-process random seed.

THE KEY INSIGHT

Universal hashing moves the randomness from an assumption about the input to a fact about your own coin flips. It is the difference between "we hope keys are random" and "we guarantee the function is".

THE TRAP

Seeding the family with the current second, or with a constant in tests that ships to production. If an attacker can predict a and b, universality is worth nothing. Symptom: an application that is fine for months and then pins one CPU core at 100% during a POST flood.

```
universal family — P[collision] ≤ 1/m for any key pair · cost 1 multiply + 1 modulus (or 1
multiply + 1 shift) · requires a per-process random seed from a CSPRNG
```

15. Composite keys: combining hashes without cancellation

You have `struct Point { int x, y; }` and two good hashes. The naive combiner is XOR — and it is wrong in three specific ways.

```
// WRONG
size_t bad(const Point& p) { return std::hash<int>{}(p.x)
                                  ^ std::hash<int>{}(p.y); }
//  (1) commutative:  (3,7) and (7,3) collide.
//  (2) self-cancelling: every (x,x) hashes to 0 -- the diagonal
//      collapses onto ONE slot.
//  (3) inherits the identity hash's structure (Topic 10).

// RIGHT: boost::hash_combine -- mix, shift, add an irrational constant
inline void hash_combine(size_t& seed, size_t v) {
    seed ^= v + 0x9e3779b97f4a7c15ULL + (seed << 6) + (seed >> 2);
}
struct PointHash {
    size_t operator()(const Point& p) const {
        size_t s = 0;
        hash_combine(s, std::hash<int>{}(p.x));
        hash_combine(s, std::hash<int>{}(p.y));
        return s;
    }
};
```

For small fixed-width fields there is a cheaper and better answer: pack them into one integer and mix once. Two 32-bit ints become `fmix64(((uint64_t)x << 32) | (uint32_t)y)` — one shift, one or, one finalizer, no per-field hashing at all, and no cancellation because the fields never share bits.

THE KEY INSIGHT

A combiner must be order-sensitive and non-cancelling. XOR is neither. The 0x9e3779b9 constant in every hash_combine you have ever seen is $2^{32}/\varphi$ — the same golden ratio as Topic 9, doing the same job.

THE TRAP

A composite hash that ignores a field the equality operator compares. If `operator==` checks x, y and z but the hash only mixes x and y, lookups still *work* — the table just gets slower as equal-hash groups grow. The reverse (hash uses more fields than `==`) is a correctness bug: equal keys land in different buckets and the map holds duplicates. Topic 83.

```
hash_combine — ~4 ops per field · order-sensitive · pack-and-mix is ~3 ops total for fields
totalling ≤64 bits and strictly better
```

PRACTICE SET — Topics 7–15

A. Conceptual

1. Name the one obligation of a hash-table hash function that people usually add but which is not required, and say why it is not.
2. Why is "uniform" a property of a (function, key set) pair rather than of the function?
3. A mask costs 1 cycle and a prime modulus 20–40. Give the condition under which paying for the modulus is nevertheless the right call.
4. Explain, in terms of carry propagation, why multiplication mixes upward and therefore why Fibonacci hashing takes the high bits.
5. Java computes `h ^ (h >>> 16)` inside HashMap. What specific defect of `Integer.hashCode()` is that repairing?
6. What single line differs between FNV-1 and FNV-1a, and what does it buy?
7. State the universal-hashing guarantee precisely. It is not "no collisions" — what is it?
8. Give the three independent things that go wrong when you combine two field hashes with XOR.
9. Why is a hash value never a valid on-disk or on-the-wire format?
10. You are hashing 4 KB blobs at 200,000/s. Which matters more, FNV-1a vs xxHash3, or chaining vs open addressing? Justify with throughput numbers.

B. Tracing and analysis

1. Compute `djb2("ab")` by hand from h = 5381, then give its index in a table with m = 16 under masking, and again under `% 13`.
2. 1,000 keys are 64 bytes apart, starting at 0x7F0000000000, in a table with m = 1024 and mask indexing under an identity hash. How many distinct slots can possibly be reached, and what is the length of the longest bucket?
3. Perfect avalanche for a 64-bit hash changes how many output bits on average? If a measurement gives 22.3, what fraction of the ideal is that, and in which direction does it bias the table?
4. A rolling hash uses $M = 10^9+7$ over 10^6 windows. Compute the expected number of colliding pairs. Repeat for $M = 2^{61}-1$. State the ratio.
5. Under a universal family with m = 4096, you insert 3,000 keys. What is the expected number of keys colliding with a specific key x?
6. Two 32-bit fields are packed as `(x << 32) | y` and then passed to `fmix64`. Explain why (3,7) and (7,3) cannot collide, using bit positions.
7. An 8-byte string key is hashed at 1 GB/s. How many nanoseconds is that, and what percentage of a 95 ns cold lookup and of a 16 ns hot lookup?

C. Code tasks

1. **Build a distribution harness.** Write a function that takes a hash function, a key list and m, and reports longest bucket, empty slots and variance. Verify it against the ideal empty count `m·exp(−n/m)`.
2. **Break it and observe.** Feed the harness 10,000 keys of the form `"user:1000"`…`"user:10999"` using **sdbm with mask indexing**, m = 16384. Record longest bucket and empty count. Then change one thing — `% 16381` instead of the mask — and record again. Explain the entire difference without changing the hash function.

3. **Break it and observe.** Build `unordered_map<Point,int,XorHash>` with the XOR combiner, insert the 2,000 diagonal points (i, i), and time 10,000 lookups. Then swap in `hash_combine` and time again. Report both, and the ratio. Print `XorHash({5,5})` and `XorHash({9,9})` to explain the cause.
4. **Measure avalanche.** For 20,000 random 64-bit values, flip one random bit and count `popcount(h(x) ^ h(x'))` for djb2 (on the decimal string) and for `fmix64`. Report both averages against the ideal of 32.0.
5. **Break it and observe.** Take 1,000 heap pointers from `new int[16]` allocations, index them into a 1024-slot table by identity hash and mask, and print the slot histogram. Then apply `fmix64` first and print again. Report used-slot counts.
6. **Rolling hash hack.** Implement the polynomial hash with B = 131 and M = 2^{32}. Search for two distinct 64-character strings over {a,b} with equal hashes by generating Thue–Morse-style pairs or by birthday search. Then switch to M = $2^{61}-1$ and run the same search for the same wall-clock budget.

ANSWER KEY — Topics 7–15

A. Conceptual

1. **Cryptographic collision resistance.** m is small, so collisions exist by pigeonhole no matter how strong the function is. What you sometimes need is **unpredictability** — an attacker unable to construct collisions — and that comes from a secret key, not from strength.
2. Because uniformity is about how the function's outputs land *for the inputs you actually have*. sdbm is indistinguishable from FNV on random strings (max bucket 5 vs 6) and catastrophic on `"user:NNNN"` (max bucket 9, 78% of slots unused).
3. When the hash function has **weak low bits and you cannot change it** — a user-supplied `std::hash`, an identity hash on pointers. The modulus consults all 64 bits; the mask consults $\log_2 m$.
4. A multiply is a sum of shifted copies; carries only travel **leftward**. So bit 0 of the input can affect bit 63 of the product, but bit 63 can never affect bit 0. The well-mixed end is the top, so Fibonacci hashing shifts right by `64 − log₂m`.
5. `Integer.hashCode()` returns the value itself, so for keys that are multiples of a power of two the low bits are constant. `h ^ (h >>> 16)` folds the high half down into the bits the mask will read.
6. FNV-1a does `h ^= c` **before** `h *= prime`. The byte is therefore spread by the immediately following multiply instead of sitting in the low 8 bits.
7. For any two distinct keys x≠y, $\mathbf{P_{h \in H}[h(x) = h(y)] \leq 1/m}$, where the probability is over the random choice of h, not over the keys.
8. **(1)** commutativity — (3,7) and (7,3) collide; **(2)** self-cancellation — every (x,x) hashes to 0; **(3)** it inherits any structure in the field hashes, since XOR does not mix bit positions at all.
9. Because it is only guaranteed **deterministic within one process**. Seeds are randomized per process (Topic 80), and library implementations change between versions. A persisted hash becomes a wrong answer, silently, on restart.
10. At 4 KB per key, FNV-1a's ~1 GB/s means **4 µs per key**, which is 40× a whole cold lookup; xxHash3 at ~15 GB/s means **270 ns**. **The hash function dominates completely** and the collision strategy is irrelevant. This is the exception to Topic 11's trap, and it is exactly why the trap is stated as "profile first".

B. Tracing and analysis

1. h = 5381; 'a' = 97: 5381×33 + 97 = 177,573 + 97 = **177,670**. 'b' = 98: 177,670×33 + 98 = 5,863,110 + 98 = **5,863,208**. Masked with 15: 5,863,208 mod 16 = **8**. Under `% 13`: 5,863,208 = 13×451,016 + 0 → **0**.
2. Every key is a multiple of 64, so the low 6 bits are zero and the mask can only produce multiples of 64 below 1024 → **16 reachable slots**. With 1,000 keys over 16 slots, the longest bucket is **63** and 1,008 slots are empty. After `fmix64`: 658 slots used, longest bucket 5.
3. Ideal is **32.0 of 64 bits**. 22.3/32.0 = **70%** — under-mixing, meaning nearby inputs produce nearby outputs, which under masking becomes **clustering** (adjacent slots) and under linear probing becomes long runs (Topic 27).
4. $C(10^6,2) \approx 5\times10^{11}$. Divided by 10^9+7 → **~500 expected collisions**. Divided by $2^{61} \approx 2.3\times10^{18}$ → $\mathbf{\sim 2\times10^{-7}}$. Ratio ≈ $\mathbf{2.3\times10^9}$.
5. Each other key collides with x with probability ≤ 1/m = 1/4096, so the expected number is **2,999/4,096 = 0.73**.

6. x occupies bits 32–63 and y bits 0–31, so the packed words are `0x0000000300000007` and `0x0000000700000003` — different 64-bit values. `fmix64` is a **bijection** (each step is invertible), so distinct inputs give distinct outputs; they can only collide after the final mask, with probability 1/m.
7. 8 bytes at 1 GB/s = **8 ns**. That is **8.4%** of a 95 ns cold lookup and **50%** of a 16 ns hot one.

C. Code tasks — what to verify

1. For n = 10,000 and m = 16,384 the ideal empty count is $16{,}384 \times e^{-0.6104}$ = **8,898**. A good hash on random keys lands within ~1% (8,888–8,942). If your harness reports far fewer empties, it has a bug: too *few* empty slots is as suspicious as too many.
2. Expected: **sdbm + mask → longest bucket 9, 12,778 empty (78% of the table unused); sdbm + % 16381 → longest bucket 3, 7,485 empty**. The mask reads only 14 bits; sdbm's `h*65599 + c` leaves the low bits of short similar keys nearly identical, so those 14 bits barely move. The modulus folds all 64 bits in. Same hash, different projection.
3. Expected on a modern x86 core: **XOR combiner ~3,100 ns per lookup, hash_combine ~8 ns — a ~390× difference**. `XorHash({5,5})` and `XorHash({9,9})` both print **0**: all 2,000 diagonal keys share one bucket, so `find` walks a 2,000-node chain. This is Topic 5's worst case reproduced in fifteen lines, without an attacker.
4. Expect **djb2 ≈ 22.3 bits, fmix64 ≈ 32.0 bits**. Anything outside roughly 28–36 is biased.
5. Heap pointers from the same size class are typically 32 or 64 bytes apart, so expect **16–32 used slots of 1024** before mixing and **~650** after. If your allocator interleaves sizes you may see more; the shape (a comb, not a spread) is the observation that matters.
6. With M = 2^{32} a birthday search finds a colliding pair in roughly 2^{16} = **65,536 random strings** — under a second. With M = $2^{61}-1$ you would need ~$2^{30.5}$ ≈ **1.5 billion** and will find nothing in the same budget. Both outcomes are the point: the modulus, not the loop, decides.

HANDNOTE SUMMARY CARD — Hash functions

```
HASH FUNCTIONS (7-15)
================================================================================
THREE OBLIGATIONS      deterministic (within a process) | uniform for YOUR keys |
                       cheap (<30 ns).   NOT required: collision resistance.

INDEX DERIVATION -- pick exactly one line of defence
  h % prime      20-40 cycles, uses all 64 bits, needs a prime table to grow
  h & (m-1)      1 cycle, uses only log2(m) LOW bits -> demands a mixed hash
  (h*GOLDEN)>>s  3-4 cycles, uses the HIGH bits of a product   <-- best default
  GOLDEN = 11400714819323198485 = 2^64/phi

THE MIXER YOU SHOULD MEMORISE
  uint64_t fmix64(uint64_t k){            // murmur3, ~1 ns, avalanche 32.0/64
    k ^= k>>33; k *= 0xff51afd7ed558ccd;
    k ^= k>>33; k *= 0xc4ceb9fe1a85ec53;
    k ^= k>>33; return k; }

STRING HASHES
  djb2   h = h*33 + c            h0 = 5381        low bits weak for short keys
  fnv1a  h ^= c; h *= 1099511628211            h0 = 1469598103934665603   BEST
  sdbm   h = c + (h<<6) + (h<<16) - h          mask-hostile: 78% slots unused
  xxh3/wyhash  10-30 GB/s vs ~1 GB/s -- only matters for keys > ~64 bytes

MEASURED (10k keys, m=16384; ideal empty = m*e^-(n/m) = 8898)
  random keys : djb2 max5/8915  fnv max6/8942  sdbm max5/8888   all fine
  "user:NNNN" : djb2 max4/10433 fnv max5/9076  sdbm max9/12778  <-- structure!
  1000 ptrs 64B apart, m=1024: identity -> 16 slots used, max 63
                               fmix64   -> 658 slots used, max 5

ROLLING HASH (window L, base B, modulus M)
  h = (h*B + s[i]) % M;  h = (h + M*B - powBL*s[i-L]) % M;
  B=131 or 1000003, M = 2^61-1.   collisions ~ C(n,2)/M
  n=1e6: M=1e9+7 -> ~500 collisions   M=2^61-1 -> 2e-7    randomize B!

UNIVERSAL FAMILY   h(k) = ((a*k + b) mod p) mod m,  p prime, a,b random at boot
  guarantee: P[h(x)=h(y)] <= 1/m for ANY x!=y, over the choice of h.
  cheap variant: (a*k) >> (64 - log2m), a odd and random.

COMPOSITE KEYS
  XOR is WRONG:  (3,7)==(7,3), every (x,x)->0  -> measured 3117 ns vs 8 ns/lookup
  hash_combine:  seed ^= v + 0x9e3779b97f4a7c15 + (seed<<6) + (seed>>2)
  better if fields fit 64 bits: fmix64((uint64)x << 32 | (uint32)y)
  RULE: hash must use exactly the fields operator== uses. Fewer = slow.
        More = wrong (equal keys in different buckets).

TRAPS
  signed key % m            -> negative index, ASAN underflow
  multiply then MASK low    -> low bits of a product are still structured
  benchmark on random keys  -> proves nothing about production keys
  persist a hash value      -> different after restart (seed randomization, T80)
  fixed public B or seed    -> hackable (T79); randomize at process start
```

SECTION 3 — SEPARATE CHAINING AND THE CORE OPERATIONS

16. Anatomy of a chained table

Separate chaining answers Topic 1's unanswered question — what happens when two keys want slot 5 — with the simplest possible answer: the slot does not hold an entry, it holds the *head of a linked list of* entries. Collisions stop being a special case; they are just a longer list.

This is the structure behind Java's `HashMap`, C++'s `std::unordered_map`, PHP's `zend_array` and (before 1.24) Go's map. It is worth building once by hand, because every later section is a modification of it.

```
struct Node { std::string key; int val; Node* next; };  // sizeof = 48

struct ChainMap {
    std::vector<Node*> buckets;      // m slots, each a list head or nullptr
    size_t n = 0;                    // live entries

    explicit ChainMap(size_t m = 8) : buckets(m, nullptr) {}

    static uint64_t hash(const std::string& k) {
        uint64_t h = 14695981039346656037ULL;         // FNV-1a ...
        for (unsigned char c : k) { h ^= c; h *= 1099511628211ULL; }
        h ^= h >> 33; h *= 0xff51afd7ed558ccdULL;    // ... + fmix64 (T13)
        h ^= h >> 33;
        return h;
    }
    size_t idx(const std::string& k) const {
        return hash(k) & (buckets.size() - 1);        // power-of-two mask
    }
};
```

```
buckets (vector<Node*>, m = 8)
 +----+
 |  0 | -> null
 |  1 | -> ["apple" |1|*] -> null
 |  2 | -> ["lemon" |9|*] -> ["grape"|7|*] -> ["cherry"|3|*] -> null
 |  3 | -> ["mango" |4|*] -> null            ^ newest at the head
 |  4 | -> null                                (O(1) insert, no tail walk)
 |  5 | -> ["elder" |5|*] -> null
 |  6 | -> null
 |  7 | -> ["kiwi"  |8|*] -> ["fig"  |6|*] -> null
 +----+
 one contiguous array of pointers + n scattered heap nodes
```

THE KEY INSIGHT

A chained hash table is an array of linked lists where the hash function decides which list. Everything you know about linked lists — head insertion is O(1), traversal is a dependent load chain, nodes are cache-hostile — applies verbatim to a bucket.

THE TRAP

`buckets.size()` must be a power of two for the mask to be correct. Construct with `ChainMap(10)` and `& 9` produces indices 0,1,8,9 only — 4 usable slots out of 10, silently. Assert it in the constructor.

```
chained table — memory: m×8 B array + n×48 B nodes · one allocation per insert · the array is
contiguous, the nodes are not
```

17. Insert: update-or-append, and duplicate-key semantics

Insert has to answer a question the ADT hides: what if the key is already there? A *map* updates in place; a *multimap* appends. Getting this wrong produces a table that contains the same key twice, where lookups return the first copy and deletes remove only one of them.

```
void insert(const std::string& k, int v) {
    if (int* p = find(k)) { *p = v; return; }        // update-in-place
    size_t i = idx(k);
    buckets[i] = new Node{k, v, buckets[i]};         // head insertion
    if (++n > buckets.size() * 3 / 4) rehash(buckets.size() * 2);
}
```

Three decisions are visible in five lines. **Search before insert** — without it the map silently becomes a multimap. **Head insertion** — O(1), and it puts the most recently inserted key first, which helps temporal-locality workloads and hurts nothing. **Grow after insert** — checking the load factor before inserting would let the table sit one entry over threshold.

```
insert("grape") into bucket 2 which already holds ["cherry"]

 before:  buckets[2] -> ["cherry"] -> null
 step 1:  new Node{"grape", 7, buckets[2]}     next points at old head
 step 2:  buckets[2] = newNode
 after:   buckets[2] -> ["grape"] -> ["cherry"] -> null
          two pointer writes; no traversal of the existing chain
```

THE KEY INSIGHT

Insert is a find followed by a prepend. The find is not overhead — it is the step that makes the container a map rather than a bag, and it is why insert and lookup have the same asymptotic cost.

THE TRAP

`operator[]` in C++ and Java *inserts* a default-constructed value on a miss. `if (m[k] == 0)` on a map with 1M keys grows it to 2M entries and doubles its memory, while looking like a read. Symptom: memory climbing on a read-only code path. Use `find`/`count`/`contains`.

```
insert — Time O(1 + α) expected, O(n) worst · 1 allocation · amortized 1.23 extra node moves per
insert from growth (Topic 22)
```

18. Lookup: hash, index, walk, compare

```
int* find(const std::string& k) {
    for (Node* c = buckets[idx(k)]; c; c = c->next)
        if (c->key == k) return &c->val;
    return nullptr;
}
```

Four costs, and they are not equal. The hash is ~10–20 ns of computation with no memory access. The bucket load is one cache miss (~80 ns cold). Each chain step is *another* dependent cache miss — the CPU cannot prefetch `c->next` because it does not know the address until the previous load lands. The key comparison may be a third miss if the string data is out of line.

Measured on the 10,000-key table built in this section: α = 0.61, longest bucket 6, 8,885 of 16,384 slots empty. A hit therefore averages **1 + α/2 ≈ 1.31 nodes** visited — but the tail of the distribution is what hurts, and the worst key in that table costs 6.

```
find("grape")
  hash("grape")            15 ns, no memory touched
  buckets[2]               ~80 ns cold (array is contiguous, but 128 KB)
  node->key == "grape"?    ~80 ns cold (heap node, unpredictable address)
     miss -> node->next    ~80 ns cold (DEPENDENT: cannot prefetch)
  ---------------------------------------------------------------
  1-node chain  ~ 175 ns cold / ~18 ns hot
  3-node chain  ~ 335 ns cold  -- chain length is a latency multiplier
```

THE KEY INSIGHT

Chain length does not cost "a few more comparisons". It costs a *serialised* cache miss each, which is why open addressing (Section 4) wins despite doing more comparisons: its extra probes are on the same cache line.

THE TRAP

Comparing hashes instead of keys — `if (c->hash == h) return &c->val;` — because "collisions are rare". They are not rare; they are the reason the chain exists. Symptom: a lookup that returns another key's value roughly once per 2^{32} operations under a 32-bit hash. Store the hash to *skip* comparisons, then still compare.

```
lookup — O(1 + α) expected · 1 hash + 1 + α/2 dependent loads · measured 1.31 nodes average, 6
worst at α = 0.61
```

19. Delete: unlink, and what the chain looks like after

Deletion in a chained table is genuinely easy, and it is the one place chaining beats open addressing outright (compare Topic 30, where deletion needs tombstones). The pointer-to-pointer idiom removes the head/middle special case.

```
bool erase(const std::string& k) {
    size_t i = idx(k);
    for (Node** pp = &buckets[i]; *pp; pp = &(*pp)->next)
        if ((*pp)->key == k) {
            Node* dead = *pp;
            *pp = dead->next;      // works whether pp is the bucket
            delete dead;           // slot or a previous node's next
            --n;
            return true;
        }
    return false;
}
```

```
erase("grape") from  buckets[2] -> [grape] -> [cherry] -> null

 pp = &buckets[2]      *pp == [grape]   match
 *pp = [grape].next    buckets[2] -----------> [cherry] -> null
 delete [grape]        node freed; no slot is left "half occupied"
 n -= 1                the table is exactly as if grape never existed
```

THE KEY INSIGHT

`Node** pp` holds the address of the pointer that must change, so the head case and the middle case are literally the same code. This is the same trick as in-place linked list deletion; nothing here is hash-specific.

THE TRAP

Erasing while iterating. `for (auto& kv : m) if (pred(kv)) m.erase(kv.first);` invalidates the iterator and the next `++` reads freed memory. Symptom: intermittent crashes or silently skipped elements; under ASAN, `heap-use-after-free`. The fix is `it = m.erase(it)`. Topic 86.

```
erase — O(1 + α) expected · 1 pointer write + 1 free · leaves no residue, unlike a tombstone ·
does not shrink the table (Topic 23)
```

20. Iteration: O(m + n), and why order is not guaranteed

To visit every entry you must walk all m slots, most of which are empty, and chase each non-empty chain. That is **O(m + n)**, not O(n) — and at α = 0.1 the empty-slot scan dominates by 10:1.

```
for (size_t i = 0; i < buckets.size(); ++i)
    for (Node* c = buckets[i]; c; c = c->next)
        visit(c->key, c->val);
// 16,384 slot reads + 10,000 node visits for a 10,000-entry table.
// 8,885 of those slot reads found nullptr and did nothing.
```

The order you observe is `(bucket index, position in chain)`. It is deterministic for a given hash seed, table size and insertion sequence — and *all three* of those change: the seed per process (Topic 80), the size on growth (Topic 22), the sequence with your data. Nothing about it is stable, and no standard promises it is.

```
insert order:  A B C D E   (all into m = 4)
iteration:     C A E   D   B     <- by bucket, then by chain (newest first)
after one growth to m = 8:
iteration:     A D   B E   C     <- completely different, same contents
```

THE KEY INSIGHT

Iteration cost is driven by m, and lookup cost by α. They pull in opposite directions: a table sized generously for fast lookups is a table that is slow to iterate.

THE TRAP

Building a table with `reserve(10'000'000)`, filling it with 500 entries, and then iterating it every tick. Symptom: a loop that reads 10M pointers to touch 500 objects — 80 MB of memory traffic per pass, at ~5 ms, while the profiler shows the function "doing nothing".

```
iteration — Time O(m + n) · at α = 0.61, 16,384 slot reads for 10,000 entries · order is
unspecified and unstable across runs
```

21. Load factor and expected chain length

Under simple uniform hashing, chain lengths follow a Poisson distribution with mean α. Three numbers follow, and they are the only formulas you need for chaining:

Quantity	Formula	At α = 0.75
Expected chain length	α	0.75
Probes for an unsuccessful search	1 + α	1.75
Probes for a successful search	1 + α/2	1.38
Fraction of slots empty	$e^{-\alpha}$	47%
Expected longest chain (n = m)	Θ(log n / log log n)	~6 at n = 10^4

```
// Measured on the ChainMap of this section, 10,000 "user:NNNN" keys:
//   n = 10000  m = 16384  alpha = 0.610
//   longest chain = 6          (predicted ~6)
//   empty slots   = 8885       (predicted m*e^-0.61 = 8898, error 0.15%)
```

Notice what α does *not* control: the worst case. Even at α = 0.61 some key costs six probes. Java's HashMap converts a bucket to a red-black tree once it holds 8 entries precisely because the Poisson tail — P(length ≥ 8) ≈ 6×10^{-8} at α = 0.75 — is negligible by accident and inevitable under attack.

THE KEY INSIGHT

At any α ≤ 1 chaining costs under 1.5 probes on average. Chaining does not degrade gracefully with α — it barely degrades at all. That is its one great advantage over open addressing, whose cost explodes past 0.9 (Topic 27).

THE TRAP

Raising the load factor to save memory. Going from 0.75 to 2.0 saves 62% of the *slot array* — which is 8 bytes per slot against 48 bytes per node, so about 11% of total memory — and raises successful-search probes from 1.38 to 2.0. You paid 45% more probes for 11% of memory.

```
chaining at α: hit 1 + α/2 · miss 1 + α · empty fraction e^−α · default threshold 0.75 in Java, 1.0 in libstdc++
```

22. Resize: doubling, full rehash, amortized O(1)

When α crosses the threshold, allocate a new array of 2m slots and reinsert every entry. Every index changes, because the mask has one more bit. Under chaining you can move the *nodes* rather than reallocate them, which is why the loop below never touches the heap.

```
void rehash(size_t m2) {
    std::vector<Node*> nb(m2, nullptr);
    for (Node* head : buckets)
        for (Node* c = head; c; ) {
            Node* next = c->next;                    // save before relink
            size_t j = hash(c->key) & (m2 - 1);      // recompute index
            c->next = nb[j]; nb[j] = c;              // splice into new bucket
            c = next;
        }
    buckets.swap(nb);
}
```

The amortized argument. Growing from 8 slots to 16,384 while inserting 10,000 keys performs 11 rehashes and moves **12,282 nodes in total — 1.23 per insert**. The geometric series is why: 6 + 12 + 24 + … + 6144 < 2×n. Each insert pays O(1) into a savings account; the rehash spends it. Doubling is not an optimisation, it is the difference between O(1) and O(n) amortized: growing by a *constant* 1,000 slots would make total work O(n²/1000).

```
m:      8   16   32   64 ... 8192  16384      total nodes moved
moved:  6   12   24   48 ... 6144  (12282 cumulative)
                                    = 1.23 per insert over 10,000 inserts
cost of the LAST rehash alone: 12,288 nodes rehashed in one call
        ~ 0.6 ms  <- one insert in ten thousand is 40,000x slower
```

THE KEY INSIGHT

Doubling makes the *total* cost linear by making each rehash pay for all the inserts since the previous one. It does nothing whatsoever for the individual insert that triggers it.

THE TRAP

The rehash of a 4M-entry map takes 30–50 ms and stalls the thread. Symptom: a p99 latency graph with periodic spikes at powers of two in traffic volume, while p50 is flat. Fix: `reserve()` up front (Topic 56) or rehash incrementally (Topic 24). Full treatment in Topic 88.

```
rehash — Time O(n + m) for one call, O(1) amortized per insert · measured 1.23 node moves per insert · one call at n = 4M stalls ~40 ms
```

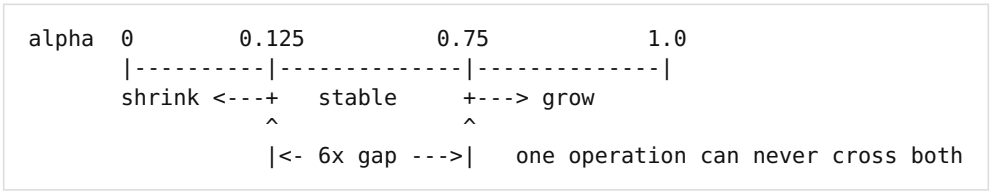
23. Shrinking, hysteresis, and the resize-thrash bug

Most standard hash tables **never shrink**. Insert 10M entries into a Java HashMap or a C++ `unordered_map`, erase 9,999,999 of them, and the slot array still occupies 16M slots. This is a deliberate choice, and it is the right one by default: it makes erase O(1) with no amortized surprise, and the memory is usually about to be reused.

If you do implement shrinking, the single rule is **hysteresis**: the shrink threshold must be well below half the grow threshold. Otherwise a workload that oscillates around the boundary rehashes on every single operation.

```
// WRONG: grow at 0.75, shrink at 0.75/2 = 0.375 exactly.
// Insert crosses to 0.76 -> grow to 2m -> alpha halves to 0.38
// Erase one -> alpha 0.374 -> shrink to m -> alpha 0.748
// Insert one -> grow again. Every op is now O(n).

// RIGHT: separate the thresholds by a factor of at least 4.
if (n > m * 3 / 4)      rehash(m * 2);     // grow  at alpha > 0.75
else if (n < m / 8 && m > 8) rehash(m / 2);  // shrink at alpha < 0.125
```



THE KEY INSIGHT

Any resize policy with a single threshold oscillates. Two thresholds with a gap wider than the resize's own effect on α is the general fix — the same reasoning as a Schmitt trigger or a thermostat's deadband.

THE TRAP

A long-lived cache that peaked at 10M entries and now holds 1,000 still holds its 128 MB slot array forever. Symptom: RSS that never returns after a traffic spike, with the heap profiler pointing at a container that reports `size() == 1000`. Fix: rebuild into a fresh map, or call `rehash(0)` in libstdc++, which shrink-to-fits.

```
shrinking — not done by default in Java, C++, Go, Python (dict shrinks only on rebuild) ·
requires ≥4× hysteresis · costs the same O(n + m) as growth
```

24. Incremental rehashing: trading complexity for p99

Redis cannot afford Topic 22's 40 ms stall on a single-threaded server, so it does not take it. It keeps **two tables**, ht[0] and ht[1], and moves one bucket per operation until ht[0] is empty. Every operation pays a tiny constant instead of one operation paying everything.

```
// Redis dict.c, in outline.
struct Dict { Bucket *ht[2]; long rehashidx; };   // -1 when not rehashing

void* dictFind(Dict* d, Key k) {
    if (d->rehashidx != -1) dictRehashStep(d);    // move ONE bucket
    for (int t = 0; t < 2; ++t) {                 // search BOTH tables
        void* v = lookupIn(d->ht[t], k);
        if (v) return v;
        if (d->rehashidx == -1) break;            // ht[1] not in use
    }
    return NULL;
}
// Insert always goes into ht[1] while rehashing, so ht[0] only shrinks.
```

```
     ht[0] (old, m = 8)          ht[1] (new, m = 16)
idx  0 1 2 3 4 5 6 7             0 1 2 ... 15
     x x x . . . . .   --->      . . x       rehashidx = 2
     ^^^^^ migrated               every op migrates one more bucket
lookup must check ht[0] AND ht[1] until rehashidx reaches m
```

THE KEY INSIGHT

Incremental rehashing does not reduce total work — it is still O(n + m). It redistributes *when* the work happens, converting one 40 ms stall into n operations that are each ~1% slower. That is a p99 optimisation, not a throughput one.

THE TRAP

Every read path must now check two tables, and every iterator must handle a key moving mid-scan. Redis solves the second with the reverse-binary cursor of `SCAN`, which guarantees elements present throughout are returned at least once but may return others twice. Symptom of getting it wrong: a key that exists disappears for the duration of a rehash. Topic 55.

```
incremental rehash — same O(n + m) total · worst single operation O(bucket) instead of O(n) ·
costs: two tables live, doubled lookup path, weaker iteration guarantees
```

25. Memory arithmetic: bytes per entry

Say the number, not "a lot". For `ChainMap` with `std::string` keys on 64-bit Linux:

Component	Bytes	Notes
`sizeof(Node)`	48	string 32 + int 4 + pad 4 + next 8
Allocator header/rounding per node	+16	glibc malloc rounds to 16 B and adds 8–16
Slot array share at α = 0.61	13	8 B pointer ÷ 0.61
Heap-allocated string data (>15 chars)	+32	zero for SSO keys of ≤15 chars
Total per entry	**~77–109**	to store a 9-byte key and a 4-byte value

```
// 10,000 entries in the ChainMap of this section:
//   nodes:  10,000 x 48 = 480 KB   (+ ~160 KB allocator overhead)
//   slots:  16,384 x  8 = 131 KB
//   total:  ~770 KB to hold 130 KB of key and value bytes -- 5.9x
//
// The flat open-addressed equivalent (Section 4), 16 B/slot at alpha 0.61:
//   16,384 x 16 = 262 KB, one allocation, zero pointer chasing.
```

THE KEY INSIGHT

Chaining's memory cost is dominated by *per-node* overhead (pointer + allocator header = 24 B), not by the empty slots everyone worries about. Open addressing wins on memory precisely by deleting the node.

THE TRAP

Estimating a map's footprint as `n × (sizeof(K) + sizeof(V))`. For `unordered_map<int,int>` that predicts 8 B/entry; the real figure is 32–48 B, a 4–6× miss. Symptom: an OOM at roughly one-fifth of the capacity you planned for.

```
chained map — ~77–109 B per string entry, ~32–48 B per int/int entry · 1 allocation per insert · 5.9× the payload in the measured case
```

PRACTICE SET — Topics 16–25

A. Conceptual

1. Why does insert call find first, and what does the container become if you delete that line?
2. Head insertion into a bucket is O(1). Name one behavioural consequence of inserting at the head rather than the tail.
3. Explain why chain traversal is more expensive than the same number of comparisons in an array, in terms of what the prefetcher can and cannot do.
4. Give the two reasons chained deletion is simpler than open-addressed deletion.
5. Iteration is O(m + n). Construct a realistic case where the m term is 99% of the cost.
6. State the successful- and unsuccessful-search probe formulas for chaining and explain why the successful one is smaller.
7. Why is doubling, rather than adding a fixed 1,024 slots, necessary for amortized O(1)?
8. What is hysteresis, and what specifically goes wrong without it?
9. Incremental rehashing does not reduce total work. What does it improve, and what two costs does it add to every read?
10. Why is per-entry memory dominated by node overhead rather than by empty slots?

B. Tracing and analysis

1. A chained table has m = 16 and holds the keys with indices `3,3,3,7,9,9,14`. Draw the buckets. Give α, the longest chain, the number of empty slots, and the average probes for a successful search computed two ways: from the formula 1 + α/2, and by counting positions directly.
2. Starting at m = 8 with a 0.75 threshold, list every table size reached while inserting 10,000 keys, and compute the total number of node relinks. Show that it is under 2n.
3. At α = 0.61 with m = 16,384, predict the number of empty slots using $m \cdot e^{-\alpha}$, then compare with the measured 8,885. Give the percentage error.
4. Compute total memory for 1,000,000 entries of `Node{std::string(9 chars), int, Node*}` at α = 0.75, including 16 bytes of allocator overhead per node. Then compute the payload bytes and the ratio.
5. A shrink threshold is set at α < 0.375 with a grow threshold of α > 0.75. Starting at n = m×0.75 + 1, trace six operations (insert, erase, insert, erase, ...) and show that each one resizes.
6. Iterating a table with m = 10,000,000 and n = 500 reads how many pointers? At 8 bytes each and 10 GB/s, how long does one pass take?
7. Under Redis-style incremental rehashing with m = 1,024 growing to 2,048, how many operations must occur before ht[0] is empty, and what is the extra work per operation?

C. Code tasks

1. **Build.** Implement `ChainMap` from Topics 16–22 complete with `find`, `insert`, `erase`, `rehash` and a `stats()` that prints n, m, α, longest chain and empty slots. Load it with 10,000 keys `"user:1000"`…`"user:10999"` and check your numbers against α = 0.610, longest 6, empty 8,885.
2. **Break it and observe.** Delete the `if (int* p = find(k))` line from `insert`. Insert `("a",1)` then `("a",2)`. Print `n`, print `*find("a")`, then `erase("a")` once and print `*find("a")` again. Write down all three observations before deciding what kind of container you now have.

3. **Break it and observe.** Construct `ChainMap(10)` — a non-power-of-two capacity — insert 1,000 keys and print the slot histogram before any rehash. How many of the ten slots are reachable? Derive the answer from `10 - 1 = 0b1001` before you run it, then add the assert that should have been there.
4. **Measure the spike.** Time every individual insert of 4,000,000 into `std::unordered_map<int,int>` and print any insert taking over 1 ms, with its index. Plot or list them. Report the worst, and explain why the indices are not evenly spaced.
5. **Break it and observe.** Implement shrinking with the *single*-threshold policy from Topic 23 (grow above 0.75, shrink below 0.375). Drive the map to exactly the boundary and then alternate insert and erase 1,000 times, counting `rehash` calls. Then widen the shrink threshold to 0.125 and repeat the same 1,000 operations.
6. **Break it and observe.** Erase from a `std::unordered_map` inside a range-`for` over it. Compile with `-fsanitize=address -g` and run. Record the sanitizer's error class and the operation it names. Then rewrite with `it = m.erase(it)`.

ANSWER KEY — Topics 16–25

A. Conceptual

1. To implement **update-in-place** semantics. Without it the structure becomes a **multimap**: the key appears twice, find returns whichever copy is nearer the head, and erase removes one of them.
2. The most recently inserted key is found **first**. That helps recency-skewed workloads and makes iteration order depend on insertion order (Topic 20). Tail insertion would require either a tail pointer or an O(chain) walk.
3. Each `c->next` is a **dependent load**: the address is not known until the previous load completes, so the hardware prefetcher cannot issue it early and the misses **serialise** (~80 ns each). Array elements have computable addresses and prefetch in parallel.
4. **(a)** Removing a node cannot break another key's search path, because each bucket is an independent list; **(b)** no marker has to be left behind, so no tombstone accumulates (contrast Topic 30).
5. A table with `reserve(10'000'000)` holding 500 entries: m = 16.7M slot reads versus 500 node visits — the m term is **99.997%**.
6. Miss = **1 + α** (you scan the whole chain); hit = **1 + α/2** (you stop halfway through on average). The hit is cheaper because success terminates early.
7. Doubling makes the sizes geometric, so total relinks 6 + 12 + … + m < 2n = **O(n)**. A fixed increment c makes the number of rehashes n/c, each costing O(n), so total work is **O(n²/c)** — still quadratic.
8. **A gap between the grow and shrink thresholds** wide enough that a single resize cannot land on the other side. Without it the table resizes on **every operation**, turning O(1) into O(n).
9. It improves the **worst individual operation** (p99), not throughput. Costs: every lookup must search **two tables**, and iteration guarantees weaken to "present throughout ⇒ returned at least once", which is why Redis `SCAN` may return duplicates.
10. Because the per-node bill is **a 8 B next-pointer plus 8–16 B of allocator header, rounded to 16 B** — 24 B or more — while an empty slot costs a single **8 B pointer**, and there are only $(1-e^{-\alpha})^{-1}$ as many slots as entries.

B. Tracing and analysis

1. Bucket 3 holds 3 keys, bucket 7 holds 1, bucket 9 holds 2, bucket 14 holds 1; n = 7, m = 16, α = **0.4375**, longest chain **3**, empty slots **12**. Formula: 1 + 0.4375/2 = **1.22**. Direct count: positions are (1,2,3) + (1) + (1,2) + (1) = 11 probes over 7 keys = **1.57**. The formula assumes uniformity; this tiny sample is clumped, which is exactly the variance Topic 5 warns about.
2. Sizes: 8, 16, 32, 64, 128, 256, 512, 1024, 2048, 4096, 8192, **16384** — 11 rehashes. Relinks: 6 + 12 + 24 + 48 + 96 + 192 + 384 + 768 + 1536 + 3072 + 6144 = **12,282**, which is **1.23 per insert** and comfortably under 2n = 20,000.
3. $16{,}384 \times e^{-0.610} = 16{,}384 \times 0.5434 =$ **8,903**. Measured 8,885. Error = 18/8,903 = **0.20%**.
4. m = 1,000,000/0.75 rounded up to a power of two = **2,097,152 slots**. Slots: 2,097,152 × 8 = **16.8 MB**. Nodes: 1,000,000 × (48 + 16) = **64 MB**. Total **80.8 MB**. Payload: 9 + 4 = 13 B × 10^6 = **13 MB**. Ratio = **6.2×**.

5. Insert → α = 0.751 → grow to 2m → α = 0.376. Erase → α = 0.3759... one more erase → below 0.375 → shrink to m → α = 0.749. Insert → 0.751 → grow. **Every second operation triggers an O(n) rehash**; the map does O(n) work per operation indefinitely.
6. **10,000,000 pointer reads** = 80 MB. At 10 GB/s that is **8 ms** per pass, to visit 500 objects.
7. **1,024 operations** (one bucket migrated each). Extra work per operation is **one bucket's worth of relinks — α nodes on average, so under one node** — plus a second table lookup on every read.

C. Code tasks — what to verify

1. Expect exactly **n = 10,000, m = 16,384, α = 0.610, longest chain 6, empty 8,885**. If your longest chain is much larger, you dropped the `fmix64` step and are masking raw FNV bits (Topic 10).
2. Expect **n = 2**, `*find("a")` **= 2** (the newest copy is at the head), and after one `erase`, `*find("a")` **= 1 — the key is still there**. You have built a multimap with map-shaped code. In production this shows up as "I deleted it and it came back".
3. `10 - 1 = 0b1001`, so `h & 9` can only produce **0, 1, 8, 9 — four reachable slots**. Six of ten slots stay empty forever and chains are ~2.5× longer than they should be. The assert is `assert((m & (m - 1)) == 0)`.
4. Typical result on x86-64 with libstdc++: **about 12 inserts exceed 1 ms out of 4,000,000**, and the worst is **≈ 330 ms** — a single insert 40,000× slower than its neighbours. The indices are not evenly spaced because libstdc++ grows through a **prime table** (the final `bucket_count()` is 5,967,347, a prime), so successive sizes are roughly, not exactly, doubling.
5. With the narrow gap expect **~1,000 rehash calls in 1,000 operations** — one per operation, each O(n). With the shrink threshold at 0.125 expect **0**. Same code, same workload; only the deadband changed.
6. Expect `ERROR: AddressSanitizer: heap-use-after-free`, with the READ attributed to `operator++` on the iterator and the "freed by thread T0 here" stack pointing at `unordered_map::erase`. The range-`for` desugars to `++it` on an iterator whose node was just deleted. `it = m.erase(it)` returns the next valid iterator; note that `erase` invalidates only the erased element's iterator in `unordered_map`, unlike `vector`.

HANDNOTE SUMMARY CARD — Chaining

```
SEPARATE CHAINING (16-25)
================================================================================
STRUCTURE   vector<Node*> buckets(m);  struct Node{K key; V val; Node* next;};
            index = hash(k) & (m-1)     <- m MUST be a power of two for the mask

THE FOUR OPERATIONS (memorise these shapes)
  find    for (Node* c = buckets[idx(k)]; c; c = c->next)
              if (c->key == k) return &c->val;
          return nullptr;
  insert  if (V* p = find(k)) { *p = v; return; }      // update-or-append
          buckets[i] = new Node{k, v, buckets[i]};     // head insert
          if (++n > m*3/4) rehash(2*m);
  erase   for (Node** pp = &buckets[i]; *pp; pp = &(*pp)->next)
              if ((*pp)->key == k) { Node* d=*pp; *pp=d->next; delete d; --n; }
  rehash  for each node: save next, recompute j, splice into nb[j], swap

FORMULAS (alpha = n/m)
  hit        1 + alpha/2        alpha=0.75 -> 1.38 probes
  miss       1 + alpha          alpha=0.75 -> 1.75 probes
  empty      m * e^-alpha       alpha=0.61 -> 8903 predicted / 8885 measured
  longest    Theta(log n / log log n)   n=10^4 -> ~6
  iteration  O(m + n)           NOT O(n)

GROWTH ARITHMETIC (8 -> 16384 while inserting 10,000 keys)
  11 rehashes, 12,282 node relinks = 1.23 per insert, total < 2n  -> O(1) amort.
  BUT the last rehash moves 12,288 nodes in ONE call.
  measured worst single insert into unordered_map<int,int> at n=2.9M: 330 ms
  (12 inserts of 4,000,000 exceeded 1 ms; p50 was ~100 ns)

SHRINK POLICY        grow at alpha > 0.75, shrink at alpha < 0.125  (>=4x gap)
  single threshold (0.75 / 0.375) => resize on EVERY operation, O(n) per op
  Java/C++/Go never shrink automatically: RSS never returns after a spike

INCREMENTAL REHASH (Redis)   ht[0] + ht[1], rehashidx, one bucket per op
  same total work; worst op O(bucket) not O(n); reads must check BOTH tables;
  iteration guarantee weakens to "at least once, maybe twice" (SCAN)

MEMORY, string key of 9 chars + int value
  Node 48 B + allocator 16 B + slot share 13 B = ~77 B for 13 B of payload
  1M entries => 80.8 MB (16.8 slots + 64 nodes) = 6.2x payload
  unordered_map<int,int> is 32-48 B/entry, NOT 8

TRAPS
  m not a power of two + mask  -> only 4 of 10 slots reachable
  no find-before-insert        -> silent multimap; "I deleted it, it came back"
  operator[] on a read path    -> inserts a default value, doubles the map
  compare hashes not keys      -> wrong value returned ~1 in 2^32 ops
  erase inside range-for       -> ASAN heap-use-after-free; use it = m.erase(it)
  reserve() huge, fill tiny    -> iteration reads 10M pointers for 500 objects
```

SECTION 4 — OPEN ADDRESSING

26. One array, no nodes: slot states and probe sequences

Open addressing deletes the linked list. Every entry lives in the slot array itself, and a collision is resolved by *looking somewhere else in the same array* according to a deterministic probe sequence. No node allocation, no next pointer, no pointer chasing — the entire table is one contiguous allocation.

The consequence is that α can never exceed 1, and that a slot now needs three states rather than two.

```
enum State : uint8_t { EMPTY = 0, FULL = 1, TOMB = 2 };  // Topic 30

struct FlatMap {
    std::vector<uint64_t> keys;
    std::vector<int>      vals;
    std::vector<State>    state;    // parallel arrays: SoA (Topic 37)
    size_t m, n = 0;

    size_t probe(size_t h, size_t i) const { return (h + i) & (m - 1); }

    int find(uint64_t k) const {
        size_t h = mix(k) & (m - 1);
        for (size_t i = 0; i < m; ++i) {
            size_t s = probe(h, i);
            if (state[s] == EMPTY) return -1;      // EMPTY terminates
            if (state[s] == FULL && keys[s] == k) return vals[s];
        }                                          // TOMB: keep going
        return -1;
    }
};
```

```
chaining                        open addressing
 [0] -> node -> node             [0][1][2][3][4][5][6][7]
 [1] -> node                      E  F  F  T  E  F  E  F
 n heap allocations              one allocation, ever
 alpha may exceed 1              alpha < 1 by construction
 delete = unlink                 delete = leave a TOMB or shift back
```

THE KEY INSIGHT

Under chaining the bucket *is* the collision list. Under open addressing the collision list is spread across the table and reconstructed by replaying the probe sequence — which is why `EMPTY` must terminate a search and `TOMB` must not.

THE TRAP

A `find` loop that stops on a tombstone. Symptom: keys inserted before a deletion become unfindable, and the map's `size()` stops matching what iteration produces. The mirror bug — an insert that does not reuse tombstones — fills the table with garbage instead.

```
open addressing — 1 allocation total · 0 pointer chasing · α < 1 mandatory · deletion becomes the hard operation
```

27. Linear probing and primary clustering

`probe(h, i) = (h + i) mod m`. The simplest sequence, and the fastest per probe: consecutive slots are on the same cache line, so probes 2 through 8 are usually free. The weakness is **primary clustering**: any key hashing anywhere into an existing run joins that run and extends it, so runs grow superlinearly and merge with their neighbours.

Knuth's analysis gives the two formulas worth memorising, and a 65,536-slot simulation matches them closely:

α	Hit: ½(1 + 1/(1−α))	Measured hit	Miss: ½(1 + 1/(1−α)²)	Measured miss	Worst probe
0.50	1.50	1.50	2.50	2.50	22
0.75	2.50	2.49	8.50	8.48	84
0.90	5.50	5.33	50.50	48.94	498
0.95	10.50	10.32	200.50	195.29	1,608

```
a run of 5 forms; every one of the 5 home slots now feeds it
 idx  10 11 12 13 14 15 16
      F  F  F  F  F  E  E
      ^-----------^
 a new key hashing to ANY of 10..14 lands at 15, making the run 6 long
 P(joining a run of length L) is proportional to L  -> rich get richer
```

THE KEY INSIGHT

The miss formula squares the 1/(1−α) term. That is the whole story of open addressing: at α = 0.95 a hit costs 10 probes and a miss costs 195. Since inserts and negative lookups are misses, the cliff is on the path you use most.

THE TRAP

Sizing a linear-probing table by memory rather than by α. Going from α = 0.75 to 0.95 saves 21% of the array and multiplies miss cost by **23×**. Symptom: a table that is fine in staging with 60% occupancy and collapses in production at 92%.

```
linear probing — hit ½(1+1/(1−α)) · miss ½(1+1/(1−α)²) · probes are cache-adjacent (~1 ns each after the first) · keep α ≤ 0.75
```

28. Quadratic probing and secondary clustering

Make the step grow with the attempt number and keys with the same home slot still collide, but keys with *different* home slots stop merging into one run. That kills primary clustering and leaves only **secondary clustering** — the shared fate of keys with identical h(k).

```
// Triangular numbers: step i lands at h + i(i+1)/2.
// For m a power of two this sequence visits EVERY slot exactly once,
// so an insert can never fail while a slot is free -- unlike the naive
// h + i*i, which visits only about half the slots.
size_t probe(size_t h, size_t i) const {
    return (h + (i * i + i) / 2) & (m - 1);
}
```

α	Quadratic hit	Quadratic miss	vs linear miss
0.50	1.44	2.15	2.50 → 14% better
0.75	1.99	4.64	8.48 → 45% better
0.90	2.87	12.24	48.94 → 75% better
0.95	3.62	25.12	195.29 → 87% better

```
linear    h, h+1, h+2, h+3, h+4      contiguous -> 1 cache line, but merges runs
quadratic h, h+1, h+3, h+6, h+10   scattered  -> a cache miss per probe past
                                                  the first, but no merging
```

THE KEY INSIGHT

Quadratic probing trades *locality* for *independence*. It wins on probe count and loses on nanoseconds per probe; at $\alpha \leq 0.75$, where linear needs only 2.5 probes that are nearly free, linear is usually faster in wall-clock terms despite the worse table.

THE TRAP

`h + i*i` with power-of-two m visits only (m+1)/2 distinct slots. Symptom: insert loops forever, or reports "table full", at $\alpha \approx 0.5$ with half the slots visibly empty. Use the triangular form, or a prime m with $\alpha < 0.5$.

```
quadratic probing — miss 45–87% cheaper than linear above α = 0.75 · each probe is a fresh cache
line · needs triangular steps or a prime m for completeness
```

29. Double hashing

Give every key its own step size: `probe(h, i) = h + i·h₂(k)`, with h_2 forced odd so it is coprime with a power-of-two m and the sequence therefore covers all slots. Now even two keys with the same home slot follow different paths, so secondary clustering disappears too.

```
size_t probe(uint64_t k, size_t h, size_t i) const {
    size_t h2 = mix(k * 0x9E3779B97F4A7C15ULL) | 1;   // ALWAYS odd
    return (h + i * h2) & (m - 1);
}
```

α	Double hit	Double miss	Quadratic miss
0.50	1.45	2.19	2.15
0.75	2.02	4.67	4.64
0.90	2.86	11.43	12.24
0.95	3.55	21.89	25.12

Double hashing is theoretically the best of the three — its behaviour matches "uniform hashing", the idealised model where each key's probe sequence is a random permutation, giving miss cost $1/(1-\alpha)$. In practice the measured gap over quadratic is **under 5% below α = 0.9**, and it costs a second hash computation per lookup. It is the right answer when α must run high and the keys are cheap to hash.

THE KEY INSIGHT

The three schemes form a ladder of independence: linear shares runs between different home slots, quadratic shares them only within one home slot, double hashing shares nothing. Each rung costs cache locality.

THE TRAP

Forgetting `| 1`. If $h_2(k)$ is even and m is a power of two, the sequence visits only m/gcd slots; if $h_2(k)$ is 0 it visits *one* slot forever. Symptom: an infinite loop inside insert, on one key in a million.

```
double hashing — approaches uniform hashing, miss ≈ 1/(1−α) · 2 hash computations · no locality · <5% better than quadratic below α = 0.9
```

30. The deletion problem: tombstones

Under chaining, delete unlinks and the table is exactly as if the key never existed (Topic 19). Under open addressing you cannot simply set the slot to EMPTY: the deleted slot may be in the middle of another key's probe sequence, and an EMPTY terminates a search. Marking it EMPTY makes every key behind it *unreachable*.

The standard fix is a **tombstone**: a third state that a search walks through but an insert may overwrite. It is correct, and it rots.

```
// Measured: m = 65,536, linear probing, n held CONSTANT at 32,768
// (alpha = 0.5 throughout). Each cycle erases 10% and inserts 10%.
//
// churn cycles   tombstones   hit probes   miss probes
//      0                 0        1.50         2.51
//      1             2,929        1.63         2.85
//      2             5,587        1.75         3.28
//      3             7,975        1.86         3.70
//      5            12,202        2.05         4.65    <-- 85% worse
//
// The table holds the same 32,768 keys the whole time.
```

```
insert A(h=3), B(h=3), C(h=3)   ->  [3]=A [4]=B [5]=C
erase B, mark EMPTY             ->  [3]=A [4]=E [5]=C
find C: probe 3 (A, miss), 4 (EMPTY -> STOP)  => "C not found"
                                     ^ C is right there, one slot away

erase B, mark TOMB              ->  [3]=A [4]=T [5]=C
find C: probe 3, 4 (TOMB, continue), 5 (HIT)  => correct
```

THE KEY INSIGHT

A tombstone counts as full for *searching* and empty for *inserting*. That asymmetry is the whole mechanism — and the reason the effective load factor for search purposes is (n + tombstones)/m, not n/m.

THE TRAP

A delete-heavy cache whose latency climbs steadily while `size()` is flat. Nothing in the metrics explains it because the entry count is constant; the tombstones are invisible. Fix: count tombstones and rehash to the same size when (n + tombs)/m crosses a threshold — a "cleanup rehash" that allocates nothing new.

```
tombstones — search load is (n + tombs)/m · +85% miss probes after five 10% churn cycles at
constant n · requires a cleanup rehash policy
```

31. Backward-shift deletion

The alternative to tombstones, available for linear probing (and Robin Hood): after removing a key, walk forward and *pull back* any entry that is displaced from its home slot and would still be findable in the vacated position. The table ends up in exactly the state it would have had if the key had never been inserted — no residue at all.

```
void erase(uint64_t k) {
    size_t s;
    if (!locate(k, &s)) return;
    state[s] = EMPTY; --n;
    size_t gap = s, i = (s + 1) & (m - 1);
    while (state[i] == FULL) {
        size_t home = mix(keys[i]) & (m - 1);
        // is `gap` still on i's probe path from its home slot?
        bool can_move = ((i - home) & (m - 1)) >= ((i - gap) & (m - 1));
        if (can_move) {
            keys[gap] = keys[i]; vals[gap] = vals[i];
            state[gap] = FULL; state[i] = EMPTY;
            gap = i;
        }
        i = (i + 1) & (m - 1);
    }
}
```

```
before  [3]=A(h3) [4]=B(h3) [5]=C(h4) [6]=EMPTY
erase A -> gap=3, scan 4: B's home is 3, and 3 is on B's path  => move
        [3]=B     [4]=gap   [5]=C(h4)
scan 5: C's home is 4; is 4 still reachable? distance(5,4)=1 >= distance(5,gap=4)=1
        => move  [4]=C  [5]=EMPTY   -- and now the run terminates properly
```

THE KEY INSIGHT

Backward shifting restores the invariant "no FULL slot is separated from its home by an EMPTY". Tombstones preserve that invariant by lying; backward shifting preserves it by repairing.

THE TRAP

It moves entries, so **every pointer, reference and iterator into the table can be invalidated by an unrelated erase**. Symptom: a dangling reference obtained before a delete of a *different* key. It is also wrong for quadratic and double hashing, where "the probe path from home" is not a contiguous span. Topic 41.

```
backward shift — O(run length) per erase, ~1.5 moves at α = 0.5 · zero tombstones, zero rot · invalidates references · linear/Robin Hood only
```

32. Robin Hood hashing

Linear probing's real problem is not the average — it is the *variance*. At α = 0.95 the mean successful lookup is 10.3 probes but some key costs **1,608**. Robin Hood fixes the tail with one rule during insert: *if the key you are carrying is further from its home slot than the key currently occupying this slot, swap them and carry the poorer key onward*. Rich keys give to poor keys.

```
void insert(uint64_t k, int v) {
    size_t h = mix(k) & (m - 1), i = 0;
    uint64_t ck = k; int cv = v;
    while (true) {
        size_t s = (h + i) & (m - 1);
        if (state[s] != FULL) { keys[s]=ck; vals[s]=cv; state[s]=FULL; ++n; return; }
        if (keys[s] == ck)    { vals[s]=cv; return; }
        size_t d_exist = (s - (mix(keys[s]) & (m - 1))) & (m - 1);
        if (d_exist < i) {                    // the resident is richer
            std::swap(keys[s], ck); std::swap(vals[s], cv);
            h = (s - d_exist) & (m - 1); i = d_exist;   // carry the poor key
        }
        ++i;
    }
}
```

α	Mean probes, linear	Mean probes, Robin Hood	Worst, linear	Worst, Robin Hood
0.50	1.50	1.50	22	**8**
0.75	2.49	2.49	84	**16**
0.90	5.33	5.33	498	**29**
0.95	10.32	10.32	1,608	**61**

THE KEY INSIGHT

The means are *identical to two decimal places* — Robin Hood cannot change the total displacement, only its distribution. It converts a 26× worst case into a 6× one. This is a p99 technique, exactly like Topic 24.

THE TRAP

Believing Robin Hood makes the table faster on average. It does not; it makes inserts slightly slower (extra swaps) in exchange for a bounded tail. If your metric is throughput at α = 0.5, plain linear probing wins.

```
Robin Hood — same mean as linear, worst case 6–26× smaller · extra swap per displaced insert · enables early exit on lookup: stop when your displacement exceeds the resident's
```

33. Cuckoo hashing

Two tables, two hash functions, and one hard guarantee: a key is in $T_1[h_1(k)]$ or $T_2[h_2(k)]$ and nowhere else, so **lookup is exactly two probes, worst case, always**. Insert places the key in

its first slot; if that slot is occupied, it *evicts* the resident, which then moves to its alternative slot, possibly evicting another, and so on.

```
bool insert(uint64_t k, int v, int max_kicks = 64) {
    for (int t = 0; t < max_kicks; ++t) {
        size_t i = h1(k) & (m - 1);
        std::swap(k, T1key[i]); std::swap(v, T1val[i]);
        if (empty_slot_taken) return true;
        size_t j = h2(k) & (m - 1);
        std::swap(k, T2key[j]); std::swap(v, T2val[j]);
        if (empty_slot_taken) return true;
    }
    rehash_with_new_hash_functions();   // a cycle was hit
    return insert(k, v);
}
```

```
insert X: T1[h1(X)] holds A  -> evict A
          A goes to T2[h2(A)] which holds B -> evict B
          B goes to T1[h1(B)] which is EMPTY -> done      (3 writes)
cycle case: X -> A -> B -> X ...  after max_kicks, rehash everything
```

THE KEY INSIGHT

Cuckoo hashing moves all the uncertainty into *insert*. Lookup and delete become deterministic O(1) worst case — the only scheme in this section that can say that — which is why it appears in hardware, routers and flow tables where a bounded lookup matters more than insert throughput.

THE TRAP

α must stay below **0.5** for two tables (about 0.91 with three hash functions, 0.97 with four). Above that, insertion cycles become common and every insert may trigger a full rehash with new hash functions. Symptom: insert latency that is 100 ns for hours and then 500 ms, repeatedly, once the table crosses its threshold.

```
cuckoo — lookup exactly 2 probes worst case · delete O(1) · insert amortized O(1) but unbounded
worst case · α ≤ 0.5 (2 tables) or 0.91 (3)
```

34. Hopscotch hashing

Hopscotch keeps a key within a fixed **neighbourhood** H (32 or 64 slots) of its home slot, and stores a bitmap in each home slot saying which neighbours currently hold keys belonging to it. A lookup reads the bitmap, then checks only the marked slots — typically one cache line, one or two probes.

```
struct Slot { uint32_t hop;   // bit i set: a key whose home is here
              uint64_t key;   //            currently sits at home+i
              int      val; };
// find(k): read slot[h].hop, iterate its set bits only.
// insert: linear-probe to any free slot; if it is beyond home+H,
//         repeatedly "hop" a nearer key forward into it to drag the
//         hole backwards until it lands inside the neighbourhood.
```

```
home = 12, H = 8, hop bitmap = 1001 0100
        slot 12 13 14 15 16 17 18 19
             ^     ^        ^
             |     |        +-- key with home 12 lives here (bit 5)
             |     +----------- bit 2
             +----------------- bit 0
 lookup checks exactly 3 slots, all inside one 64-byte line
```

THE KEY INSIGHT

Hopscotch converts "follow a probe sequence" into "read a bitmap, then test a bounded set". It is the direct ancestor of the Swiss table's control bytes (Topic 38), which do the same thing with SIMD instead of a bitmap.

THE TRAP

If the neighbourhood cannot be cleared — no reachable key can be hopped forward — the insert fails and forces a resize even though the table has free slots. Symptom: a table that resizes at α = 0.85 despite a configured threshold of 0.95.

```
hopscotch — lookup bounded by H, usually 1 cache line · insert may cascade hops · concurrency-friendly (readers need no locks) · superseded in practice by Swiss tables
```

35. Chaining vs open addressing: the decision table

Dimension	Chaining	Open addressing
Memory per entry (int/int)	32–48 B (node + header + slot)	**16–20 B** (slot + control byte)
Allocations	One per insert	**One per table**
Cost at α = 0.9	**1.45 probes**, but each is a dependent miss	5.3 probes, mostly same cache line
Behaviour above α = 0.9	**Degrades linearly**	Cliff: miss 49 probes at 0.90, 195 at 0.95
α > 1 possible	**Yes**	No
Deletion	**Trivial unlink**	Tombstones (rot) or backward shift (invalidates)
Reference stability	**Nodes never move**	Any insert can move any entry
Large or non-movable values	**Fine**	Bad: every resize copies all values
Adversarial keys	Degrades to O(n) lists; treeify helps	Degrades to O(n) runs; harder to bound

```
// Rule of thumb, in one line each:
//   small trivially-copyable keys and values, alpha <= 0.8
//        -> open addressing (Swiss table). 2-3x faster, half the memory.
//   large values, stable references required, or alpha may exceed 1
//        -> chaining.
//   pointers into the map held across inserts
//        -> chaining, or store a stable index into a separate vector.
```

THE KEY INSIGHT

Chaining wins on probe *count*; open addressing wins on *nanoseconds*, because 5 probes inside one cache line cost less than 1.5 dependent DRAM misses. Compare 5.33 × ~1 ns against 1.45 × ~80 ns.

THE TRAP

Migrating from `std::unordered_map` to a flat table for the performance, while the codebase holds pointers to mapped values across inserts. Symptom: rare, load-dependent corruption that vanishes under a debugger — the canonical hard bug. Check for stored references *before* you migrate.

```
choose open addressing by default for small POD keys · choose chaining for stability, large values or α > 1 · the deciding question is almost never speed, it is whether references must survive
```

PRACTICE SET — Topics 26–35

A. Conceptual

1. Why must EMPTY terminate a search but TOMB must not?
2. Explain primary clustering in terms of the probability of joining a run of length L.
3. The miss formula for linear probing squares the $(1-\alpha)$ term and the hit formula does not. Why is the miss formula the one that matters in practice?
4. What does quadratic probing fix, and what does it fail to fix?
5. Why must the second hash in double hashing be odd for a power-of-two table?
6. A table's `size()` is constant and its latency is climbing. Name the mechanism and the metric you should have been exporting.
7. Why is backward-shift deletion wrong for double hashing?
8. Robin Hood does not change the mean probe count. What does it change, and what is the extra lookup optimisation it unlocks?
9. Cuckoo hashing gives worst-case O(1) lookup. Where did the cost go?
10. Give the one question that decides between chaining and open addressing more often than speed does.

B. Tracing and analysis

1. m = 8, linear probing. Insert keys with home slots 5, 5, 6, 1, 5 in that order. Draw the table after each insert and give the probe count for each.
2. From the same table, erase the key at slot 6 by writing EMPTY. Now search for the third key you inserted. Show exactly where the search stops and why.
3. Compute $\frac{1}{2}(1 + 1/(1-\alpha))$ and $\frac{1}{2}(1 + 1/(1-\alpha)^2)$ for $\alpha = 0.8$ and 0.99. Comment on the ratio between them.
4. A linear table has m = 65,536 and n = 32,768, plus 12,202 tombstones. What is the load factor the *search* experiences, and what miss cost does the Knuth formula predict for it? Compare with the measured 4.65.
5. With triangular quadratic probing on m = 16 and h = 5, list the first eight slots probed. Confirm no repeats.
6. In a Robin Hood table, a key sits at slot 100 with home slot 96. You are inserting a key whose home is 99 and you have reached slot 100. Who wins the slot, and what happens next?
7. A cuckoo table with two subtables of 2^{20} slots holds 900,000 keys. Compute α and say whether inserts are in the safe regime.
8. Compare the wall-clock cost of 1.45 dependent DRAM probes (80 ns each) with 5.33 same-cache-line probes (1 ns each, after a first 80 ns miss). Which is faster and by how much?

C. Code tasks

1. **Build.** Implement `FlatMap` with EMPTY/FULL/TOMB states and linear probing, plus `avg_probes_hit()` and `avg_probes_miss()` instrumentation. Load it to $\alpha = 0.5$, 0.75, 0.9 and 0.95 and check your numbers against 1.50/2.50, 2.49/8.48, 5.33/48.94, 10.32/195.29.
2. **Break it and observe.** Change `erase` to write EMPTY instead of TOMB. Insert three keys that share a home slot, erase the middle one, then look up the third. Record what `find` returns and how many probes it made. Then check `size()` against the number of keys iteration can reach.

3. **Break it and observe.** Restore tombstones. Hold n constant at α = 0.5 and run five cycles of "erase 10%, insert 10%". After each cycle print n, tombstone count, and average hit and miss probes. Explain a graph in which latency doubles while `size()` is a flat line, then implement the cleanup rehash that fixes it and show the numbers returning.
4. **Break it and observe.** Implement quadratic probing as `h + i*i` on a power-of-two table and insert until it fails. Report the load factor at failure and how many slots were still EMPTY. Then switch to `h + i(i+1)/2` and repeat.
5. **Measure the tail.** Add Robin Hood insertion to your linear table. At α = 0.5, 0.75, 0.9 and 0.95, print mean probes and the maximum displacement for both variants. Confirm the means match and compare the maxima.
6. **Break it and observe.** Build a flat map, take a pointer to a mapped value, insert enough keys to trigger a resize, then write through the old pointer. Run under `-fsanitize=address`. Record the error. Repeat the same experiment with `std::unordered_map` and explain the difference in outcome.

ANSWER KEY — Topics 26–35

A. Conceptual

1. EMPTY proves no key with this home slot was ever placed further along the probe path, so the search can stop — that is what makes a miss cheap. A TOMB proves only that *a* key left; keys inserted after it may still lie beyond, so stopping there would make them **unreachable**.
2. A new key joins a run if its home slot is any of the run's L slots (or the slot just past it), so **P(joining) ∝ L**. Longer runs therefore grow faster, and two runs that meet merge into one longer run — positive feedback, hence "clustering".
3. Because **every insert performs a miss search** (to find a free slot), as does every negative lookup — the common case in caches and dedupe sets. At α = 0.95 that is 195 probes against 10.
4. Fixes **primary** clustering: keys with different home slots no longer share a run. Does not fix **secondary** clustering: keys with the *same* home slot still follow the identical sequence.
5. Because the step must be **coprime with m** to visit all m slots. Every odd number is coprime with a power of two; an even step visits only m/2 or fewer slots, and a step of 0 loops on one slot forever.
6. **Tombstone rot.** You should have been exporting **(n + tombstones)/m** — the load the search actually experiences — not `size()`.
7. Because the correctness test "is the gap still on this key's path from its home?" assumes the path is the **contiguous span** home…i. Under double hashing the path is a stride-$h_2(k)$ sequence that differs per key, so a shifted entry can be moved off its own probe path and become invisible.
8. It changes the **variance** — the worst displacement falls from 1,608 to 61 at α = 0.95. It also enables **early exit**: when your current displacement exceeds the resident's, the key cannot be present, because it would have stolen this slot.
9. Into **insert**, which becomes an eviction chain of unbounded length and may trigger a full rehash with new hash functions on a cycle.
10. **"Does any code hold a pointer, reference or iterator into the map across an insert?"** If yes, chaining (or a stable index into a separate vector); speed does not enter into it.

B. Tracing and analysis

1. K1(h5)→slot 5, **1 probe**. K2(h5)→5 full, slot 6, **2**. K3(h6)→6 full, slot 7, **2**. K4(h1)→slot 1, **1**. K5(h5)→5,6,7 full → slot 0, **4**. Final: [0]=K5 [1]=K4 [5]=K1 [6]=K2 [7]=K3.
2. Slot 6 becomes EMPTY. Searching K3 (home 6) probes slot 6, sees EMPTY and **stops immediately, reporting "not found"** — although K3 is at slot 7. K5 (home 5) is also lost: it probes 5 (K1), 6 (EMPTY) and stops. **One EMPTY write orphaned two keys.**
3. α = 0.8: hit = ½(1+5) = **3.0**, miss = ½(1+25) = **13.0**, ratio **4.3×**. α = 0.99: hit = ½(1+100) = **50.5**, miss = ½(1+10,000) = **5,000.5**, ratio **99×**. The gap between hit and miss is itself $1/(1-\alpha)$.
4. Search load = (32,768 + 12,202)/65,536 = **0.686**. Knuth predicts $\frac{1}{2}(1 + 1/(1-0.686)^2)$ = ½(1 + 10.14) = **5.6**. Measured 4.65 — the same order, and lower because tombstones do not extend runs quite as effectively as live keys do (a search may still terminate at a genuine EMPTY beyond them).
5. Steps i(i+1)/2 = 0,1,3,6,10,15,21,28 added to 5, mod 16: **5, 6, 8, 11, 15, 4, 10, 1**. No repeats; the triangular sequence is a permutation of all 16 slots.

6. Resident displacement = 100−96 = 4. Yours = 100−99 = 1. The resident is **poorer** (further from home), so **it keeps the slot** and you continue probing at 101 with displacement 2. Robin Hood only swaps when the resident is *richer*.
7. α = 900,000/(2 × 1,048,576) = **0.429** — below the 0.5 threshold, so **inserts are in the safe regime**, though close enough that a 20% growth would put it at risk.
8. Chaining: 1.45 × 80 = **116 ns**. Open addressing: 80 + 4.33 × 1 = **84 ns**. Open addressing is **~1.4× faster despite 3.7× more probes**. This single comparison is why Section 5 exists.

C. Code tasks — what to verify

1. Your numbers should land within a few percent of the table in Topic 27. If your miss counts are much lower, check that `find` is really probing until EMPTY and that your test keys are genuinely absent.
2. Expect `find` **to return "not found" after 2 probes for a key that is present**, and `size()` to disagree with the number of keys iteration can reach — iteration walks slots directly and still sees them. This is data loss with no error and no crash.
3. Expect the sequence **0 → 2,929 → 5,587 → 7,975 → 10,177 → 12,202 tombstones** and miss probes **2.51 → 4.65 (+85%)** with n pinned at 32,768. The cleanup rehash — rebuilding into a same-size array, dropping tombstones — should return miss probes to ~2.5 and cost one O(m) pass.
4. `h + i*i` on power-of-two m reaches only **(m+1)/2 distinct slots**, so expect failure at roughly **α ≈ 0.5 with about half the table still EMPTY**. The triangular form inserts successfully until the table is genuinely full.
5. Means must match to two decimals (**1.50 / 2.49 / 5.33 / 10.32**). Maxima: linear **22 / 84 / 498 / 1,608**; Robin Hood **8 / 16 / 29 / 61**.
6. Expect `heap-use-after-free` on the flat map: the resize freed the old slot array. With `std::unordered_map` the write **succeeds and is visible**, because rehashing moves node pointers, not nodes — the standard guarantees references to elements remain valid across rehash. That guarantee is the whole content of Topic 41.

HANDNOTE SUMMARY CARD — Open addressing

```
OPEN ADDRESSING (26-35)
================================================================================
SLOT STATES   EMPTY terminates a search | FULL compare | TOMB walk through,
              overwrite on insert.   search load = (n + tombs)/m
PROBE SEQUENCES
  linear      (h + i) & (m-1)               cache-adjacent, primary clustering
  quadratic   (h + i(i+1)/2) & (m-1)        triangular = visits ALL slots
              h + i*i on power-of-2 reaches only (m+1)/2 slots -> false "full"
  double      (h + i*h2(k)) & (m-1)         h2 MUST be odd (|1) or infinite loop
  cuckoo      T1[h1(k)] or T2[h2(k)] ONLY    lookup = exactly 2 probes, always
  hopscotch   home + bitmap of a 32/64 neighbourhood; ancestor of Swiss (T38)

MEASURED PROBES  (m = 65536, random keys)
  alpha    linear hit/miss     quad hit/miss    double hit/miss   linear worst
  0.50       1.50 / 2.50        1.44 / 2.15      1.45 / 2.19          22
  0.75       2.49 / 8.48        1.99 / 4.64      2.02 / 4.67          84
  0.90       5.33 / 48.94       2.87 / 12.24     2.86 / 11.43        498
  0.95      10.32 / 195.29      3.62 / 25.12     3.55 / 21.89       1608
  Knuth:  hit = (1 + 1/(1-a))/2      miss = (1 + 1/(1-a)^2)/2   <- squared!

ROBIN HOOD -- same mean, bounded tail (steal the slot if resident is richer)
  worst displacement   a=0.50: 8   0.75: 16   0.90: 29   0.95: 61
  vs linear                    22        84        498       1608
  bonus: early exit on lookup when your displacement > resident's

TOMBSTONE ROT (n pinned at 32768, alpha 0.5, 10% churn per cycle)
  cycles   0      1      2      3      5
  tombs    0   2929   5587   7975  12202
  miss  2.51   2.85   3.28   3.70   4.65      <- +85% with size() unchanged
  FIX: cleanup rehash into a same-size array when (n+tombs)/m > threshold

BACKWARD-SHIFT DELETE (linear / Robin Hood only)
  pull a later entry back if ((i-home)&(m-1)) >= ((i-gap)&(m-1))
  zero tombstones, but MOVES entries -> invalidates references

CHAINING vs OPEN ADDRESSING
  memory/entry     32-48 B          vs   16-20 B
  allocations      one per insert   vs   one per table
  alpha 0.9        1.45 probes      vs   5.33 probes
  wall clock       1.45 x 80ns=116  vs   80 + 4.3x1ns = 84 ns   <- flat wins
  above 0.9        degrades linearly vs  CLIFF (195 probes at 0.95)
  delete           trivial unlink   vs   tombstones or shifting
  references       stable forever   vs   any insert may move any entry
  THE DECIDING QUESTION: does any code hold a pointer into the map? -> chaining

TRAPS
  write EMPTY on erase        -> keys behind it become unreachable, silently
  h + i*i, power-of-two m     -> "table full" at alpha 0.5
  h2 even or zero             -> infinite loop on one key in a million
  size by memory not alpha    -> 0.75 -> 0.95 saves 21% RAM, costs 23x on miss
  cuckoo above alpha 0.5      -> periodic 500 ms full rehashes
```

SECTION 5 — CACHE BEHAVIOUR AND MODERN LAYOUTS

36. The memory hierarchy: 1 ns vs 80 ns

Every complexity result so far counted *probes*. The machine does not charge by the probe; it charges by the cache line. One 64-byte line delivers 16 32-bit integers, 8 pointers, or 4 sixteen-byte slots — all for the price of one miss.

Level	Latency	Size	What it means for a table
L1	~1 ns (4 cycles)	32–48 KB	A table under ~4,000 small entries is essentially free
L2	~4 ns	0.5–2 MB	~64,000 entries
L3	~15 ns	8–64 MB	~1M entries, shared with every other process
DRAM	~80–100 ns	GB	Every probe past L3 costs 80× an L1 hit
TLB miss	+~30 ns	—	Random access over >2 MB adds page-walk cost too

```
// Same 4,000,000-entry map, uint64 -> int, random-key lookups, measured:
//   flat open-addressed table       51.4 ns per find
//   std::unordered_map (chained)    55.5 ns per find
// At 100,000 entries (fits in L3) the same code gives 9.2 ns vs 20.6 ns.
// The flat table's advantage is 2.25x when the data is cached and
// 1.08x when every lookup is a DRAM miss -- because at that point BOTH
// are paying the same 80 ns for the first touch.
```

```
one 64-byte cache line:
 +-------------------------------------------------------------+
 | 16 x int32   |  or 8 x pointer  | or 4 x {uint64 key, int val}  |
 +-------------------------------------------------------------+
 chained probe:  bucket ptr (line A) -> node (line B) -> node (line C)
                 3 lines, SERIAL, ~240 ns cold
 flat probe:     slot i, i+1, i+2, i+3 all in line A
                 1 line, ~80 ns cold, then ~1 ns per extra probe
```

THE KEY INSIGHT

Probe count is the wrong unit. Count *distinct cache lines touched, in dependency order*. Five probes on one line beat 1.5 probes on 1.5 lines, which is the entire result of Topic 35 restated in hardware terms.

THE TRAP

Benchmarking a map with 1,000 entries in a tight loop and shipping the conclusion. Everything fits in L1 and every structure looks fast; the flat table shows a 2× win that becomes 1.08× at 4M entries. Benchmark at production size, with a working set that does not fit in cache.

```
memory hierarchy — L1 1 ns · L3 15 ns · DRAM 80 ns · 64 B line · optimise for lines touched, not operations performed
```

37. AoS, SoA, and separating metadata from slots

An array of structs (AoS) puts key, value and state together, so one cache line holds 4 complete entries but also drags 12 bytes of value data you may not need. Structure of arrays (SoA) stores them in parallel arrays, so a scan of *state* alone touches 64 states per line instead of 4.

```
// AoS: 16 B per entry, 4 entries per cache line
struct Slot { uint64_t key; int val; uint8_t state; };  // + 3 B padding
std::vector<Slot> table;

// SoA: the state array is 1 B per entry -> 64 states per cache line
std::vector<uint64_t> keys;
std::vector<int>      vals;
std::vector<uint8_t>  ctrl;     // scanned 64-at-a-time (or 16 with SIMD)
```

```
AoS scan for a free slot, 64-byte line:
 [k|v|s][k|v|s][k|v|s][k|v|s]        4 states examined per line

SoA scan of the ctrl array:
 [s][s][s][s][s][s] ... x64          64 states examined per line
 => 16x fewer cache lines to find a candidate, and the keys are
    only touched for slots that already matched.
```

THE KEY INSIGHT

Separating metadata is what makes a bounded scan cheap. The Swiss table is SoA taken one step further: the metadata byte carries 7 bits of the *hash*, so a scan can reject non-matching keys without ever reading a key.

THE TRAP

SoA costs a second (and third) memory stream. For a table small enough that everything is in L1, the extra indirection makes SoA slightly *slower*. It wins when the metadata scan is the hot path and the payload is large.

```
SoA — metadata density 64/line vs 4/line · costs one extra array base pointer and one more TLB
entry · pays off above L2-sized tables
```

38. Swiss tables: h1, h2, groups, SIMD

The Swiss table (Google's `absl::flat_hash_map`, Rust's `hashbrown` since 1.36, Go's built-in map since 1.24) splits the 64-bit hash into two parts:

- **h1** — the upper 57 bits, which choose a *group* of slots (16 in absl and Rust, 8 in Go).
- **h2** — the low 7 bits, stored in a one-byte **control byte** per slot, alongside two reserved values for empty and deleted.

A lookup loads the group's 16 control bytes as one SIMD register, compares all 16 against h2 in a single instruction, and gets back a 16-bit mask of candidate slots. Only those candidates have their full keys compared. Sixteen probe steps happen at once.

```
// The core of an SSE2 Swiss lookup, in outline:
__m128i ctrl  = _mm_loadu_si128((__m128i*)&control[group]);
__m128i want  = _mm_set1_epi8(h2);
uint32_t mask = _mm_movemask_epi8(_mm_cmpeq_epi8(ctrl, want));
while (mask) {                          // usually 0 or 1 bits set
    int i = __builtin_ctz(mask);        // candidate slot
    if (keys[group * 16 + i] == k) return vals[group * 16 + i];
    mask &= mask - 1;                   // clear the lowest set bit
}
// If any control byte in the group is EMPTY, the key is absent: stop.
// Otherwise continue to the next group (quadratic on group index).
//
// Go uses 8-slot groups and SWAR -- the same comparison done with
// ordinary 64-bit arithmetic when SIMD is unavailable.
```

```
control bytes:  0x80 = EMPTY   0xFE = DELETED   0b0hhhhhhh = h2 of a live key
 group of 16:   [3A][80][7F][3A][12][80][80][3A][80][80][80][80][80][80][80][80]
 want h2 = 3A -> mask = 1000 1001    three candidates, one instruction
 false-positive rate per candidate = 1/128 -> almost always exactly one
 key compare per lookup.
```

THE KEY INSIGHT

The control byte is a 7-bit fingerprint that turns "compare the key" — a possible pointer dereference — into "compare a byte you already loaded". 128 possible values means a wrong candidate arises about once in 128 probes.

THE TRAP

h2 comes from the *low* bits of the hash and h1 from the high bits, so a hash with weak low bits (Topic 10) makes every control byte in a group identical, and every lookup compares 16 keys. Symptom: a Swiss table that performs like a linked list on integer keys hashed with the identity function — which is exactly why Rust and Go hash integers rather than using them directly.

```
Swiss lookup — 1 group load (1 cache line) + 1 SIMD compare + ~1 key compare · 1 byte metadata overhead per slot · 16 probe steps per instruction
```

39. Load factor 7/8 and group-local tombstones

Because a group scan is a single instruction, Swiss tables can run at loads that would destroy a classical linear-probing table. absl and hashbrown operate between 7/16 and **7/8 = 0.875**; Go 1.24 raised its map's maximum load from 13/16 = 0.8125 to the same 7/8. Compare Topic 27: at α = 0.875, plain linear probing needs about 32 probes for a miss.

```
// Capacity arithmetic for a Swiss table (absl-style):
//   groups   = m / 16
//   capacity = m * 7 / 8            (one slot per group kept sentinel-free)
//   growth   = double m when n > capacity
//
// 1,000,000 entries:
//   m = 2^21 = 2,097,152 slots, capacity 1,835,008, alpha = 0.477 after growth
//   memory  = 2,097,152 x (8 key + 4 val + 1 ctrl) = 27.3 MB
//   the same in std::unordered_map: ~48 B/entry + slot array = ~56 MB
```

Deletion still needs a marker, but the marker is smarter: if the group containing the deleted slot has any EMPTY byte, the probe sequence never continued past this group, so the slot can be set straight back to **EMPTY** rather than DELETED. Only when the group is completely full does a real tombstone appear. In practice that removes most of Topic 30's rot.

THE KEY INSIGHT

Higher load factors are not a memory trick — they are what you earn by making probes cheap. Cheap probes buy density; density buys fewer cache lines; fewer cache lines buy speed. The three are one decision.

THE TRAP

Copying the 7/8 load factor into a scalar linear-probing table because "Google uses it". Without group scanning you have imported the 32-probe miss and none of the mechanism. Load factor is only meaningful together with the probe strategy.

```
Swiss capacity — max load 7/8 · 1 B metadata per slot · 13 B/entry for uint64→int32 · tombstone only when a group has no EMPTY byte
```

40. Go 1.24, Rust hashbrown, absl: the measured deltas

The Swiss layout is now the industry default, and the published numbers are consistent in shape: large wins on microbenchmarks, single-digit-percent wins on whole applications, and real memory savings.

System	Change	Reported effect
Rust 1.36 (2019)	`std::collections::HashMap` became hashbrown	SwissTable layout with quadratic group probing and SIMD lookup
Go 1.24 (2025)	Built-in map replaced bucket + overflow chains	Up to ~60% faster map operations in microbenchmarks; ~1.5% geometric-mean CPU improvement across the full benchmark suite
Go 1.24, production	Same change, at Datadog scale	Hundreds of gigabytes of memory saved across their fleet
This book's measurement	Hand-written flat table vs `std::unordered_map`	2.25× at 100,000 entries; 1.08× at 4,000,000 (both DRAM-bound); 109 MB vs 195 MB

```
// Read the microbenchmark numbers carefully:
//   "60% faster map operations"  -- true, in a loop that does nothing else
//   "1.5% faster overall"        -- also true, and the number that pays
// A real service spends 1-5% of CPU in map operations. Making them
// twice as fast is worth 0.5-2.5% of total CPU: real, but not a rewrite
// justification on its own. The memory saving is often the better argument.
```

THE KEY INSIGHT

The honest summary is that the Swiss layout is a better default, not a speedup you can bank. If your profile does not show map operations near the top, switching implementations will change nothing you can measure.

THE TRAP

Datadog's own report of the Go 1.24 upgrade began with an *unexpected 20% RSS increase* on some services before the wins appeared — growth policy and allocation size classes interact. A layout change is a performance experiment, not a guaranteed improvement; measure your workload before and after.

```
Swiss migration — expect 1.5–3× on map-bound microbenchmarks, 1–3% application CPU, 20–45% map
memory · verify with your own before/after, including RSS
```

41. Reference stability: what a flat table costs you

This is the guarantee that decides architectures, and it is worth stating exactly. `std::unordered_map` promises that **references and pointers to elements remain valid across rehashing**; only iterators are invalidated. That is possible only because the elements are nodes and rehashing moves pointers, not nodes. A flat table moves the elements themselves, so every insert can invalidate every pointer.

Container	Pointers/references after insert	Iterators after insert
`std::unordered_map`	**Valid, always**	Invalid if a rehash occurred
`std::map`	Valid, always	Valid
`absl::flat_hash_map`	**Invalid on any growth**	Invalid on any growth
`absl::node_hash_map`	Valid (nodes again, by choice)	Invalid on growth
Go map	N/A — `&m[k]` is a compile error	N/A
Rust `HashMap`	Enforced by the borrow checker at compile time	Same

```
// The bug this guarantee prevents, in C++:
auto& v = flat_map[key];      // reference into the slot array
flat_map[other_key] = 1;      // may reallocate and move every entry
v += 1;                       // writes into freed memory -- ASAN:
                              // heap-use-after-free
// The same code with std::unordered_map is CORRECT and defined.
```

Go and Rust designed the problem away rather than documenting it: Go forbids taking the address of a map element at all, which is why `m[k].field = x` does not compile for struct values, and Rust's borrow checker refuses the second mutable borrow at compile time.

THE KEY INSIGHT

"Which is faster" is a tuning question. "Do references survive" is an interface question, and interface questions decide which container you can use at all.

THE TRAP

A cache of objects where callers keep a pointer to the cached value. Migrating that map to a flat table introduces corruption that only appears when the table happens to grow — load-dependent, timing-dependent, and invisible in tests. If you need both flat performance and stability, store `vector<T>` values and keep *indices* in the map.

```
stability — nodes: stable refs, +24 B/entry, +1 indirection · flat: no stability, 16–20 B/entry,
no indirection · the index-into-a-vector pattern buys both for one extra load
```

PRACTICE SET — Topics 36–41

A. Conceptual

1. Restate Topic 35's conclusion using cache lines rather than probes.
2. Why does the flat table's advantage shrink from 2.25× to 1.08× as the table grows past L3?
3. What exactly does separating the control bytes into their own array buy, measured in entries examined per cache line?
4. Which bits of the hash become h1 and which become h2, and why does that choice make a weak-low-bits hash catastrophic for a Swiss table?
5. Why can a Swiss table run at α = 0.875 when a scalar linear-probing table cannot?
6. Under what condition can a Swiss table mark a deleted slot EMPTY instead of DELETED, and why is that legal?
7. "Go 1.24 made maps 60% faster" and "Go 1.24 improved CPU by 1.5%" are both true. Reconcile them.
8. State the C++ standard's stability guarantee for `std::unordered_map` precisely — for references and for iterators.
9. How do Go and Rust remove the dangling-reference problem instead of documenting it?
10. Describe the pattern that gives you flat-table performance and stable handles at the same time, and its cost.

B. Tracing and analysis

1. A 64-byte cache line holds how many of each: `int32`, `uint64`, 8-byte pointers, 16-byte slots, 1-byte control bytes?
2. A chained lookup touches the bucket array, one node, then a second node. Compute the cold cost at 80 ns per line and compare with a flat lookup that touches one line and probes four slots inside it.
3. A Swiss table holds 1,000,000 entries of `uint64 → int32`. Compute m after growth (power of two, capacity = 7m/8), the total bytes including control bytes, and the bytes per entry.
4. The same 1,000,000 entries in `std::unordered_map` at α = 0.75, with 32 B nodes plus 16 B allocator overhead. Compute total bytes and the ratio to your answer for B3.
5. A control byte holds 7 bits of hash. What is the probability that a non-matching slot produces a false candidate, and how many key comparisons per lookup do you expect in a group of 16 that contains one match?
6. At α = 0.875, use Knuth's formulas to compute the miss probe count for scalar linear probing. Compare with a Swiss lookup's cost in *instructions*.
7. Your service spends 3% of CPU inside map operations. A new map implementation is 2× faster on those operations. Compute the total CPU saving, and state what else would have to be true for the migration to be worth a week of work.

C. Code tasks

1. **Build and measure.** Implement a flat linear-probing `uint64 → int` map and benchmark `find` against `std::unordered_map` at n = 1,000, 100,000 and 4,000,000 with α ≈ 0.5 and random keys. Report ns per lookup and the ratio at each size, plus the memory each uses.
2. **Break it and observe.** Take `auto& v = flat[key];`, then insert keys until the table grows, then write through `v`. Build with `-fsanitize=address -g` and record the exact error class and the allocation stack. Then run the identical code with `std::unordered_map` and record what happens instead. Explain the difference in one sentence about nodes.

3. **Break it and observe.** Implement a 16-slot-group control-byte lookup. First take h2 from the *same* high bits as h1 (or use an identity hash on multiples of 64). Print, for 10,000 lookups, the average number of full key comparisons per lookup. Then take h2 from the low 7 bits of a properly mixed hash and print it again.
4. **Measure the layout.** Write the same table twice, AoS (`vector<Slot>`) and SoA (three parallel vectors). Time a scan that counts free slots in each, at 4M entries. Report the ratio and explain it with the per-line entry counts from B1.
5. **Break it and observe.** Take your scalar linear-probing table from Section 4 and raise its maximum load factor to 0.875 "because Swiss tables use 7/8". Measure average miss probes and p99 lookup latency before and after.
6. **Stability by construction.** Refactor task C2's code to store values in a `std::vector<Value>` and keep `unordered_map<Key, size_t>` indices. Show that the ASAN error disappears, and state the one new failure mode you have introduced.

ANSWER KEY — Topics 36–41

A. Conceptual

1. Chaining touches **1.45 cache lines in dependency order**; open addressing touches **one line and probes inside it**. 1.45 × 80 ns beats nothing; 80 + 4 × 1 ns beats it.
2. Because past L3 **both** designs pay one unavoidable ~80 ns DRAM miss for the first touch, and that fixed cost dominates the difference in what happens afterwards.
3. **64 states per line instead of 4** (16×), because a control byte is 1 B and an AoS slot is 16 B.
4. h1 = the **high 57 bits** (group selection), h2 = the **low 7 bits** (control byte). If the low bits are constant across keys — aligned pointers, identity hashes — every control byte in a group is the same value, every slot becomes a candidate, and the SIMD filter degenerates into 16 key comparisons.
5. Because its probe unit is a **group of 16 compared in one instruction**, so the number of *instructions* grows far more slowly than the number of *slots* examined.
6. When the deleted slot's group still contains at least one **EMPTY** control byte. That proves no probe sequence ever ran past this group, so nothing can be orphaned — the exact condition Topic 30 needs.
7. The first is a **microbenchmark** of map operations in isolation; the second is a whole-program figure where maps are a few percent of CPU. 60% of 3% is ~1.8%.
8. **References and pointers to elements remain valid** under rehash and insert; **iterators are invalidated** by a rehash. Erasure invalidates only the erased element's iterator and references.
9. Go makes `&m[k]` a **compile error**; Rust's **borrow checker** refuses a second mutable borrow while the first is live. Neither can be violated at run time.
10. Store values in a `vector` and keep **indices** in the map. Cost: **one extra dependent load** per access, plus you must manage the vector's own erasure (tombstones or swap-and-fix-up).

B. Tracing and analysis

1. **16** int32, **8** uint64, **8** pointers, **4** 16-byte slots, **64** control bytes.
2. Chained: **3 × 80 = 240 ns**, serialised. Flat: **80 + 3 × ~1 = 83 ns**. The flat table does more probes and is **2.9× faster**.
3. Capacity must be ≥ 1,000,000, so $7m/8 \geq 10^6 \rightarrow m \geq 1{,}142{,}858 \rightarrow m = \mathbf{2^{21} = 2{,}097{,}152}$. Bytes = 2,097,152 × (8 + 4 + 1) = **27.3 MB**; per entry = **27.3 B**.
4. $m = 2^{21}$ slots × 8 B = 16.8 MB; nodes = 10^6 × 48 = 48 MB; total **64.8 MB**, which is **2.4×** the Swiss figure.
5. 7 bits → **1/128 = 0.78%** per non-matching slot. In a group of 16 with one true match: 1 + 15/128 = **1.12 key comparisons** expected.
6. Miss = ½(1 + 1/(1−0.875)²) = ½(1 + 64) = **32.5 probes**. A Swiss lookup at the same load is roughly **one group load, one SIMD compare and ~1.1 key compares** — call it a dozen instructions.
7. 3% × 50% = **1.5% of total CPU**. It is worth a week only if something else comes with it: the **memory saving** (typically 20–45% of map footprint), removal of a latency spike, or the map being on a critical path whose p99 matters more than its mean.

C. Code tasks — what to verify

1. Expect roughly **2.3 vs 4.6 ns (2.0×) at 1,000; 9.2 vs 20.6 ns (2.25×) at 100,000; 51 vs 56 ns (1.08×) at 4,000,000**, and memory around **109 MB vs 195 MB** at 4M. If your flat table is slower at 4M, check that you are not recomputing the hash inside the probe loop.
2. Expect `heap-use-after-free` with the "freed by" stack inside your `grow()`/`rehash()`. With `std::unordered_map` the write is **correct and defined**: rehashing relinks node pointers, and the node containing the element never moves.
3. With h2 taken from the same bits as h1, expect **~16 key comparisons per lookup** — the filter passes everything. With a mixed hash and the low 7 bits, expect **~1.1**. Same table, same load factor, a 14× difference in work.
4. Expect the SoA scan to be **8–16× faster**, converging on the ratio of bytes touched (1 B vs 16 B per entry). If the ratio is much smaller, the compiler vectorised the AoS version's stride — check with `-fno-tree-vectorize`.
5. Expect average miss probes to rise from ~2.5 (at α = 0.75) to **~32**, and p99 lookup latency to rise by an order of magnitude. The load factor was never the mechanism; the group scan was.
6. The ASAN error disappears because the `vector` is only appended to and you hold an index, not a pointer. The new failure mode: **a stale index** after an erase — an index is not a weak reference and nothing detects that its slot has been reused. Generation counters or tombstoned vector entries are the usual fix.

HANDNOTE SUMMARY CARD — Cache and layout

```
CACHE BEHAVIOUR AND MODERN LAYOUTS (36-41)
================================================================================
LATENCY BUDGET     L1 ~1 ns | L2 ~4 ns | L3 ~15 ns | DRAM ~80-100 ns | TLB +30 ns
64-BYTE LINE HOLDS 16 int32 | 8 uint64 | 8 pointers | 4 x 16B slots | 64 ctrl B

THE UNIT THAT MATTERS: distinct cache lines touched IN DEPENDENCY ORDER.
  chained  bucket -> node -> node  = 3 serial lines = 240 ns cold
  flat     slot i, i+1, i+2, i+3   = 1 line         =  83 ns cold

MEASURED: flat linear-probing vs std::unordered_map, uint64->int, alpha 0.5
  n = 1,000        2.3 ns  vs   4.6 ns    2.00x
  n = 100,000      9.2 ns  vs  20.6 ns    2.25x     (fits in L3)
  n = 4,000,000   51.4 ns  vs  55.5 ns    1.08x     (both DRAM-bound)
  memory at 4M     109 MB  vs  195 MB
  LESSON: benchmark at production size; the win evaporates past L3.

SWISS TABLE (absl::flat_hash_map, Rust hashbrown 1.36+, Go 1.24+)
  h1 = high 57 bits -> which GROUP        h2 = low 7 bits -> control byte
  ctrl: 0x80 EMPTY   0xFE DELETED   0b0hhhhhhh = live key's h2
  lookup: load 16 ctrl bytes -> SIMD compare vs h2 -> 16-bit candidate mask
          -> ~1.12 full key compares (false candidate rate 1/128)
  Go uses 8-slot groups + SWAR (SIMD within a register), no vector unit needed
  max load 7/8 = 0.875  (Go 1.23 was 13/16 = 0.8125)
  delete -> EMPTY if the group still has an EMPTY byte, else DELETED
  memory, uint64->int32, 1M entries: 2^21 slots x 13 B = 27.3 MB = 27.3 B/entry
           std::unordered_map equivalent: 64.8 MB = 2.4x

PUBLISHED DELTAS   Rust 1.36 adopted hashbrown; Go 1.24 reports up to ~60% on
  map microbenchmarks and ~1.5% geomean CPU overall; Datadog saved hundreds of
  GB in production -- but first saw a ~20% RSS INCREASE on some services.
  Rule: 60% of 3% of CPU = 1.8%. Migrate for memory or p99, not for the 60%.

REFERENCE STABILITY -- the interface question, not the tuning question
  std::unordered_map   refs/pointers VALID across rehash; iterators invalid
  std::map             both valid
  absl::flat_hash_map  both INVALID on any growth
  absl::node_hash_map  refs valid (nodes, by choice)
  Go                   &m[k] does not compile
  Rust                 borrow checker rejects it at compile time
  PATTERN for both: map<K, size_t> index into a vector<V>. Cost: 1 extra load,
  plus stale-index risk (use a generation counter).

TRAPS
  benchmark at 1,000 entries   -> everything fits in L1, every design "wins"
  h2 from the same bits as h1  -> 16 key compares per lookup instead of 1.1
  copy 7/8 load into a scalar  -> miss cost 32 probes; you took the number and
                                  left the mechanism behind
  keep a ref into a flat table -> heap-use-after-free on the next growth
  SoA on an L1-sized table     -> extra streams, slightly slower
```

PART II

Shapes and Tooling

What the structure becomes when you vary it, and what the standard library already gives you. Sections 6–7: sets, multimaps, ordered maps, perfect hashing and Bloom filters, then the real implementations in C++, Java, Python, PHP, JavaScript, Go, Rust and Redis.

SECTION 6 — VARIANTS

42. Hash sets

A set is a map with the value removed. Every mechanism in Sections 3–5 is unchanged; only the slot payload shrinks. That makes sets meaningfully cheaper — for `uint64` keys a flat set is 9 bytes per slot against 13 — and it makes one operation, *membership*, the whole interface.

```
std::unordered_set<std::string> seen;
if (!seen.insert(line).second) {         // insert returns {iter, inserted}
    // this line was already present -- one hash, one probe, no second lookup
}
// Anti-pattern: two hashes for one question.
if (seen.count(line) == 0) seen.insert(line);   // hashes `line` TWICE
```

```
map slot   [ key | value | ctrl ]     13-21 B for uint64 -> int
set slot   [ key |       | ctrl ]      9 B
                  ^ the only difference in the entire data structure
```

The set operations people reach for a library for — union, intersection, difference — are all "iterate the smaller, probe the larger", which is O(min(a,b)) hashes and probes. Doing it the other way round is a common and silent 100× error when the sets differ hugely in size.

THE KEY INSIGHT

Intersection is a loop over the *smaller* set. Union is a copy of the larger plus a loop over the smaller. Never iterate the big one.

THE TRAP

`count()` then `insert()` costs two hashes and two probe sequences to answer one question. On a hot dedupe path that is a 2× regression that no profiler flags as a bug, because both calls look reasonable.

hash set — same complexity as the map · ~30% less memory · use `insert().second`, not `count()` then `insert()`

43. Multimaps and map-of-lists

When one key legitimately maps to many values you have two options, and they are not equivalent.

```
// (a) a real multimap: duplicate keys inside the table
std::unordered_multimap<std::string, int> mm;
mm.emplace("bus", 1); mm.emplace("bus", 2);
auto [lo, hi] = mm.equal_range("bus");     // iterate the equal keys

// (b) map of vectors: one key, one entry, a container as the value
std::unordered_map<std::string, std::vector<int>> mv;
mv["bus"].push_back(1);                    // creates the vector on demand
mv["bus"].push_back(2);
```

Aspect	Multimap	Map of vectors
Key storage	Stored once *per value*	Stored once total
Memory, 1 key × 1,000 values, 20-byte key	~68 KB	**~4 KB**
Iterating one key's values	Probe sequence, values scattered	**Contiguous vector**
Erase one (key, value) pair	Direct	Find + linear scan of the vector
Counting values for a key	O(count)	**O(1)** — `.size()`

THE KEY INSIGHT

A multimap re-stores the key for every value; a map of vectors stores it once and keeps the values contiguous. The multimap only wins when you frequently erase individual (key, value) pairs.

THE TRAP

`mv[key]` default-constructs an empty vector on a miss, so a "read" leaves an empty vector behind. Iterating later then yields keys with zero values, and `size()` counts keys that logically do not exist. Use `find` for reads. Same root cause as Topic 17's trap.

```
map-of-vectors — 1 key copy, contiguous values, O(1) count · multimap — 1 key copy per value,
cheap pair erase · prefer map-of-vectors by default
```

44. Insertion-ordered maps

Topic 20 established that iteration order is arbitrary. Three widely used implementations restore it, all with the same trick: keep a **dense array in insertion order** and let the hash part store *indices* into it rather than entries.

```
// The shape shared by Python 3.7+ dict, PHP 7+ arrays and LinkedHashMap:
//
//   indices : [ -1, 2, -1, 0, -1, 1, -1, -1 ]   sparse, m entries, i32 each
//   entries : [ (h,k,v), (h,k,v), (h,k,v) ]     dense, n entries, insertion
//                                                order, iterated directly
// Iteration is a scan of `entries` -- O(n), not O(m + n) -- and it is in
// insertion order for free. Lookup costs one extra indirection.
```

```
classic table (m = 8, n = 3)      compact ordered (m = 8, n = 3)
 [ ][k1][ ][k3][ ][k2][ ][ ]       indices [-1][1][-1][0][-1][2][-1][-1]
 iterate: scan 8 slots             entries [k3,v3][k1,v1][k2,v2]
 order: k1, k3, k2 (arbitrary)     iterate: scan 3 entries, insertion order
 memory: 8 x 24 B = 192 B          memory: 8 x 4 + 3 x 24 = 104 B
```

The memory saving is the reason Python adopted it: the sparse array holds 4-byte indices instead of 24-byte entries, so a dict became **20–25% smaller** and ordered as a side effect. PHP's `zend_array` does the same and adds a further optimisation (Topic 51). Java's `LinkedHashMap` takes the other route — a doubly linked list threaded through the nodes — which costs 16 extra bytes per entry and is the basis of its LRU mode (Topic 72).

THE KEY INSIGHT

Ordering is not an add-on to hashing; it is a second data structure — an array or a list — with the hash table demoted to an index over it. That is exactly the LRU cache pattern of Topic 72, arrived at from the other direction.

THE TRAP

Deletion leaves a hole in the dense array. Python and PHP mark it and compact later, so iteration must skip holes, and a delete-heavy dict can hold a dense array far larger than `len()`. Symptom: a Python dict whose memory does not fall after `del` — it falls only on the next resize.

```
compact ordered map — iteration O(n) in insertion order · 20–25% less memory than a classic
layout · +1 indirection per lookup · delete leaves holes until a rebuild
```

45. Perfect and minimal perfect hashing

If the key set is **known in advance and never changes** — reserved words, opcodes, HTTP header names, Unicode tables — you can search offline for a hash function with *zero collisions* on exactly those keys. Lookup then costs one hash and one probe, guaranteed, with no probe loop, no load factor and no tombstones.

- **Perfect hash**: no collisions, table may be larger than n.
- **Minimal perfect hash (MPH)**: no collisions *and* exactly n slots — a bijection from keys to 0..n−1.

```
// gperf, the classic generator, ships with GCC:
//   $ gperf --output-file=kw.c keywords.txt
// produces a static table plus:
//     if (len <= MAX_WORD_LENGTH && len >= MIN_WORD_LENGTH) {
//         unsigned key = hash(str, len);          // asso_values lookup
//         if (key <= MAX_HASH_VALUE) {
//             const char* s = wordlist[key].name;
//             if (*str == *s && !strcmp(str + 1, s + 1)) return &wordlist[key];
//         }
//     }
// One hash, one array read, one string compare. No loop at all.
//
// Modern MPH constructions (CHD, BBHash, PTHash) reach 2-4 BITS per key
// of index overhead for millions of keys, built in seconds.
```

THE KEY INSIGHT

A perfect hash moves all the collision work to build time, where it is allowed to take a second. It is the right answer surprisingly often — compilers, protocol parsers and static asset maps all have fixed key sets.

THE TRAP

A perfect hash gives no answer for keys outside the set — it will map an unknown key onto some existing slot. **You must still compare the key.** Symptom of forgetting: an HTTP parser that treats `X-Forwarded-Fro` as `X-Forwarded-For`.

```
MPH — lookup: 1 hash + 1 probe + 1 compare, worst case · index overhead 2–4 bits/key · build offline · keys must be fixed forever
```

46. Bloom filters

A Bloom filter answers "have I seen this?" with *no* stored keys: a bit array of m bits and k hash functions. Insert sets k bits; a query that finds all k bits set answers "probably yes", and any zero bit answers **"definitely no"**. False positives are possible; false negatives are impossible.

```
struct Bloom {
    std::vector<uint64_t> bits;  size_t m, k;
    void add(uint64_t h) {
        uint64_t a = h, b = h >> 32 | 1;          // double hashing (T29)
        for (size_t i = 0; i < k; ++i) {
            size_t p = (a + i * b) % m;
            bits[p >> 6] |= 1ULL << (p & 63);
        }
    }
    bool maybe(uint64_t h) const {
        uint64_t a = h, b = h >> 32 | 1;
        for (size_t i = 0; i < k; ++i) {
            size_t p = (a + i * b) % m;
            if (!(bits[p >> 6] >> (p & 63) & 1)) return false;  // certain
        }
        return true;                                             // probable
    }
};
```

Bits per element	Optimal k = 0.693·m/n	False-positive rate	1M elements
4	3	14.7%	0.5 MB
8	6	2.1%	1.0 MB
10	7	**0.82%**	1.25 MB
16	11	0.046%	2.0 MB
Exact hash set (20-byte keys)	—	0%	**~48 MB**

THE KEY INSIGHT

1.25 MB against 48 MB for a million keys, in exchange for being wrong 0.82% of the time in *one* direction. That trade is worth taking whenever a false positive costs only an extra check against the real store.

THE TRAP

There is no delete. Clearing bits would break other keys that share them — that is a false *negative*, which destroys the filter's only guarantee. Symptom: a "we removed the user from the filter" change that makes an authorisation check miss legitimate entries. Use a counting Bloom filter or rebuild.

```
Bloom — insert/query O(k), typically 7 · 10 bits/element for 0.82% FP · no deletion, no iteration, no key recovery · false negatives impossible
```

47. Count-min sketch and HyperLogLog

Two more structures that replace exact storage with bounded error, both built from the same ingredient: several independent hash functions over a small array. Use them when the exact answer would not fit in memory.

Count-min sketch — approximate *frequencies*. A d×w matrix of counters; increment row i at column $h_h(k)$; query returns the *minimum* of the d counters. Counts are never underestimated, and are overestimated with probability δ by at most εN.

HyperLogLog — approximate *cardinality*. Hash each element, use the first p bits to select one of 2^p registers, and store the maximum number of leading zeros seen. Redis uses p = 14, giving 2^{14} = 16,384 registers in about 12 KB with a standard error of 1.04/√16384 = **0.81%** — for any cardinality, from 10 to 10^9.

```
// The comparison that motivates both, for counting distinct users:
//   std::unordered_set<uint64_t>  100,000,000 ids -> ~3.2 GB, exact
//   HyperLogLog (p = 14)                          ->   12 KB, +/- 0.81%
//
// Redis, in practice:
//   PFADD  visitors:2026-08-13  user:9001 user:9002
//   PFCOUNT visitors:2026-08-13          -> 2
//   PFMERGE visitors:week  visitors:2026-08-13 ...   (unions are exact)
```

THE KEY INSIGHT

Bloom answers membership, count-min answers frequency, HyperLogLog answers cardinality — and all three replace "store the keys" with "store a few bits per hash". The moment you accept a bounded error, memory stops scaling with n.

THE TRAP

Count-min overestimates, so a heavy-hitter query on a skewed stream can report a rare key as frequent when it collides with a hot one. Never use a count-min value for anything that must be exact — billing, quotas, rate limits with hard cutoffs.

```
count-min — d×w counters, error ≤ εN with probability 1−δ · HyperLogLog — 12 KB for 0.81% error
at any cardinality · both mergeable, neither reversible
```

PRACTICE SET — Topics 42–47

A. Conceptual

1. What is the only structural difference between a hash set and a hash map, and roughly what does it save for a `uint64` key?
2. Why is `insert().second` strictly better than `count()` followed by `insert()`?
3. Give the rule for intersecting two sets of very different sizes, and the cost of getting it backwards.
4. State the two situations in which a real multimap beats a map of vectors.
5. Describe the compact-ordered layout in one sentence, and say where the insertion order actually lives.
6. Why does deleting from a compact-ordered dict not immediately return memory?
7. A minimal perfect hash gives one probe with no loop. What must you still do on every lookup, and what breaks if you skip it?
8. Why can a Bloom filter never support deletion by clearing bits?
9. Which of Bloom, count-min and HyperLogLog can be merged across shards, and what does that enable?
10. Name a use of a count-min sketch that is always wrong, and say why.

B. Tracing and analysis

1. One key with 1,000 values, 20-byte keys, 4-byte values. Compute the memory for a multimap (key stored per value, ~48 B/node) and for a map of vectors. Give the ratio.
2. A compact-ordered dict has m = 8 index slots (4 B each) and 3 entries (24 B each). Compute its memory and compare with a classic layout of 8 × 24 B slots.
3. For a Bloom filter with 10 bits per element, compute the optimal k from $0.693 \cdot m/n$ and the false-positive rate from $(1-e^{-kn/m})^k$.
4. You need a 0.1% false-positive rate. Rearrange the formula (or read the table) to find the bits per element required, and give the total size for 50,000,000 elements.
5. Compare that Bloom filter with an exact `unordered_set` of 50,000,000 20-byte keys at ~80 B/entry. State both sizes and the ratio.
6. HyperLogLog with p = 14 registers: compute the standard error from $1.04/\sqrt{(2^p)}$, and state the memory. What is the error for p = 16?
7. A count-min sketch has d = 5 rows and w = 2,000 columns over a stream of N = 10,000,000 events. Compute $\varepsilon = e/w$ and the maximum expected overestimate εN.

C. Code tasks

1. **Build and measure.** Implement the Bloom filter from Topic 46 with m/n = 10 and k = 7. Insert 1,000,000 random keys, then query 1,000,000 keys known to be absent and report the measured false-positive rate against the predicted 0.82%.
2. **Break it and observe.** Add a `remove()` that clears the k bits for a key. Insert keys A and B, remove A, then query B. Repeat over 10,000 keys and count how many *present* keys the filter now denies. Explain why this is a worse bug than a false positive.
3. **Break it and observe.** Write a "count words per document" routine using `map<string, vector<int>>` where the *read* path uses `mv[key].size()`. Query 1,000 keys that were never inserted, then print `mv.size()` and iterate. Record what you find, then rewrite the read path with `find`.

4. **Measure the shapes.** Store one key with 100,000 values as both an `unordered_multimap` and an `unordered_map<string, vector<int>>`. Measure process RSS for each, and time a full iteration of that key's values in both.
5. **Break it and observe.** Build a small perfect-hash lookup for ten HTTP header names (by hand or with `gperf`), then *delete the final string comparison*. Query `"X-Forwarded-Fro"` and `"Content-Lengthx"` and record what the table returns.
6. **Observe the overestimate.** Build a count-min sketch with d = 4, w = 512 and feed it a Zipfian stream of 1,000,000 events over 100,000 distinct keys. Compare the sketch's estimate with the true count for the top 10 keys and for 10 keys that appeared exactly once.

ANSWER KEY — Topics 42–47

A. Conceptual

1. The **value field** in the slot. For `uint64` keys a flat set is about **9 B/slot against 13** — roughly 30%.
2. Because `insert` already performs the lookup and returns whether it happened: **one hash and one probe sequence instead of two**.
3. **Iterate the smaller set, probe the larger.** Backwards, the cost is O(larger) — for a 10-element set against a 1,000,000-element set that is a **100,000×** difference in hashes performed.
4. When you frequently **erase individual (key, value) pairs**, and when values arrive interleaved and you never need one key's values contiguously.
5. A sparse array of **indices** plus a dense array of **entries**; the order lives in the **dense array**, which is appended to and iterated directly.
6. Because deletion only **marks a hole** in the dense array; compaction happens on the next resize, so memory falls later, not on `del`.
7. You must still **compare the key**. A perfect hash is only perfect for keys in the set; an unknown key maps onto some existing slot and is accepted as that key.
8. Because bits are **shared between keys**. Clearing them can turn another key's "yes" into a "no" — a **false negative**, which is the one error the structure promises never to make.
9. **All three.** Bloom and count-min merge with bitwise OR and elementwise addition, HyperLogLog with a register-wise maximum. That is what lets you compute a global result from per-shard sketches without shipping the raw data.
10. Anything requiring exactness: **billing, quotas, hard rate-limit cutoffs**. Count-min never underestimates but can overestimate, so a rare key colliding with a hot one is reported as frequent.

B. Tracing and analysis

1. Multimap: 1,000 × ~68 B (48 B node + 20 B key) ≈ **68 KB**. Map of vectors: 68 B for the single entry + 1,000 × 4 B = **~4.1 KB**. Ratio ≈ **17×**.
2. Compact: 8×4 + 3×24 = 32 + 72 = **104 B**. Classic: 8×24 = **192 B**. The compact layout is **46% smaller** and iterates 3 entries instead of scanning 8 slots.
3. $k = 0.693 \times 10 = 6.93 \rightarrow \mathbf{7}$. $(1-e^{-0.7})^7 = (0.5034)^7 = \mathbf{0.0082 = 0.82\%}$.
4. 0.1% needs about **14.4 bits per element** (k = 10). For 50,000,000 elements: 50e6 × 14.4 / 8 = **90 MB**.
5. Exact set: 50e6 × 80 B = **4.0 GB**. Ratio **44×**, in exchange for one wrong answer in a thousand, always in the "false yes" direction.
6. $1.04/\sqrt{16384} = 1.04/128 =$ **0.81%**, in about **12 KB**. For p = 16: 1.04/256 = **0.41%**, in about 48 KB — halving the error costs 4× the memory.
7. $\varepsilon = e/w = 2.718/2000 =$ **0.00136**. Maximum expected overestimate $= \varepsilon N = 0.00136 \times 10^7$ = **13,590 events** — irrelevant for a key seen a million times, fatal for a key seen five times.

C. Code tasks — what to verify

1. Expect a measured rate within about 10% of **0.82%** — roughly 8,200 false positives in 1,000,000 absent queries. A rate near zero means your k hash positions are correlated

(reuse the double-hashing trick from Topic 29 rather than hashing the key k times with the same function).

2. Expect on the order of **k×(collision rate)** present keys to be denied — with 10,000 removals over a 10-bit filter, hundreds. The bug is worse because the filter's contract is "no false negatives", so all downstream code skips the expensive check entirely and the record is simply **never found**.
3. Expect `mv.size()` to be **1,000 larger** than the number of keys you inserted, with 1,000 keys mapping to empty vectors, and iteration to yield them. The `find` version leaves the map untouched.
4. Expect roughly **7–17× more RSS** for the multimap and a noticeably faster iteration for the vector (contiguous, prefetchable) — often **5–20×** on the scan itself.
5. Expect a **match**: the perfect hash maps the unknown string onto a slot and, with no comparison, reports the header stored there. This is how a parser comes to treat `X-Forwarded-Fro` as `X-Forwarded-For`.
6. Expect the **top-10 estimates to be exact or nearly so** (their true counts dwarf the noise) and the **once-seen keys to be reported as tens or hundreds**, because they collide with heavy hitters in every row. That asymmetry is the structure's defining property.

HANDNOTE SUMMARY CARD — Variants

```
VARIANTS (42-47)
================================================================================
HASH SET      map minus the value.  uint64 slot: 9 B vs 13 B (~30% less)
  if (!seen.insert(x).second) { /* duplicate */ }    ONE hash, not two
  intersection/union: ALWAYS iterate the SMALLER set, probe the larger

MULTIMAP vs MAP-OF-VECTORS
  multimap        key stored per VALUE     1 key x 1000 values ~ 68 KB
  map<K,vector<V>> key stored once, values contiguous          ~ 4 KB  (17x)
  multimap wins only when you erase individual (key,value) pairs
  TRAP: mv[k] on a read path leaves an empty vector behind -> size() inflates

COMPACT ORDERED MAP (Python 3.7+, PHP 7+, and the shape of LinkedHashMap)
  indices[m] : int32 sparse      entries[n] : (hash,key,value) dense, in order
  iteration O(n) IN INSERTION ORDER; 20-25% smaller than the classic layout
  cost: +1 indirection per lookup; delete leaves holes until the next resize

PERFECT / MINIMAL PERFECT HASHING  (keys fixed forever: keywords, headers)
  perfect = no collisions;  minimal = exactly n slots, a bijection to 0..n-1
  lookup = 1 hash + 1 array read + 1 key compare, NO LOOP, worst case
  gperf for small sets; CHD/BBHash/PTHash reach 2-4 BITS per key
  TRAP: unknown keys map onto some slot -- you MUST still compare the key

BLOOM FILTER    m bits, k hashes.  no false negatives, ever.  no deletion.
  k_opt = 0.693 * m/n         fp = (1 - e^(-kn/m))^k
  bits/elem     4      8     10      16
  k             3      6      7      11
  fp        14.7%   2.1%  0.82%  0.046%
  1M keys at 10 bits = 1.25 MB   vs exact set of 20-byte keys = ~48 MB (38x)
  positions: a = h, b = (h>>32)|1, p_i = (a + i*b) % m   <- double hashing

COUNT-MIN SKETCH   d x w counters; query = MIN over rows
  never underestimates; overestimate <= eps*N with eps = e/w
  d=5, w=2000, N=1e7 -> eps=0.00136 -> up to 13,590 over. Fine for hot keys,
  useless for rare ones. NEVER for billing, quotas or hard cutoffs.

HYPERLOGLOG     cardinality in fixed memory
  p=14 -> 16384 registers, ~12 KB, error 1.04/sqrt(2^p) = 0.81%
  p=16 -> 0.41% error for 48 KB   (halve the error, 4x the memory)
  100M distinct ids: exact set ~3.2 GB   vs   HLL 12 KB
  Redis: PFADD / PFCOUNT / PFMERGE (merges are exact)

ALL THREE SKETCHES MERGE: Bloom by OR, count-min by +, HLL by register max.
```

SECTION 7 — THE STANDARD LIBRARY LAYER

48. C++ std::unordered_map

The standard specifies `unordered_map` so tightly that a fast implementation is impossible. It mandates a **bucket interface** (`bucket_count`, `bucket(k)`, `begin(n)`), **reference stability** across rehash (Topic 41), and forward iterators over all elements — which together force separate chaining with real nodes.

```
std::unordered_map<uint64_t,int> m;
m.max_load_factor(1.0);          // the DEFAULT is 1.0, not 0.75
m.reserve(1'000'000);            // sizes buckets so no rehash occurs
m.bucket_count();                // libstdc++: a PRIME (13, 29, 59, 127, ...)
                                 // 4M inserts ends at 5,967,347 buckets
// libstdc++ keeps ONE singly linked list through all elements, plus a
// bucket array of pointers INTO that list -- so iteration is O(n),
// not O(m + n), at the cost of an extra pointer hop on erase.
```

Measured against a hand-written flat table on the same keys: **4.6 ns vs 2.3 ns at n = 1,000; 20.6 vs 9.2 at 100,000; 55.5 vs 51.4 at 4,000,000**. It is not slow because the implementers were careless; it is slow because the specification requires nodes.

THE KEY INSIGHT

Every "why is `unordered_map` slow" answer reduces to two clauses in the standard: pointer stability and the bucket interface. If you need neither, use `absl::flat_hash_map` or a hand-rolled table.

THE TRAP

`std::hash<int>` is the identity function. libstdc++ survives it with prime bucket counts (Topic 8); a power-of-two table does not. Symptom: copying a key type into a flat map and losing 10× throughput.

```
unordered_map — node-based, ~48 B/entry · default max_load_factor 1.0 · prime bucket counts ·
references stable, iterators not
```

49. Java HashMap: 0.75, treeify at 8

Java's table is a power-of-two array of bins; each bin is a linked list that **converts to a red-black tree** once it holds 8 entries (and the table has at least 64 bins), and converts back at 6. That turns the worst case from O(n) to O(log n) — the direct answer to the hash-flooding attack of Topic 79.

```
// The constants worth knowing (java.util.HashMap):
//   DEFAULT_INITIAL_CAPACITY  = 16
//   DEFAULT_LOAD_FACTOR       = 0.75f
//   TREEIFY_THRESHOLD         = 8      bin becomes a red-black tree
//   UNTREEIFY_THRESHOLD       = 6      and back again (hysteresis, T23)
//   MIN_TREEIFY_CAPACITY      = 64     below this, resize instead
//
// The spread function -- Topic 10's repair, in the container:
static final int hash(Object key) {
    int h;
    return (key == null) ? 0 : (h = key.hashCode()) ^ (h >>> 16);
}
```

Because the table is a power of two and growth is a doubling, an entry's new index is either `i` or `i + oldCap` — decided by a single bit. Java exploits that to split each bin into a "lo" and "hi" list without rehashing any key.

THE KEY INSIGHT

Treeification is a *bounded degradation* strategy: accept O(log n) in the tail rather than defend the hash function. It costs nothing in the common case, since a bin reaches 8 entries with probability $\sim 6 \times 10^{-8}$ under a decent hash.

THE TRAP

Treeification requires keys to be `Comparable`, or it falls back to comparing identity hash codes — so a badly-hashing non-comparable key still degrades. And `hashCode`/`equals` must agree (Topic 83); mutable keys break both regardless.

```
Java HashMap — power-of-two bins, load 0.75 · treeify at 8 bins/64 capacity · h ^ (h >>> 16) spread
· resize splits bins by one bit
```

50. Python dict: compact and ordered

Since 3.6 the dict is the compact two-array layout of Topic 44: a sparse array of indices plus a dense array of `(hash, key, value)` entries in insertion order. Iteration is O(n) and ordered — guaranteed by the language since 3.7 — and the dict is 20–25% smaller than the pre-3.6 layout.

```
>>> import sys
>>> d = {}; sys.getsizeof(d)              # 64 bytes, empty
>>> d = dict.fromkeys(range(1000)); sys.getsizeof(d)   # ~36 KB, ~36 B/entry
>>> hash("abc")                           # DIFFERENT on every process start
-2039875923834234    # PYTHONHASHSEED randomization -- Topic 80
>>> hash(1) == hash(1.0) == hash(True)    # True: 1 == 1.0 == True
True                                       # so {1: 'a'}[True] is 'a'
```

Growth happens when the dense array is 2/3 full, and the new size is `used * 3` rounded up to a power of two — sized by *live* entries, so a dict that has had many deletions actually shrinks on its next resize. Instances also use **key-sharing dicts** (PEP 412): objects of the same class share one keys table, so `__dict__` costs only the values array.

THE KEY INSIGHT

Python's dict is the reference implementation of "the hash table is an index over an ordered array". Ordering was a free by-product of a memory optimisation — not a feature that was designed in.

THE TRAP

`hash(1) == hash(1.0) == hash(True)`, and they compare equal, so `{1: 'int', True: 'bool'}` is a one-element dict. Symptom: a counter keyed on mixed `int`/`bool`/`float` that silently merges categories. Topic 84.

```
Python dict — compact ordered, ~36 B/entry · resize at 2/3 full to used×3 · per-process hash seed · int/float/bool keys unify
```

51. PHP arrays: packed vs hashed

A PHP array is one type doing two jobs. When keys are integers in ascending order from 0, the `zend_array` is **packed**: since PHP 8.2 it stores a bare array of 16-byte zvals with no keys and no hash slots, and element access is pure offset arithmetic — a C array in all but name. Introduce a string key, or a non-sequential integer key, and it converts to a **hashed** representation with 32-byte Buckets plus a 4-byte index slot each.

```
<?php
$a = [1, 2, 3];              // PACKED: 3 zvals, ~16 B each
$a[] = 4;                    // still packed
$a["x"] = 5;                 // -> converted to a real hash table, ~36 B/elem
unset($a[1]); $a[] = 9;      // holes are kept; nNextFreeElement does not
                             // go backwards -- keys become 0,2,3,"x",4

$b = ["10" => "a", 10 => "b"];   // ONE element: "10" is coerced to int 10
var_dump(array_keys($b));        // [0 => int(10)]
```

Iteration is always insertion order, because values live in that ordered `arData`/`arPacked` array (Topic 44). The hash function is DJBX33A (Topic 11's djb2) with a per-process seed, and collisions are resolved by an index chain stored in the negative offsets of the same allocation.

THE KEY INSIGHT

The single biggest PHP memory lever is keeping arrays packed: sequential integer keys, no `unset` of middle elements, and `array_values()` after filtering. A packed array costs roughly **16 bytes per element against ~36**.

THE TRAP

Numeric string keys are silently coerced: `"10"` becomes `10`, but `"010"` and `"1e2"` stay strings. Symptom: an array keyed by IDs read from a database where `"07"` and `7` are different keys, so a lookup misses exactly for zero-padded ids.

```
PHP array — packed ~16 B/element (8.2+), hashed ~36 B · always insertion-ordered · DJBX33A + per-process seed · numeric-string keys coerced to int
```

52. JavaScript Object vs Map

They are not two APIs over one structure. A plain `Object` is optimised by V8 as a *shape* (hidden class) with fixed offsets, falling back to a real dictionary only when you delete keys or add too many dynamically. A `Map` is an ordered hash table from the start, with real keys of any type.

```
const o = {};  o[1] = 'a';  o['1'] = 'b';   // ONE key: "1" -- object keys
Object.keys(o);                              // are strings (or symbols)
// ["1"]

const m = new Map(); m.set(1,'a'); m.set('1','b');  // TWO keys: 1 and "1"
m.size;                                              // 2

const o2 = { b: 1, 2: 'x', a: 3, 1: 'y' };
Object.keys(o2);   // ["1","2","b","a"]  <- integer-like keys FIRST,
                   //    in ascending order, then strings in insertion order
```

That ordering rule is specified, not incidental, and it is why `JSON.stringify` of an object with numeric-looking keys reorders them. A `Map` preserves pure insertion order, is faster for frequent add/delete, and does not collide with `Object.prototype` names such as `constructor` or `__proto__`.

THE KEY INSIGHT

Use `Object` for records with a fixed known shape; use `Map` for a keyed collection whose keys are data. The two differ in key type, ordering and prototype exposure — all three are correctness issues, not performance ones.

THE TRAP

`obj[userInput]` where the input is `"__proto__"` or `"constructor"`. Symptom: prototype pollution — an assignment that changes behaviour for every object in the process. `Map`, or `Object.create(null)`, removes the class of bug entirely.

`Object — string/symbol keys, integer-like keys sort first, prototype chain exposed · Map — any key type, true insertion order, no prototype, .size in O(1)`

53. Go maps: Swiss internals and random iteration

Since Go 1.24 the built-in map is a Swiss table (Topic 38) with 8-slot groups and SWAR control-word matching, a maximum load of 7/8, and no overflow buckets. Three language-level behaviours matter more than the internals.

```
m := map[string]int{"a": 1, "b": 2, "c": 3}

// 1. Iteration order is deliberately RANDOMISED per range statement.
for k := range m { fmt.Println(k) }   // a different order every run

// 2. You cannot take the address of an element.
//    p := &m[k]                      // compile error
//    m[k].Field = 1                  // compile error for struct values
type S struct{ N int }
ms := map[string]S{}
// ms["x"].N = 5                      // invalid; read, modify, write back:
v := ms["x"]; v.N = 5; ms["x"] = v

// 3. Concurrent read+write is a FATAL error, not a data race you might
//    survive:  "fatal error: concurrent map writes"  -- kills the process.
```

THE KEY INSIGHT

Go randomises iteration order *on purpose*, to stop programs depending on it. Every other language in this section has the same lack of guarantee; Go is the only one that enforces it.

THE TRAP

A map read concurrent with a write is detected by the runtime and **crashes the whole process**, with no recovery. Symptom: a production panic under load that never reproduces in tests. Use `sync.RWMutex` or `sync.Map` (Topic 87).

```
Go map — Swiss layout, 8-slot groups, load 7/8 · randomised iteration · no element addresses · concurrent write = fatal, not UB
```

54. Rust HashMap: SipHash by default

Rust ships the safest defaults of any language here and makes them swappable. `std::collections::HashMap` is hashbrown (a Swiss table) with **SipHash-1-3 keyed by a per-process random seed** — so hash flooding (Topic 79) is off by default, at a cost of roughly 1 ns per byte plus fixed overhead.

```
use std::collections::HashMap;
let mut m: HashMap<u64, i32> = HashMap::new();

// The entry API: ONE hash, one probe, for read-modify-write.
*m.entry(k).or_insert(0) += 1;          // not: if m.contains_key -> m[k] += 1

// Opting out of SipHash when input is trusted -- 2-4x faster for small keys:
use rustc_hash::FxHashMap;              // or ahash::AHashMap
let mut fast: FxHashMap<u64, i32> = FxHashMap::default();
```

The borrow checker enforces Topic 41's stability rule at compile time: you cannot hold `&mut` into the map and insert at the same time, so the dangling-reference bug is unrepresentable rather than documented.

THE KEY INSIGHT

The `entry` API is the general fix for the two-lookup anti-pattern that appears in every language: `contains_key` then `insert` hashes twice. C++'s equivalent is `try_emplace`, Python's is `dict.setdefault` or `defaultdict`.

THE TRAP

Swapping to FxHash "for speed" on a map keyed by user-supplied strings reintroduces hash flooding — FxHash is a fast non-keyed mixer with easily constructed collisions. Only use it for internal, trusted keys.

```
Rust HashMap — Swiss + SipHash-1-3, randomly keyed · entry API = one hash · FxHash/ahash 2–4× faster but attackable
```

55. Redis hashes: listpack to hashtable

Redis's `HASH` type has two encodings. Small hashes are a **listpack** — a flat, contiguous, sequentially scanned byte array with no hash function at all — because at ten fields a linear scan of one cache line beats a hash table (Topic 6's small-n rule, in production). It converts to a real `dict` when either threshold is crossed.

```
127.0.0.1:6379> CONFIG GET hash-max-listpack-entries
1) "hash-max-listpack-entries"
2) "128"                        # more than 128 fields  -> convert
127.0.0.1:6379> CONFIG GET hash-max-listpack-value
2) "64"                         # any value over 64 bytes -> convert

127.0.0.1:6379> HSET booth:42 terminal EXE-BUS-01 status active
127.0.0.1:6379> OBJECT ENCODING booth:42
"listpack"
# The conversion is ONE-WAY: deleting fields does not convert back.
```

The top-level keyspace is the incrementally rehashed two-table dict of Topic 24, which is why `KEYS *` is O(n) and forbidden in production while `SCAN` exists: SCAN's reverse-binary cursor tolerates a rehash mid-iteration, guaranteeing that elements present throughout are returned at least once, though possibly more than once.

THE KEY INSIGHT

Redis chooses the encoding per key, at run time, from measured thresholds. That is the same decision as Topic 6's "n < 30 → linear scan", made automatic — and it is why a million tiny hashes cost far less than a million small dicts would.

THE TRAP

Encoding conversion is one-way. One 65-byte value converts a hash to the dict encoding *permanently*, roughly tripling its memory, and deleting that field does not convert it back. Symptom: memory that steps up and never comes down after a single oversized write.

```
Redis hash — listpack below 128 entries and 64-byte values, then dict · conversion is one-way · use SCAN, never KEYS · HSCAN may return duplicates
```

56. Choosing capacity up front: reserve

Every rehash in Topic 22 is avoidable if you know the size. The call differs by language and two of them mean different things by the same argument.

Language	Call	Argument means
C++ `unordered_map`	`m.reserve(n)`	**Elements** — buckets sized to n/ max_load_factor
C++ (older code)	`m.rehash(b)`	**Buckets** — a common off-by-load-factor bug
Java	`new HashMap<>(cap)`	Buckets: pass `(int)(n/0.75f)+1`
Go	`make(map[K]V, n)`	Elements (a hint)
Rust	`HashMap::with_capacity(n)`	Elements
Python	—	No API; build from an iterable so CPython can presize

```
// Measured effect on 4,000,000 inserts of uint64 -> int:
//   no reserve : 22 rehashes, worst single insert ~330 ms, total ~1.9 s
//   reserve(4M): 0 rehashes,  worst single insert ~1 us,   total ~0.9 s
// Roughly 2x on total time, and a 300,000x improvement in the tail.
```

THE KEY INSIGHT

`reserve` is the cheapest performance fix in this book: one line, no design change, and it removes an entire class of latency spike. If you know n even approximately, say so.

THE TRAP

Over-reserving. `reserve(10'000'000)` for 500 entries allocates the full slot array immediately and makes iteration O(m) forever (Topic 20). Reserve what you expect, not what you fear.

```
reserve — removes all rehashes · ~2× total insert throughput at 4M · tail latency from 330 ms to microseconds · check whether the argument is elements or buckets
```

PRACTICE SET — Topics 48–56

A. Conceptual

1. Name the two clauses in the C++ standard that force `unordered_map` to be node-based, and say what each one would cost to give up.
2. Why does libstdc++ use prime bucket counts when almost every other implementation uses powers of two?
3. Java treeifies a bin at 8 entries and untreeifies at 6. Which earlier topic is that gap an instance of?
4. Explain how Java can split a bin on resize without rehashing any key.
5. Python's dict became ordered as a side effect of what change?
6. In PHP, name the three things that convert a packed array into a hashed one, and the per-element cost of that conversion.
7. Give two correctness differences (not performance) between a JavaScript `Object` and a `Map`.
8. Go randomises map iteration order. What class of bug is that design choice preventing?
9. Why is Rust's default hash slower than FxHash, and when is that trade the wrong one?
10. Redis converts a hash from listpack to dict but never back. What operational symptom does that produce?
11. Give the one-line difference between C++'s `reserve(n)` and Java's `new HashMap<>(n)`.

B. Tracing and analysis

1. You need a Java `HashMap` to hold 1,000,000 entries without ever resizing. What capacity do you pass, and what is the actual table size that results?
2. A Java bin holds 8 entries at $\alpha = 0.75$ and the table has 128 bins. Does it treeify? Justify from the constants.
3. A Python dict holds 1,000 entries at ~36 B each. Compute its size, then compute the size of the equivalent `unordered_map<int,int>` in C++ at 48 B/entry plus a 2,048-slot pointer array.
4. A PHP array holds 100,000 sequential integer keys. Compute its memory when packed (16 B/element) and after one string key is added (36 B/element). State the increase.
5. Given `const o = {b:1, 2:'x', a:3, 1:'y'}`, write out `Object.keys(o)` in the exact specified order and explain each position.
6. A Go map holds 1,000,000 `int64 → int64` entries at load 7/8. How many slots does it allocate, and how many bytes including one control byte per slot?
7. Inserting 4,000,000 entries without `reserve` costs 22 rehashes and ~1.9 s. With `reserve` it is ~0.9 s. What fraction of the original time was rehashing, and what was the tail improvement?

C. Code tasks

1. **Measure the specification.** Insert 4,000,000 `uint64 → int` pairs into `std::unordered_map` and print `bucket_count()`, `load_factor()` and `max_load_factor()`. Confirm the bucket count is prime. Then repeat with `reserve(4'000'000)` first and compare total time.
2. **Break it and observe.** Write a Java (or C++) key class whose `hashCode` returns a constant. Insert 100,000 entries and time lookups. Then implement `Comparable` on the key and time again. Explain the difference using the treeify threshold, and state what the time would be in a language without treeification.

3. **Break it and observe.** In Python, build `d = {1: 'int', True: 'bool', 1.0: 'float'}` and print `d`, `len(d)` and `d[1]`. Then do the same in JavaScript with an `Object` using keys `1` and `"1"`, and again with a `Map`. Record all three results.
4. **Observe the encoding switch.** In Redis, `HSET` a hash with 100 small fields and check `OBJECT ENCODING` and `MEMORY USAGE`. Add one field with a 100-byte value, then check both again. Then `HDEL` that field and check a third time.
5. **Break it and observe.** In PHP, build an array of 100,000 sequential integers and print `memory_get_usage()`. Then `unset()` every tenth element and print again. Then `$a = array_values($a)` and print a third time.
6. **Break it and observe.** Write a Go program with one goroutine writing to a map and another reading, with no lock. Run it 10 times. Record what the runtime prints and whether the process survives. Then add `sync.RWMutex` and run again.

ANSWER KEY — Topics 48–56

A. Conceptual

1. **Reference stability across rehash** and the **bucket interface** (`bucket_count`, `bucket(k)`, local iterators). Giving up the first allows a flat layout (2× faster, half the memory); giving up the second removes the requirement to expose buckets at all.
2. Because `std::hash` for integers is the **identity function**, so the low bits of the hash carry the key's structure. A prime modulus mixes all bits; a mask would expose the weakness directly (Topic 8).
3. **Hysteresis** (Topic 23). A single threshold would convert and revert on every insert/erase pair at the boundary.
4. Because capacity doubles and is a power of two, the new index is `i` or `i + oldCap`, determined by the **single new high bit** of the stored hash. Java splits each bin into a "lo" and "hi" list by testing that one bit.
5. The **compact two-array layout** adopted for **memory** (20–25% smaller). Order came free with the dense entry array and was only guaranteed by the language afterwards.
6. A **string key**, a **non-sequential integer key**, or a key **below the current next-free index**. Cost: roughly **16 B/element → ~36 B/element**.
7. Any two of: **Object keys are coerced to strings** (1 and "1" are the same key), **integer-like keys iterate first in ascending order**, and **Object inherits from** `Object.prototype` so `__proto__` and `constructor` collide with real data.
8. Programs **depending on iteration order** — code that passes in testing and breaks after an unrelated change to the hash seed, the table size or the insertion sequence (Topic 81).
9. Because it is **SipHash-1-3 with a per-process random key**, which resists constructed collisions (Topic 79). The trade is wrong when keys are **internal and trusted** and hashing shows up in the profile — integer ids, interned symbols, indices.
10. **Memory that steps up and never comes down.** A single oversized field triples the hash's footprint permanently; deleting it does not convert back.
11. C++ `reserve(n)` takes **elements** and sizes buckets to n/max_load_factor; Java's constructor takes **buckets**, so you must pass `n/0.75 + 1` yourself.

B. Tracing and analysis

1. Pass `(int)(1'000'000 / 0.75f) + 1 = 1,333,334`. Java rounds up to the next power of two: **2,097,152 bins**, giving α = 0.477 and no resize.
2. **No.** `MIN_TREEIFY_CAPACITY` is 64 and the table has 128 bins, so capacity is fine — but the bin must reach `TREEIFY_THRESHOLD = 8` *and* the check happens when a **ninth** element is appended. At exactly 8 it is still a list; the next insert converts it.
3. Python: 1,000 × 36 = **36 KB**. C++: 1,000 × 48 = 48 KB plus 2,048 × 8 = 16 KB = **64 KB** — about **1.8×** the Python dict, for a container that is unordered.
4. Packed: 100,000 × 16 = **1.6 MB**. Hashed: 100,000 × 36 = **3.6 MB**. A **2.25× increase** from adding one string key.
5. **["1", "2", "b", "a"]**. Integer-like keys come first in *ascending numeric* order (1 then 2), then string keys in *insertion* order (b was inserted before a).
6. Capacity 7m/8 ≥ 10^6 → m ≥ 1,142,858 → **m = 2^{21} = 2,097,152 slots**. Bytes = 2,097,152 × (8 + 8 + 1) = **35.7 MB**.

7. Rehashing was **(1.9 − 0.9)/1.9 = 53%** of total insert time. The tail improved from **~330 ms to ~1 µs — about 300,000×**.

C. Code tasks — what to verify

1. Expect `bucket_count()` = **5,967,347 (prime)**, load ~0.67, `max_load_factor()` = **1.0**. With `reserve`, expect total time to roughly **halve** and the bucket count to be chosen once.
2. With a constant `hashCode` and a `Comparable` key, Java treeifies and lookups cost **O(log n) ≈ 17 compares**. Without `Comparable`, it falls back to identity-hash ordering and is much worse. In C++ or Python, where there is no treeification, the same key class gives **O(n) — 100,000 comparisons per lookup**, which is Topic 79's attack in one class.
3. Python prints `{1: 'float'}` **with** `len(d) == 1` — all three keys are equal and hash equal, so the last value wins while the *first* key object is kept. JavaScript's Object gives **one key** `"1"`; the `Map` gives **two entries**, `1` and `"1"`.
4. Expect `"listpack"` first, `"hashtable"` **after the 100-byte value**, with `MEMORY USAGE` roughly **2–3×** higher, and **still** `"hashtable"` **after the** `HDEL` — the encoding never returns.
5. Expect roughly **1.6 MB packed**, then a **jump** after the `unset` calls (the array converts to hashed and keeps holes), then a **fall back toward the packed figure** after `array_values()`, which renumbers the keys 0..n−1 and lets the array repack.
6. Expect `fatal error: concurrent map read and map write` (or `concurrent map writes`) and a **dead process** — not a panic you can `recover()` from — on most of the ten runs. With the mutex, all ten runs complete.

HANDNOTE SUMMARY CARD — Standard libraries

```
THE STANDARD LIBRARY LAYER (48-56)
================================================================================
LANG / RUNTIME     LAYOUT              LOAD    ORDER          NOTES
 C++ unordered_map chaining, nodes     1.00    none           refs stable; prime
                                                              bucket counts
 absl flat_hash    Swiss, flat         0.875   none           refs INVALID
 Java HashMap      chaining + RB tree  0.75    none           treeify 8 / untree 6
                                                              MIN_TREEIFY_CAP 64
 Python dict       compact 2-array     0.667   INSERTION      ~36 B/entry, seeded
 PHP array         compact 2-array     1.00    INSERTION      packed 16 B vs 36 B
 JS Object         hidden class/dict   -       ints first!    keys are strings
 JS Map            ordered hash        -       INSERTION      any key type
 Go map (1.24+)    Swiss, 8-slot SWAR  0.875   RANDOMISED     no &m[k]; fatal on
                                                              concurrent write
 Rust HashMap      hashbrown Swiss     0.875   none           SipHash-1-3, keyed
 Redis hash        listpack -> dict    -       insertion      128 entries / 64 B

CONSTANTS WORTH MEMORISING
  Java: cap 16, load 0.75, TREEIFY 8, UNTREEIFY 6, MIN_TREEIFY_CAPACITY 64
        spread: h ^ (h >>> 16);  resize splits a bin into i and i+oldCap
  Python: resize at 2/3 used, new size = used*3 rounded to a power of two
  PHP: DJBX33A + per-process seed; "10" -> int 10, but "010" stays a string
  Redis: hash-max-listpack-entries 128, hash-max-listpack-value 64 (ONE-WAY)
  libstdc++: bucket_count is PRIME (4M inserts -> 5,967,347)

THE TWO-LOOKUP ANTI-PATTERN AND ITS FIX, PER LANGUAGE
  C++     m.try_emplace(k, v)  /  auto [it, ok] = m.insert({k, v});
  Rust    *m.entry(k).or_insert(0) += 1;
  Python  d.setdefault(k, []) / collections.defaultdict / Counter
  Java    m.computeIfAbsent(k, x -> new ArrayList<>())
  Go      v, ok := m[k]        // the comma-ok form is already one lookup

reserve / capacity -- CHECK THE UNITS
  C++  reserve(n)            n = ELEMENTS
  C++  rehash(b)             b = BUCKETS      <- classic off-by-load-factor bug
  Java new HashMap<>(cap)    cap = BUCKETS -> pass (int)(n/0.75f)+1
  Go   make(map[K]V, n)      n = elements (hint)
  Rust HashMap::with_capacity(n)   n = elements
  measured, 4M inserts: no reserve = 22 rehashes, 1.9 s, worst insert 330 ms
                        reserve    =  0 rehashes, 0.9 s, worst insert ~1 us

CROSS-LANGUAGE TRAPS
  Python  hash(1)==hash(1.0)==hash(True) -> {1:'a', True:'b'} has ONE entry
  JS      obj[1] and obj["1"] are the SAME key; obj["__proto__"] pollutes
  PHP     "07" and 7 are DIFFERENT keys; "7" and 7 are the SAME key
  Go      &m[k] does not compile; concurrent read+write kills the process
  Java    treeify needs Comparable keys, else it degrades anyway
  Redis   KEYS * is O(n) and blocks; use SCAN (may return duplicates)
```

PART III

Using It

The fifteen problem patterns that account for nearly every hash-table interview question, and the seven composite designs that combine a map with a second structure. Sections 8–9.

SECTION 8 — PROBLEM PATTERNS

57. Frequency counting

The base pattern from which half of this section is built: replace "how many times does x appear?" — an O(n) scan per query — with one pass that builds a map from value to count, after which every query is O(1).

```
std::unordered_map<int,int> freq;
for (int x : a) ++freq[x];              // operator[] default-constructs 0

// Most frequent element:
int best = 0, bestc = 0;
for (auto& [v, c] : freq) if (c > bestc) { bestc = c; best = v; }

// Anagram check -- two counts, or one count and one decrement:
bool anagram(const std::string& a, const std::string& b) {
    if (a.size() != b.size()) return false;
    std::unordered_map<char,int> f;
    for (char c : a) ++f[c];
    for (char c : b) if (--f[c] < 0) return false;   // one pass, early exit
    return true;
}
```

```
a = [3, 1, 3, 7, 3, 1]
                  build              query
 pass 1:  {3:1}{3:1,1:1}{3:2,...}   freq[3] -> 3   O(1)
 total:   O(n) time, O(k) space     freq[9] -> 0   O(1) (and INSERTS 9!)
          k = distinct values
```

THE KEY INSIGHT

The decrement variant is strictly better than building two maps: one map, one pass over each string, and it exits early on the first mismatch instead of comparing complete tallies.

THE TRAP

`freq[x]` on a *query* inserts a zero entry (Topic 17). A loop that checks `if (freq[x] > 0)` over 10^6 absent keys grows the map to 10^6 entries. Use `freq.count(x)` or `freq.find(x)` for reads.

```
frequency map — build O(n) · query O(1) · space O(k) distinct · use int[26] or int[256] when the
alphabet is small (Topic 71)
```

58. The seen-set

"Have I met this before?" — duplicate detection, cycle detection, visited marking. A set, not a map, because there is no value to store; the membership bit *is* the answer.

```
// Contains duplicate, one pass, early exit:
bool has_dup(const std::vector<int>& a) {
    std::unordered_set<int> seen;
    for (int x : a)
        if (!seen.insert(x).second) return true;    // ONE hash per element
    return false;
}
// First repeating character, with its index:
int first_repeat(const std::string& s) {
    std::unordered_set<char> seen;
    for (int i = 0; i < (int)s.size(); ++i)
        if (!seen.insert(s[i]).second) return i;
    return -1;
}
```

THE KEY INSIGHT

`insert().second` answers "was it there?" and performs the insertion in a single probe sequence. Every `if (s.count(x)) ... else s.insert(x)` is a doubled cost for the same information.

THE TRAP

Building the whole set before checking. `set<int> s(a.begin(), a.end()); return s.size() != a.size();` is correct but processes all n elements even when the duplicate is at index 1, and it allocates the full set. The early-exit loop is O(1) space in the best case.

```
seen-set — O(n) time, O(k) space · early exit on the first repeat · prefer a sorted-array pass
when memory is tight and O(n log n) is acceptable
```

59. Complement lookup: the Two Sum family

The single most-asked pattern. Instead of testing every pair — $O(n^2)$ — walk once and ask the map for the *complement* that would complete the answer. The map turns "does a partner exist?" into an O(1) question.

```
std::vector<int> two_sum(const std::vector<int>& a, int target) {
    std::unordered_map<int,int> pos;             // value -> index
    for (int i = 0; i < (int)a.size(); ++i) {
        auto it = pos.find(target - a[i]);        // ask BEFORE inserting
        if (it != pos.end()) return {it->second, i};
        pos[a[i]] = i;
    }
    return {};
}
```

```
a = [2, 7, 11, 15], target = 9
 i=0  need 9-2=7   map {}            miss -> insert 2:0
 i=1  need 9-7=2   map {2:0}         HIT  -> answer (0, 1)
 one pass, one hash per element, O(n) instead of O(n^2)
```

THE KEY INSIGHT

Query before insert. It is what prevents an element pairing with itself, and it is why no "i ≠ j" check is needed anywhere in the loop.

THE TRAP

With duplicates, `pos[a[i]] = i` keeps only the *last* index. For "return any pair" that is fine; for "count all pairs summing to target" you need `unordered_map<int,int>` of counts and must handle `a[i] == target - a[i]` separately, or you will double-count by C(c,2) instead of counting it once.

```
complement lookup — O(n) time, O(n) space, replaces O(n²) · extends to 3Sum (fix one element,
two-sum the rest: O(n²))
```

60. Last-seen index

Store *where* you saw a value, not just that you did. That one change turns membership into distance, which answers a whole family of "within k", "longest between", "distance to previous" questions.

```
// Contains a duplicate within distance k:
bool near_dup(const std::vector<int>& a, int k) {
    std::unordered_map<int,int> last;             // value -> last index
    for (int i = 0; i < (int)a.size(); ++i) {
        auto it = last.find(a[i]);
        if (it != last.end() && i - it->second <= k) return true;
        last[a[i]] = i;                            // overwrite: nearest wins
    }
    return false;
}
```

THE KEY INSIGHT

Overwriting with the newest index is correct precisely because you scan left to right: the nearest previous occurrence is always the most recent one, so older indices can never produce a smaller distance.

THE TRAP

Keeping a vector of all indices per value "in case you need them". That is O(n) space with worse constants and O(occurrences) per query, for a problem the single-index version solves in O(1). Only keep the list when the question asks for pairs, not for the nearest.

```
last-seen index — O(n) time, O(k) space · one map, one overwrite per element · basis of the
sliding-window start pointer in Topic 64
```

61. Canonical-key grouping

Group items that are "the same" under some equivalence by computing a **canonical form** and using it as the map key. Anagrams, shifted strings, isomorphic shapes, normalised phone numbers — all the same pattern; only the canonicaliser changes.

```
// Group anagrams. Canonical form = the sorted letters.
std::vector<std::vector<std::string>> group_anagrams(
        const std::vector<std::string>& ws) {
    std::unordered_map<std::string, std::vector<std::string>> g;
    for (const auto& w : ws) {
        std::string key = w;
        std::sort(key.begin(), key.end());        // O(L log L)
        g[key].push_back(w);
    }
    std::vector<std::vector<std::string>> out;
    for (auto& [k, v] : g) out.push_back(std::move(v));
    return out;
}
// Cheaper canonical form for a 26-letter alphabet: the count vector
// rendered as a string, e.g. "1#0#2#..." -- O(L) instead of O(L log L).
```

```
"eat" -> "aet" --+
"tea" -> "aet" --+--> g["aet"] = [eat, tea, ate]
"ate" -> "aet" --+
"tan" -> "ant" -----> g["ant"] = [tan, nat]
```

THE KEY INSIGHT

The canonical form must satisfy exactly one property: two items are equivalent **if and only if** their forms are equal. Any form with that property works; choose the cheapest to compute.

THE TRAP

A canonical form that is *not* injective — for example summing character codes for anagrams, so `"ad"` and `"bc"` collide. Symptom: groups that contain items that are not equivalent, and no error anywhere. This is Topic 83's hash/equals contract restated at the algorithm level.

```
canonical grouping — O(n × canon cost) · space O(total input) · sorted-string key O(L log L),
count-vector key O(L)
```

62. Prefix sum plus map

The subarray-sum family. Since sum(i..j) = P(j) − P(i−1), asking "is there a subarray summing to k ending here?" becomes "have I seen the prefix P(j) − k before?" — a map lookup, replacing an $O(n^2)$ double loop.

```
int subarrays_summing_to_k(const std::vector<int>& a, int k) {
    std::unordered_map<long long,int> seen{{0, 1}};   // empty prefix: crucial
    long long run = 0;
    int count = 0;
    for (int x : a) {
        run += x;
        auto it = seen.find(run - k);
        if (it != seen.end()) count += it->second;     // all earlier starts
        ++seen[run];
    }
    return count;
}
```

```
a = [1, 2, 3], k = 3
 run=1  need -2  miss     seen {0:1, 1:1}
 run=3  need  0  HIT (1)  count=1        <- the whole prefix [1,2]
 run=6  need  3  HIT (1)  count=2        <- [3]
 the {0:1} seed is what lets a prefix that IS the answer be counted
```

THE KEY INSIGHT

Seed the map with `{0: 1}`. Without it, every subarray that starts at index 0 is missed — the single most common bug in this pattern, and it passes most small test cases.

THE TRAP

This pattern does **not** transfer to "longest subarray with sum ≤ k" or any inequality: a map answers equality only. Inequalities need a sorted structure or a monotonic deque. Reaching for a hash map there is a common dead end.

```
prefix sum + map — O(n) time, O(n) space · equality queries only · store counts for "how many", first-index for "longest"
```

63. Prefix state: parity, remainder, balance

The prefix need not be a sum. Any value you can accumulate and compare for equality works — and the powerful cases compress the state so that many prefixes collide *on purpose*.

Problem	Prefix state	Map key
Subarray sum divisible by k	running sum	`sum mod k` (k keys, not n)
Equal 0s and 1s	+1 / −1 balance	the balance
Even count of every vowel	5-bit parity mask	the mask (32 keys)
Longest balanced parentheses	depth	depth → first index

```cpp
// Longest substring where every vowel appears an even number of times.
int longest_even_vowels(const std::string& s) {
    std::unordered_map<int,int> first{{0, -1}};    // mask -> earliest index
    int mask = 0, best = 0;
    const std::string V = "aeiou";
    for (int i = 0; i < (int)s.size(); ++i) {
        size_t p = V.find(s[i]);
        if (p != std::string::npos) mask ^= 1 << p;   // toggle that parity bit
        auto it = first.find(mask);
        if (it != first.end()) best = std::max(best, i - it->second);
        else first[mask] = i;                          // keep the EARLIEST
    }
    return best;
}
```

THE KEY INSIGHT

For "longest", store the **first** index a state was seen and never overwrite. For "count", store how *many* times. For "exists", store nothing but membership. The pattern is one line different in each case.

THE TRAP

`sum % k` in C++ is negative for negative sums, so `-3 % 5 == -3`, and prefixes that should match do not. Normalise with `((sum % k) + k) % k`. Symptom: correct answers on all-positive tests, wrong answers whenever the input contains a negative number.

```
prefix state — O(n) time · space O(distinct states), often O(k) or O(2^bits) rather than O(n) · seed with the empty-prefix state
```

64. Sliding window with a frequency map

Two pointers plus a map of what is currently inside the window. The map makes "is the window still valid?" an O(1) question, so each element enters and leaves exactly once and the whole scan is O(n) despite the nested loop.

```
// Longest substring without repeating characters.
int longest_unique(const std::string& s) {
    std::unordered_map<char,int> last;     // char -> last index seen
    int start = 0, best = 0;
    for (int i = 0; i < (int)s.size(); ++i) {
        auto it = last.find(s[i]);
        if (it != last.end() && it->second >= start)
            start = it->second + 1;         // jump, never step back
        last[s[i]] = i;
        best = std::max(best, i - start + 1);
    }
    return best;
}
```

```
s = a b c a b b
 i:  0 1 2 3 4 5
       [a b c]        start=0
         [b c a]      i=3: 'a' last at 0 >= start -> start = 1
           [c a b]    i=4
               [b]    i=5: 'b' last at 4 -> start = 5
 start only ever moves RIGHT -> total work O(n)
```

THE KEY INSIGHT

The `>= start` guard is what makes this correct: a stale index from before the window must be ignored, not obeyed. Without it, `start` can jump backwards and the window becomes invalid.

THE TRAP

For the counting variant ("at most k distinct"), you must **erase** keys whose count reaches zero, not merely decrement. Otherwise `map.size()` keeps counting characters that have already left the window and the answer is silently too small.

```
sliding window + map — O(n) time, O(alphabet) space · each element enters and leaves once · guard stale indices with >= start
```

65. Two-map bijection

Word patterns, isomorphic strings, and any "one-to-one correspondence" check needs **two** maps. One map proves a function exists; the second proves it is injective.

```
bool isomorphic(const std::string& a, const std::string& b) {
    if (a.size() != b.size()) return false;
    std::unordered_map<char,char> fwd, rev;
    for (size_t i = 0; i < a.size(); ++i) {
        auto [f, f_new] = fwd.insert({a[i], b[i]});
        auto [r, r_new] = rev.insert({b[i], a[i]});
        if (f->second != b[i] || r->second != a[i]) return false;
    }
    return true;
}
// "egg" -> "add"  true.   "foo" -> "bar"  false: o->a and o->r.
// "badc" -> "baba" false ONLY because of the reverse map: d->b and b->b.
```

THE KEY INSIGHT

One map gives a mapping; two maps give a bijection. Ask yourself which the problem requires — "isomorphic" and "pattern matches" always mean the latter, and the single-map solution passes about 80% of test cases.

THE TRAP

The single-map version fails only on inputs where two distinct source characters map to the same target. Those are exactly the cases hand-written tests omit, so this bug reaches production and interviews alike.

```
two-map bijection — O(n) time, O(alphabet) space · one map = function, two maps = bijection ·
same idea as a foreign key with a unique index
```

66. Meet in the middle

When brute force is 2^n and n is around 40, split the input in half, enumerate each half's $2^{n/2}$ possibilities, store one half in a map, and look up the complement for the other. $2^{40} = 10^{12}$ becomes $2 \times 2^{20} = 2{\times}10^6$.

```
// Count subsets of `a` (n <= 40) that sum to target.
long long count_subsets(const std::vector<int>& a, long long target) {
    int n = a.size(), h = n / 2;
    std::unordered_map<long long,int> left;
    for (int m = 0; m < (1 << h); ++m) {
        long long s = 0;
        for (int i = 0; i < h; ++i) if (m >> i & 1) s += a[i];
        ++left[s];
    }
    long long total = 0;
    for (int m = 0; m < (1 << (n - h)); ++m) {
        long long s = 0;
        for (int i = 0; i < n - h; ++i) if (m >> i & 1) s += a[h + i];
        auto it = left.find(target - s);
        if (it != left.end()) total += it->second;
    }
    return total;
}
```

THE KEY INSIGHT

The map is what makes the join O(1) per candidate. Meet in the middle without a hash table is a sort plus binary search — $O(2^{n/2} \cdot n/2)$ instead of $O(2^{n/2})$, and only worth it when you need order.

THE TRAP

Memory. At n = 44 each half is 2^{22} = 4.2M entries; at ~40 B each that is 170 MB and the build itself takes hundreds of milliseconds of rehashing. `reserve(1 << h)` before the loop (Topic 56).

meet in the middle — time $O(2^{n/2})$, space $O(2^{n/2})$ · viable to n ≈ 40–44 · always `reserve`

67. Graphs as maps

When vertices are not 0..n−1 integers — they are strings, pairs, coordinates or arbitrary ids — the adjacency structure is a map of vectors and the visited set is a hash set. The algorithm does not change; only the container does.

```
using Graph = std::unordered_map<std::string, std::vector<std::string>>;

int bfs(const Graph& g, const std::string& src, const std::string& dst) {
    std::unordered_set<std::string> seen{src};
    std::queue<std::pair<std::string,int>> q;
    q.push({src, 0});
    while (!q.empty()) {
        auto [u, d] = q.front(); q.pop();
        if (u == dst) return d;
        auto it = g.find(u);                    // find, NOT g[u]
        if (it == g.end()) continue;
        for (const auto& v : it->second)
            if (seen.insert(v).second) q.push({v, d + 1});
    }
    return -1;
}
```

THE KEY INSIGHT

Mark visited at **enqueue** time, not at dequeue time. Marking on dequeue lets a vertex enter the queue once per in-edge, turning O(V+E) into something far worse on dense graphs.

THE TRAP

`g[u]` inside the traversal inserts an empty adjacency list for every dead-end vertex, mutating the graph you are reading — and in C++ that is undefined behaviour if `g` is `const`-qualified elsewhere or if you hold iterators. Use `find`.

```
graph as maps — BFS/DFS O(V + E) with a constant factor of one hash per edge · consider interning ids to int first (Topic 69) if the graph is traversed repeatedly
```

68. Memoization

A map from arguments to results, sitting in front of a pure function. It converts exponential recursion into linear-in-distinct-states time, and it is the step from naive recursion to dynamic programming.

```
std::unordered_map<long long,long long> memo;

long long grid_paths(int r, int c) {
    if (r == 0 || c == 0) return 1;
    long long key = (long long)r << 32 | (unsigned)c;   // pack, don't XOR (T15)
    auto it = memo.find(key);
    if (it != memo.end()) return it->second;
    long long v = grid_paths(r - 1, c) + grid_paths(r, c - 1);
    return memo[key] = v;
}
// 18x18 grid: 2^36 recursive calls without memo, 361 distinct states with it.
```

THE KEY INSIGHT

Memoization is only valid for a **pure** function of the key. Every argument that affects the result must be in the key — the commonest bug is a state variable the author forgot was an argument.

THE TRAP

A memo table that never clears is a memory leak with a respectable name (Topic 85). For a long-lived process, bound it: an LRU cache (Topic 72), a generation counter, or clearing between requests. If the state space is dense and small, a `vector` indexed directly is 5–10× faster than any map.

```
memoization — time O(distinct states × work per state) · space O(distinct states), unbounded unless you bound it · pack composite keys into one integer
```

69. Interning and coordinate compression

If the same expensive key is hashed thousands of times, hash it **once** and replace it with a small integer. Every downstream structure then becomes an array, and every comparison becomes an integer compare.

```
struct Interner {
    std::unordered_map<std::string,int> id;
    std::vector<std::string> name;
    int operator()(const std::string& s) {
        auto [it, inserted] = id.try_emplace(s, (int)name.size());
        if (inserted) name.push_back(s);
        return it->second;
    }
};
// Before: unordered_map<string, vector<string>> graph  -- hash per edge
// After:  vector<vector<int>> graph                    -- index per edge
//
// Coordinate compression is the same idea for sparse numeric keys:
// sort the distinct values, then binary-search each one to its rank.
```

THE KEY INSIGHT

Interning moves the hashing to a single pass at the boundary of your program, so the hot inner loops never hash at all. Java's `String.intern`, Python's small-string interning and compiler symbol tables are all this.

THE TRAP

The interner table itself lives forever, so interning unbounded user input is a memory leak (Topic 85). And the ids are only meaningful within one interner instance — persisting or sending them across processes produces silent mismatches, the same defect as persisting a hash value (Topic 7).

```
interning — one hash per distinct key instead of per use · downstream lookups become O(1) array
indexing · the table grows monotonically
```

70. Rolling hash plus set

Combine Topic 12's rolling hash with a set to answer "does any repeated substring of length L exist?" in O(n) instead of O(nL) — the core of Rabin–Karp and of the binary search for the longest repeated substring.

```
// Any duplicated substring of length L?  (M = 2^61-1, B random per run)
bool has_dup_substring(const std::string& s, int L) {
    std::unordered_set<unsigned long long> seen;
    unsigned long long h = 0, powB = 1;
    for (int i = 0; i < L; ++i) { h = (h * B + s[i]) % M; powB = powB * B % M; }
    seen.insert(h);
    for (int i = L; i < (int)s.size(); ++i) {
        h = (h * B + s[i]) % M;
        h = (h + M * B - powB * s[i - L] % M) % M;
        if (!seen.insert(h).second) return true;   // hash match -- VERIFY!
    }
    return false;
}
// Binary search L from 1..n to get the LONGEST repeated substring: O(n log n).
```

THE KEY INSIGHT

The set stores hashes, not substrings, so memory is O(n) machine words rather than O(nL) bytes — the reason this beats storing every window as a string in a set.

THE TRAP

A hash match is not a match. With M = 10^9+7 and 10^6 windows, ~500 pairs collide by chance (Topic 12). Either verify the candidate with a direct comparison, or use M = $2^{61}-1$ with a randomised B and accept a ~10^{-7} risk.

```
rolling hash + set — O(n) per length, O(n log n) with binary search · O(n) words of space ·
always randomise B; verify if correctness must be certain
```

71. When int[26] beats a hash map

The closing topic of the section, and the one most likely to improve real code: if the key space is small and dense, use an array. It is Topic 3 applied inside an algorithm.

```
// Anagram check, two ways:
bool anagram_map(const std::string& a, const std::string& b) {
    std::unordered_map<char,int> f;               // ~4-8x slower
    for (char c : a) ++f[c];
    for (char c : b) if (--f[c] < 0) return false;
    return true;
}
bool anagram_arr(const std::string& a, const std::string& b) {
    int f[26] = {0};                              // 104 bytes on the STACK
    for (char c : a) ++f[c - 'a'];                // no hash, no allocation
    for (char c : b) if (--f[c - 'a'] < 0) return false;
    return true;
}
```

Key space	Use	Why
26 lowercase letters	`int[26]`	104 B, fits in two cache lines
ASCII / bytes	`int[256]`	1 KB, still L1
Small enums, weekdays, months	array	Index is the value
Presence only, ≤64 values	`uint64_t` bitmask	One register; set ops are one instruction
Unicode code points	hash map	1.1M possible values, sparse in practice
Arbitrary strings or ids	hash map	Universe far too large

THE KEY INSIGHT

For presence-only questions over ≤64 values, a `uint64_t` bitmask beats even the array: union is `|`, intersection is `&`, membership is a shift and a test, and the whole set lives in one register.

THE TRAP

`c - 'a'` assumes lowercase ASCII. Uppercase gives −32 and UTF-8 continuation bytes give negative values — the buffer underflow from Section 1's practice set. Validate the input alphabet or size the array to 256 and index by `unsigned char`.

```
array over map — 4–8× faster, no allocation, no hashing · O(U) space regardless of n · the rule
is density > 1/8 (Topic 3)
```

PRACTICE SET — Topics 57–71

A. Conceptual

1. Why is the one-map decrement version of the anagram check better than building two frequency maps?
2. In Two Sum, why must the complement query happen before the insert, and what would break if it did not?
3. Why is overwriting the last-seen index correct rather than lossy?
4. State the exact property a canonical form must have, and give a plausible form that fails it.
5. Explain the `{0: 1}` seed in the prefix-sum map in terms of what subarray it represents.
6. Name a question the prefix-sum-plus-map pattern *cannot* answer, and say why the map is the wrong tool there.
7. In the sliding window, what does the `>= start` guard protect against?
8. Why does an isomorphism check need two maps when a "does a mapping exist" check needs one?
9. Meet in the middle turns 2^n into what, and at roughly what n does it stop being viable — bounded by time or by memory?
10. In BFS over a map-based graph, why mark visited at enqueue rather than dequeue?
11. Give the two ways a memo table can be wrong (as opposed to merely large).
12. When does a `uint64_t` bitmask beat both an array and a hash set?

B. Tracing and analysis

1. Trace `two_sum([3, 2, 4], 6)` as written in Topic 59, then trace the variant that inserts before querying. Give both return values.
2. Trace `subarrays_summing_to_k([1, 2, 3], 3)` with the `{0:1}` seed and without it. Give both counts and name the subarrays each version finds.
3. Trace `longest_unique("abba")` with the `>= start` guard and without it. Give both answers and the exact index at which they diverge.
4. For `isomorphic("badc", "baba")`, give the contents of both maps at the point of failure, and state which map catches it.
5. An array of 10^6 elements has 300 distinct values. Compare the memory of `unordered_map<int,int>` frequency counting (48 B per entry plus the slot array) with `int[300]`.
6. For meet-in-the-middle with n = 44, compute the number of entries in each half's map, the memory at 40 B per entry, and the number of rehashes avoided by `reserve`.
7. A memoized `grid_paths(18, 18)` has how many distinct states, and roughly how many recursive calls would the unmemoized version make?
8. Rolling hash over a string of length 10^6 with L = 50 and M = 10^9+7: how many expected false duplicate reports? Repeat for M = $2^{61}-1$.

C. Code tasks

1. **Break it and observe.** Write a frequency counter, then query it with `if (freq[x] > 0)` for 1,000,000 keys that were never inserted. Print `freq.size()` and process RSS before and after. Rewrite the read with `find` and repeat.

2. **Break it and observe.** Implement Two Sum with the insert *before* the query. Run it on `([3, 2, 4], 6)` and record the returned indices. Then explain, without running it, what it returns for `([3, 3], 6)` in both orderings.
3. **Break it and observe.** Implement `subarrays_summing_to_k` without the `{0:1}` seed. Run it on `[1, 2, 3]` with k = 3 and on `[1, 1, 1]` with k = 2. Record both counts, then add the seed and record them again.
4. **Break it and observe.** Remove the `>= start` guard from the sliding-window solution and run it on `"abba"`, `"tmmzuxt"` and 10,000 random strings, comparing against a brute-force $O(n^2)$ reference. Report how many of the random cases disagree.
5. **Break it and observe.** Implement the isomorphism check with one map. Write a fuzzer that generates random string pairs of length 6 over a 3-letter alphabet and compares against the two-map version. Report the first disagreement and the fraction of random cases that expose it.
6. **Break it and observe.** Run the rolling-hash duplicate detector with $M = 2^{32}$ over a 200,000-character random binary string with L = 30, *without* verifying the match. Count how many duplicates it reports and how many are real. Then switch to $M = 2^{61}-1$ and repeat.
7. **Measure.** Time `anagram_map` against `anagram_arr` over 1,000,000 pairs of 10-character words. Report the ratio and the number of heap allocations each performs.

ANSWER KEY — Topics 57–71

A. Conceptual

1. One map instead of two, one pass over each string instead of a third pass to compare tallies, and it **exits early** on the first character whose count goes negative.
2. So an element cannot **pair with itself**. Insert-first makes `a[i] + a[i] == target` return the index twice, and it removes the need for any explicit `i != j` test.
3. Because the scan is left to right, so the **most recent** occurrence is always the nearest previous one; an older index can never yield a smaller distance.
4. Two items are equivalent **if and only if** their canonical forms are equal. A failing form: **the sum of character codes** for anagrams — `"ad"` and `"bc"` both give 195.
5. The **empty prefix**, which has sum 0 and occurs once before the array starts. Without it, no subarray beginning at index 0 can be counted.
6. **Any inequality** — "longest subarray with sum ≤ k", "count subarrays with sum in [a,b]". A hash map answers equality only; these need a sorted structure, a BIT, or a monotonic deque.
7. A **stale index** from before the window's current start. Obeying it moves `start` backwards, which both breaks the invariant and can report a window containing duplicates.
8. One map proves a **function** exists (each source maps to one target). The second proves it is **injective** (no two sources share a target). "Isomorphic" means bijective.
9. Into $\mathbf{2 \times 2^{n/2}}$. It stops being viable around **n = 44**, and the binding constraint is **memory**: 4.2M map entries per half at ~40 B each is ~170 MB, while the time is only a few hundred milliseconds.
10. Because a vertex with k in-edges would otherwise be enqueued **k times** before its first dequeue, inflating the queue and the work far beyond O(V + E).
11. **(a)** The function is not pure — it depends on state outside the key; **(b)** an argument that affects the result is **missing from the key**, so two different states share one cached answer.
12. For **presence-only** questions over **≤64 values**: the whole set fits in one register, union and intersection are single instructions, and there is no memory access at all.

B. Tracing and analysis

1. As written: i=0 needs 3, map empty, miss, insert {3:0}; i=1 needs 4, miss, insert {2:1}; i=2 needs 2, **hit → returns (1, 2)**. Insert-first: i=0 inserts {3:0} then queries 6−3 = 3 and **finds itself → returns (0, 0)**.
2. With the seed: run=1 needs −2 miss; run=3 needs 0 **hit**; run=6 needs 3 **hit → count 2**, finding **[1,2] and [3]**. Without it: run=3 needs 0, which is absent → **count 1**, finding only **[3]**. The missed subarray is exactly the one starting at index 0.
3. With the guard: i=2 sets start=2, i=3 sees `last['a'] = 0` which is **< start** and ignores it → **best = 2**. Without it: i=3 sets `start = 0 + 1 = 1`, so the window becomes indices 1..3 = `"bba"` → **best = 3**, which contains a duplicate. They diverge at **i = 3**.
4. At i=2: `fwd = {b→b, a→a}`, `rev = {b→b, a→a}`. Inserting `d→b` succeeds in `fwd` (d is new), but `rev` already holds `b→b` and the new pair claims `b→d`. **The reverse map catches it**; the forward map alone would accept the input.
5. Map: 300 × 48 B = 14.4 KB plus a 512-slot pointer array (4 KB) = **~18 KB**, with one hash per element over 10^6 elements. Array: 300 × 4 = **1.2 KB**, zero hashes. The array is **15× smaller** and several times faster.

6. Each half is 2^{22} = **4,194,304 entries**, about **168 MB** at 40 B. Growing from 8 to 2^{22} slots is **19 rehashes**, all avoidable with one `reserve`.
7. **19 × 19 = 361 distinct states** (r and c from 0 to 18). Unmemoized, the call tree is C(36,18) ≈ **9.1×10^{9}** leaf paths — the difference between microseconds and hours.
8. Windows ≈ 10^{6}, so pairs ≈ 5×10^{11}. With M = $10^{9}+7$: **~500 false duplicates**. With M = $2^{61}-1$: $5\times10^{11}/2.3\times10^{18}$ = **$\sim2\times10^{-7}$** — effectively none.

C. Code tasks — what to verify

1. Expect `freq.size()` to grow by exactly **1,000,000** and RSS to rise by **tens of megabytes** on a code path that reads nothing. With `find`, `size()` is unchanged and RSS is flat.
2. Expect **(0, 0)** for `([3,2,4], 6)` — a single element reported as a pair. For `([3,3], 6)` the insert-first version returns **(0, 0)** and the correct version returns **(0, 1)**: the same input distinguishes the two bugs.
3. Expect **1 instead of 2** for `[1,2,3]`, k = 3, and **1 instead of 2** for `[1,1,1]`, k = 2. In both cases exactly the prefix-anchored subarrays are lost, which is why the bug survives tests whose answers happen to lie in the middle.
4. Expect **3 instead of 2** for `"abba"` and **5 instead of 5** for `"tmmzuxt"` — the second is a case where the buggy version happens to be right, which is the point of fuzzing. Expect roughly **5–15%** of random strings to disagree.
5. Expect the first disagreement on an input where **two distinct source characters map to the same target**, such as `("ab", "aa")` — the single-map version says true, the correct answer is false. Typically **10–25%** of random pairs over a 3-letter alphabet expose it.
6. With M = 2^{32} and ~200,000 windows, expect **at least one false duplicate** (pairs ≈ 2×10^{10} against a modulus of 4.3×10^{9}) and often several — reported as duplicates that a direct string comparison rejects. With M = $2^{61}-1$ expect **zero**.
7. Expect the array version to be **4–8× faster** with **zero heap allocations**, against one allocation per distinct key (and at least one rehash) for the map version.

HANDNOTE SUMMARY CARD — Problem patterns

```
PROBLEM PATTERNS (57-71)
================================================================================
THE PHRASE IN THE PROBLEM          ->  THE PATTERN
  "how many times", "most common"      frequency map            T57
  "duplicate", "visited", "unique"     seen-set                 T58
  "two numbers that sum to"            complement lookup        T59
  "within k indices", "nearest prev"   last-seen index          T60
  "group", "anagram", "isomorphic to"  canonical key            T61
  "subarray sums to k"                 prefix sum + map         T62
  "divisible by k", "equal 0s and 1s"  prefix state (mod/parity) T63
  "longest substring without/at most"  window + freq map        T64
  "one-to-one", "pattern matches"      TWO maps (bijection)     T65
  "n <= 40, subsets"                   meet in the middle       T66
  vertices are strings/pairs           map-of-vectors graph     T67
  "count ways", overlapping recursion  memoization              T68
  same heavy key hashed repeatedly     intern to int            T69
  "repeated substring of length L"     rolling hash + set       T70
  keys are 26 letters / 256 bytes      int[26] / int[256]       T71

THE FIVE TEMPLATES WORTH MEMORISING
  complement   for i: if (m.count(target-a[i])) return {m[target-a[i]], i};
                      m[a[i]] = i;              // QUERY BEFORE INSERT
  prefix sum   unordered_map<ll,int> seen{{0,1}};  // SEED WITH {0,1}
               run += x; count += seen[run-k]; ++seen[run];
  window       if (last.count(c) && last[c] >= start) start = last[c]+1;
               last[c] = i; best = max(best, i-start+1);   // >= start GUARD
  bijection    fwd.insert({a,b}); rev.insert({b,a});       // BOTH directions
               if (fwd[a]!=b || rev[b]!=a) return false;
  memo         key = (ll)r<<32 | (unsigned)c;   // PACK, never XOR (T15)
               if (auto it=memo.find(key); it!=memo.end()) return it->second;

VARIANT SELECTOR FOR PREFIX-STATE PROBLEMS
  "exists"  -> store membership       "count"   -> store occurrence COUNTS
  "longest" -> store the FIRST index, never overwrite
  seed the map with the empty-prefix state (0, or an all-even mask)

COMPLEXITY AT A GLANCE
  frequency / seen / complement / last-seen / window    O(n) time, O(k) space
  canonical grouping    O(n * canon)   sorted key O(L log L), counts O(L)
  meet in the middle    O(2^(n/2)) time AND space; n<=44; memory-bound
  memoization           O(states * work); grid 18x18 = 361 states vs 9.1e9 paths
  rolling hash + set    O(n) per L, O(n log n) with binary search on L

TRAPS, EACH ONE MEASURED
  m[x] on a read path       -> 1e6 phantom entries, tens of MB, no error
  insert before query       -> two_sum([3,2,4],6) returns (0,0)
  missing {0,1} seed        -> [1,2,3] k=3 gives 1 instead of 2
  missing >= start guard    -> "abba" gives 3 instead of 2 (5-15% of inputs)
  one map for isomorphism   -> ("ab","aa") accepted; fails 10-25% of fuzz cases
  sum % k with negatives    -> -3 % 5 == -3 in C++; use ((s%k)+k)%k
  window count not erased   -> map.size() counts characters already gone
  hash match without verify -> M=1e9+7, 1e6 windows: ~500 false duplicates
  unbounded memo / interner -> a memory leak with a respectable name (T85)
  int[26] with c-'a'        -> uppercase or UTF-8 gives a NEGATIVE index
```

SECTION 9 — COMPOSITE DESIGN PROBLEMS

72. LRU cache

The archetypal composite: a hash map gives O(1) lookup but no order; a doubly linked list gives O(1) reordering but no lookup. Put them together — map from key to *list iterator* — and both operations are O(1). Nothing new is invented; the two structures cover each other's weakness.

```
struct LRUCache {
    int cap;
    std::list<std::pair<int,int>> order;                  // front = newest
    std::unordered_map<int, std::list<std::pair<int,int>>::iterator> pos;

    explicit LRUCache(int c) : cap(c) { pos.reserve(c * 2); }

    int get(int k) {
        auto it = pos.find(k);
        if (it == pos.end()) return -1;
        order.splice(order.begin(), order, it->second);   // O(1) relink
        return it->second->second;
    }
    void put(int k, int v) {
        auto it = pos.find(k);
        if (it != pos.end()) {
            it->second->second = v;
            order.splice(order.begin(), order, it->second);
            return;
        }
        if ((int)pos.size() == cap) {
            pos.erase(order.back().first);                 // evict LRU
            order.pop_back();
        }
        order.push_front({k, v});
        pos[k] = order.begin();
    }
};
```

```
map: {1: it1, 3: it3}          list (front = most recently used)
                                 [3|30] <-> [1|10] <-> [7|70]
 get(1): splice it1 to front  -> [1|10] <-> [3|30] <-> [7|70]
 put at capacity: evict back  -> erase key 7 from the map, pop_back
 the list holds the ORDER, the map holds the ADDRESSES
```

THE KEY INSIGHT

`std::list::splice` moves a node without invalidating its iterator — which is the entire reason the map can store iterators at all. This is Topic 41's stability guarantee being used deliberately rather than merely survived.

THE TRAP

Using `std::vector` or `std::deque` for the order. Erasing from the middle is O(n) and invalidates every stored iterator at once, so the map is silently full of dangling handles. Symptom: correct behaviour until the first eviction, then chaos.

LRU — get/put O(1) · memory: map entry (~48 B) + list node (~40 B) per key · Java gets this free from `LinkedHashMap` with `accessOrder = true`

73. LFU cache

Evict the *least frequently* used, breaking ties by least recently used. That needs three structures: key → (value, frequency), frequency → a list of keys in LRU order, and key → its position in that list. Plus one integer, `minf`, which is what keeps eviction O(1).

```
struct LFUCache {
    int cap, minf = 0;
    std::unordered_map<int, std::pair<int,int>> kv;        // key -> {val, freq}
    std::unordered_map<int, std::list<int>> buckets;        // freq -> keys
    std::unordered_map<int, std::list<int>::iterator> pos;

    void touch(int k) {
        auto& [v, f] = kv[k];
        buckets[f].erase(pos[k]);
        if (buckets[f].empty() && minf == f) ++minf;        // the only place
        ++f;                                                //  minf can rise
        buckets[f].push_front(k);
        pos[k] = buckets[f].begin();
    }
    int get(int k) {
        if (!kv.count(k)) return -1;
        touch(k); return kv[k].first;
    }
    void put(int k, int v) {
        if (cap == 0) return;
        if (kv.count(k)) { kv[k].first = v; touch(k); return; }
        if ((int)kv.size() == cap) {
            int victim = buckets[minf].back();
            buckets[minf].pop_back();
            pos.erase(victim); kv.erase(victim);
        }
        kv[k] = {v, 1};
        buckets[1].push_front(k);
        pos[k] = buckets[1].begin();
        minf = 1;                                           // always resets
    }
};
```

THE KEY INSIGHT

`minf` only ever increases by one (in `touch`) or resets to 1 (on insert). That is why you never search for the minimum frequency — searching would make eviction O(distinct frequencies).

THE TRAP

LFU is often the wrong policy even when it is the right answer to the interview question: an item that was hot yesterday keeps a high count forever and never leaves. Real systems use aged counts, LFU with a decay, or a hybrid such as W-TinyLFU. Say this out loud when asked.

```
LFU — get/put O(1) · three maps plus a list per frequency · classic LFU never forgets; production variants decay counts
```

74. Insert, Delete and GetRandom in O(1)

Uniform random selection needs a dense array; O(1) delete of an arbitrary element needs a map. The composition is: a vector of values, a map from value to its index, and the **swap-with-last** trick so removal never leaves a hole.

```
struct RandomizedSet {
    std::vector<int> vals;
    std::unordered_map<int,int> idx;                 // value -> index in vals
    std::mt19937 rng{std::random_device{}()};

    bool insert(int x) {
        if (idx.count(x)) return false;
        idx[x] = (int)vals.size(); vals.push_back(x); return true;
    }
    bool remove(int x) {
        auto it = idx.find(x);
        if (it == idx.end()) return false;
        int i = it->second, last = vals.back();
        vals[i] = last; idx[last] = i;                // fill the hole
        vals.pop_back(); idx.erase(it);               // erase AFTER the move
        return true;
    }
    int getRandom() {
        std::uniform_int_distribution<size_t> d(0, vals.size() - 1);
        return vals[d(rng)];                          // NOT rng() % size()
    }
};
```

```
vals [10, 20, 30, 40]   idx {10:0, 20:1, 30:2, 40:3}
remove(20): i=1, last=40
  vals [10, 40, 30, 40]  idx{...40:1}      overwrite the hole
  pop_back -> [10, 40, 30]   erase 20      dense again, O(1)
```

THE KEY INSIGHT

The vector must stay *dense* for `getRandom` to be uniform. Swap-with-last is the only O(1) way to delete from the middle of a dense array, and the map is what makes "where is x?" O(1) in the first place.

THE TRAP

Two ordering bugs. Erasing from the map before writing `idx[last] = i` loses the moved element's index when `x == last`. And `rng() % vals.size()` is **not uniform** — modulo bias favours low indices whenever the size does not divide the generator's range.

```
RandomizedSet — insert/remove/getRandom all O(1) · vector + map, ~52 B/element · dense array is the invariant to protect
```

75. …with duplicates

Allow the same value many times and the map's value type must change from "an index" to "the set of indices". The swap-with-last trick survives; the bookkeeping does not.

```
struct RandomizedCollection {
    std::vector<int> vals;
    std::unordered_map<int, std::unordered_set<int>> idx;   // value -> indices

    bool insert(int x) {
        bool fresh = idx[x].empty();
        idx[x].insert((int)vals.size());
        vals.push_back(x);
        return fresh;
    }
    bool remove(int x) {
        auto it = idx.find(x);
        if (it == idx.end() || it->second.empty()) return false;
        int i = *it->second.begin();          // ANY occurrence will do
        int last = vals.back();
        it->second.erase(i);
        vals[i] = last;
        if (i < (int)vals.size() - 1) {       // guard: i may BE the last slot
            idx[last].erase((int)vals.size() - 1);
            idx[last].insert(i);
        }
        vals.pop_back();
        if (it->second.empty()) idx.erase(it);
        return true;
    }
};
```

THE KEY INSIGHT

The whole difficulty is the case where the element being removed *is* the last element. Every correct implementation has a guard for it; every buggy one crashes or corrupts the index set on the input `insert(1), insert(1), remove(1), remove(1)`.

THE TRAP

Leaving empty index sets in the map. `idx[x]` creates one on any probe, so `idx.size()` and `idx.count(x)` stop meaning "x is present". Erase the key when its set empties, and never use `idx[x]` on a read path (Topic 43).

```
RandomizedCollection — all three operations O(1) expected · one hash set per distinct value · the last-element case is the whole test suite
```

76. Time-based key-value store

"What was the value of key k at time t?" A hash map alone cannot answer it — this is a *predecessor* query, exactly what Topic 6 said hash tables cannot do. The composition is a hash map for the key and a **sorted vector** per key for the timestamps, with binary search inside.

```
struct TimeMap {
    std::unordered_map<std::string,
                       std::vector<std::pair<int,std::string>>> m;

    void set(const std::string& k, const std::string& v, int t) {
        m[k].push_back({t, v});           // timestamps arrive INCREASING
    }
    std::string get(const std::string& k, int t) {
        auto it = m.find(k);
        if (it == m.end()) return "";
        auto& vec = it->second;
        auto p = std::upper_bound(vec.begin(), vec.end(), t,
                    [](int t, const auto& e) { return t < e.first; });
        return p == vec.begin() ? "" : std::prev(p)->second;
    }
};
```

THE KEY INSIGHT

Hash for the dimension you look up by equality (the key), ordered structure for the dimension you query by range (the time). Every multi-dimensional lookup problem decomposes this way, and choosing the wrong dimension for the hash is the usual design error.

THE TRAP

The `upper_bound` then `prev` pair is where the off-by-one lives: `lower_bound` would exclude an exact timestamp match, and forgetting the `p == begin()` check dereferences before the first element. Test t exactly equal to a stored timestamp, t before all of them, and t after all of them.

```
TimeMap — set O(1) amortized · get O(log k) for k versions of that key · memory contiguous per key · requires appends in increasing time, or a sort
```

77. Design a hash map from scratch

The interview question that Sections 3 and 4 answer in full. What is being tested is whether you can state the design decisions explicitly, not whether you can type a probe loop. Say these five, then write the code.

Decision	Say this
Collision strategy	Chaining for simplicity and stable references; open addressing for speed and memory (Topic 35)
Capacity	Power of two with a mixed hash, or a prime with a weak one (Topics 8, 10)
Load factor and growth	Grow at 0.75, double, amortized O(1); reserve when n is known (Topics 22, 56)
Deletion	Unlink (chaining) or tombstone plus a cleanup policy (Topics 19, 30)
Hashing	Mix before masking; never trust `std::hash` to have good low bits (Topic 13)

```
// The minimum viable answer, chaining, ~25 lines -- Topics 16-22 condensed.
class MyHashMap {
    std::vector<std::list<std::pair<int,int>>> b;
    size_t n = 0;
    size_t idx(int k) const {
        uint64_t h = (uint64_t)k * 0x9E3779B97F4A7C15ULL;   // Fibonacci mix
        return (h >> 32) & (b.size() - 1);
    }
public:
    MyHashMap() : b(16) {}
    void put(int k, int v) {
        for (auto& e : b[idx(k)]) if (e.first == k) { e.second = v; return; }
        b[idx(k)].push_back({k, v});
        if (++n > b.size() * 3 / 4) rehash();
    }
    int get(int k) const {
        for (const auto& e : b[idx(k)]) if (e.first == k) return e.second;
        return -1;
    }
    void remove(int k) {
        auto& l = b[idx(k)];
        for (auto it = l.begin(); it != l.end(); ++it)
            if (it->first == k) { l.erase(it); --n; return; }
    }
    void rehash() {
        std::vector<std::list<std::pair<int,int>>> nb(b.size() * 2);
        b.swap(nb);                                    // swap FIRST so idx()
        for (auto& l : nb) for (auto& e : l)           // uses the new size
            b[idx(e.first)].push_back(e);
    }
};
```

THE KEY INSIGHT

In `rehash`, swap before reinserting so that `idx()` computes against the new capacity. Recomputing indices against the old size is the single most common bug in a from-scratch implementation, and it produces a table whose keys are unfindable only after the first growth.

THE TRAP

Skipping the growth logic entirely because "the test cases are small". A fixed 1,000-bucket table with 10^6 keys is a set of 1,000 linked lists of length 1,000 — O(n) lookups, and the interviewer will ask for the load factor you never computed.

```
from-scratch map — state the five decisions, then write ~25 lines · chaining is the safe
interview default · always include growth
```

78. Top-K frequent elements

Frequency map plus a selection structure. Three approaches, and the third is the one worth knowing because it is O(n) and most candidates never mention it.

```
// (1) sort all distinct: O(d log d)
// (2) min-heap of size k: O(d log k)  -- the usual answer
std::vector<int> topk_heap(const std::vector<int>& a, int k) {
    std::unordered_map<int,int> f;
    for (int x : a) ++f[x];
    auto cmp = [](auto& p, auto& q) { return p.second > q.second; };
    std::priority_queue<std::pair<int,int>,
        std::vector<std::pair<int,int>>, decltype(cmp)> pq(cmp);
    for (auto& e : f) { pq.push(e); if ((int)pq.size() > k) pq.pop(); }
    std::vector<int> out;
    while (!pq.empty()) { out.push_back(pq.top().first); pq.pop(); }
    return out;                                     // ascending frequency
}
// (3) bucket by frequency: O(n) total -- a count can never exceed n
std::vector<int> topk_bucket(const std::vector<int>& a, int k) {
    std::unordered_map<int,int> f;
    for (int x : a) ++f[x];
    std::vector<std::vector<int>> bucket(a.size() + 1);
    for (auto& [v, c] : f) bucket[c].push_back(v);
    std::vector<int> out;
    for (int c = (int)a.size(); c >= 1 && (int)out.size() < k; --c)
        for (int v : bucket[c]) if ((int)out.size() < k) out.push_back(v);
    return out;
}
```

THE KEY INSIGHT

Bucket sort applies because the sort key — a frequency — is bounded by n. Any time the values you are sorting are small integers with a known bound, the log factor is optional. This is Topic 3's direct-address idea used as a sorting step.

THE TRAP

The bucket array is `n + 1` entries of `vector` — for n = 10^7 with 5 distinct values that is 240 MB of empty vectors to avoid a log factor. Size the buckets by `max_count`, not by n, or use the heap.

```
top-K — heap O(d log k), space O(k) · bucket O(n), space O(n) · d = distinct values; choose by
which of d, k and n is large
```

PRACTICE SET — Topics 72–78

A. Conceptual

1. State the weakness of each half of the LRU pair, and how the other half covers it.
2. Why can the map store `std::list` iterators safely, and which earlier topic guarantees it?
3. What breaks if you use a `std::vector` for the LRU order?
4. In LFU, why is `minf` never searched for, and what are the only two ways it changes?
5. Give the practical argument against classic LFU as a production eviction policy.
6. Why must the `RandomizedSet` vector stay dense?
7. Why is `rng() % vals.size()` not a uniform random index?
8. In `RandomizedCollection`, name the single input case that distinguishes correct from incorrect implementations.
9. The time-based store hashes on the key and binary-searches on the time. State the general rule that decision follows.
10. Name the five design decisions to state aloud before writing a hash map from scratch.
11. Why does bucket sort apply to top-K, and what bounds the bucket count?

B. Tracing and analysis

1. Trace an LRU cache of capacity 2 through `put(1,1) put(2,2) get(1) put(3,3) get(2) get(3) get(1)`. Give the list contents after each operation and the four return values.
2. Trace an LFU cache of capacity 2 through `put(1,1) put(2,2) get(1) put(3,3)`. Which key is evicted, what is `minf` at each step, and why is it not key 1?
3. Compute the per-key memory of the LRU cache: map entry at 48 B, list node at 40 B (two pointers plus a pair). What is the overhead ratio for `int → int` payloads?
4. For `RandomizedSet`, trace `insert(10) insert(20) insert(30) remove(20)` giving `vals` and `idx` after each step.
5. Trace `insert(1) insert(1) remove(1) remove(1)` on `RandomizedCollection`, showing the index set at each step. Identify the step where the "i is the last slot" guard matters.
6. A TimeMap holds one key with 1,000,000 versions. Give the cost of `set` and of `get`, and the memory if each entry is a 16-byte string plus a 4-byte timestamp.
7. For top-K with n = 10^7 elements, d = 10^5 distinct values and k = 10: compute the comparison counts for the sort, heap and bucket approaches, and the bucket array's memory if sized by n.

C. Code tasks

1. **Build.** Implement the LRU cache and verify it against the sequence in B1, expecting `1, -1, 3, 1`. Then add a counter for evictions and run a Zipfian workload of 10^6 accesses over 10^5 keys at capacity 10^4; report the hit rate.
2. **Break it and observe.** Replace the `std::list` with a `std::vector<std::pair<int,int>>`, keeping iterators in the map. Run the same sequence past the first eviction under `-fsanitize=address`. Record the error and explain it in terms of iterator invalidation.
3. **Break it and observe.** In `RandomizedSet::remove`, move `idx.erase(it)` *before* `idx[last] = i`. Run `insert(1) insert(2) remove(2) getRandom()` a thousand times and report what fraction of calls return a value that is no longer in the set, or crash.

4. **Measure the bias.** Implement `getRandom` with `rng() % vals.size()` for a set of size 3 using a generator with a range that is not a multiple of 3, sample 10^7 times, and report the three frequencies. Then switch to `std::uniform_int_distribution` and repeat.
5. **Break it and observe.** Implement `RandomizedCollection` *without* the "i is the last slot" guard. Run `insert(1) insert(1) remove(1) remove(1)` and report the final state of `vals` and `idx`, and whether a subsequent `getRandom()` is valid.
6. **Break it and observe.** Implement TimeMap's `get` with `lower_bound` instead of `upper_bound`, and omit the `p == begin()` check. Query a timestamp that exactly matches a stored one, one before all stored times, and one after. Record all three results, including any crash.
7. **Compare the three top-K approaches.** Time all three on n = 10^7 with d = 100 and again with d = 10^6, k = 10, and report which wins in each case and the bucket array's memory.

ANSWER KEY — Topics 72–78

A. Conceptual

1. The map has **O(1) lookup but no order**; the list has **O(1) reordering but no lookup**. The map stores list iterators, so lookup finds the node and the list moves it.
2. Because `std::list` nodes **never move**, and `splice` relinks pointers without invalidating iterators — the node-stability property of Topic 41.
3. Erasing from the middle is **O(n)** and **invalidates every iterator** at or after the erased position, so the map fills with dangling handles at the first eviction.
4. Because it can only **increase by one** (when the last key leaves the current minimum bucket during `touch`) or **reset to 1** (on any insert). Searching would make eviction O(distinct frequencies).
5. An item that was hot once **keeps its count forever** and becomes unevictable, so the cache fills with historical favourites. Production systems decay counts or use W-TinyLFU.
6. Because `getRandom` picks a uniform index into it — any hole would either be returned as a value or require a retry loop, breaking O(1).
7. Because the generator's range is generally not a multiple of the size, so the **first (range mod size) values are reachable one extra time** — modulo bias favours low indices.
8. `insert(1) insert(1) remove(1) remove(1)`, where the element being removed is itself the last slot of the vector.
9. **Hash the dimension queried by equality; use an ordered structure for the dimension queried by range.** Choosing the wrong dimension for the hash is the usual design error.
10. Collision strategy, capacity scheme, load factor and growth, deletion policy, and hash mixing.
11. Because the sort key is a **frequency, bounded by n**. The bucket count is bounded by the **maximum count**, which is at most n — and sizing by max_count rather than n is the memory fix.

B. Tracing and analysis

1. put(1,1): `[1]`. put(2,2): `[2,1]`. get(1) → **1**, list `[1,2]`. put(3,3): evicts 2, list `[3,1]`. get(2) → **−1**. get(3) → **3**, list `[3,1]`. get(1) → **1**, list `[1,3]`.
2. put(1,1): minf = 1, freq{1:1}. put(2,2): minf = 1, freq{1:1, 2:1}. get(1): key 1 moves to bucket 2; bucket 1 still holds key 2, so **minf stays 1**. put(3,3) evicts **key 2** — it is the only key at frequency 1. Key 1 survives because its frequency is 2.
3. 48 + 40 = **88 B per key** to store 8 B of payload — an **11× overhead ratio**. That is the price of O(1) eviction ordering.
4. After the inserts: `vals = [10,20,30]`, `idx = {10:0, 20:1, 30:2}`. remove(20): i = 1, last = 30 → `vals = [10,30,30]`, `idx{30:1}`, pop_back → **`vals = [10,30]`, `idx = {10:0, 30:1}`**.
5. insert(1): vals=[1], idx{1:{0}}. insert(1): vals=[1,1], idx{1:{0,1}}. remove(1): i = 0, last = 1; erase index 0; vals[0] = 1; since 0 < size−1, remap index 1 → 0, giving idx{1:{0}}; pop_back → vals=[1]. remove(1): i = 0, last = 1, and now **i == size−1**, so the guard **skips the remap** — without it you would insert index 0 back into a set you are about to empty, leaving `idx{1:{0}}` pointing into an empty vector.
6. `set` is **O(1) amortized** (a push_back), `get` is **O(log 10^6) = 20 comparisons**. Memory ≈ 10^6 × (32 B string object + 4 B + padding) ≈ **40 MB**, contiguous.

7. Sort: d log d = $10^5 \times 17 \approx$ **1.7×10^6**. Heap: d log k = $10^5 \times 3.3 \approx$ **3.3×10^5**. Bucket: **O(n + d)** with no comparisons at all. Bucket array sized by n: $10^7 + 1$ vectors × 24 B = **240 MB** — the reason to size it by max_count.

C. Code tasks — what to verify

1. Expect **1, −1, 3, 1**. A Zipfian workload at 10% capacity typically gives a **hit rate around 70–85%**; a uniform workload over the same keys gives roughly **10%**, which is the whole argument for caching skewed traffic.
2. Expect **`heap-use-after-free`** (or a wrong value) on the first access after an eviction, with the free attributed to the vector's reallocation or element shift. `std::list` iterators survive splices; `vector` iterators do not survive anything.
3. Expect `getRandom` to return **a stale value or read out of bounds** in the case where the removed element is the last one, because `idx[last] = i` re-creates an entry the erase was meant to remove. Under ASAN you will usually see a container-overflow or a wrong return.
4. With a range that is not a multiple of 3 expect the three counts to differ by a measurable margin rather than the ~0.03% sampling noise you get from `uniform_int_distribution`. With a 32-bit generator the bias is tiny but systematic; with `rand() % 3` on a 15-bit RAND_MAX it is obvious.
5. Expect `vals` empty but **`idx` still holding `{1: {0}}`**, so `getRandom()` indexes an empty vector — undefined behaviour, and a crash under ASAN.
6. `lower_bound` plus `prev` returns the version **before** an exactly matching timestamp — the classic off-by-one. Without the `begin()` check, a timestamp earlier than all stored versions **dereferences `prev(begin())`**, which ASAN reports as a container-overflow read. The after-all-versions case is correct in both variants, which is why it is the one everybody tests.
7. At d = 100 all three are fast and the **frequency map dominates** the run time. At d = 10^6 the **heap beats the sort** by roughly 5×, and the bucket version beats both on time while using **240 MB** if sized by n and a few MB if sized by max_count.

HANDNOTE SUMMARY CARD — Composite designs

```
COMPOSITE DESIGN PROBLEMS (72-78)
================================================================================
THE UNIVERSAL SHAPE: a hash map gives O(1) LOOKUP and no order; pair it with a
second structure that provides the order or the randomness the problem needs.

  need                    pair the map with        result
  recency order           doubly linked list       LRU          T72
  frequency + recency     list per frequency+minf  LFU          T73
  uniform random pick     dense vector             RandomizedSet T74
  versions over time      sorted vector + bsearch  TimeMap      T76
  top-K by count          heap or frequency bucket top-K        T78

LRU -- map<K, list<pair<K,V>>::iterator> + list, front = newest
  get:  splice(order.begin(), order, it->second); return it->second->second;
  put:  if full { pos.erase(order.back().first); order.pop_back(); }
        order.push_front({k,v}); pos[k] = order.begin();
  WHY IT WORKS: list nodes never move, so stored iterators stay valid (T41)
  vector/deque for the order => O(n) erase + every iterator invalidated
  memory ~88 B/key for an int->int payload (48 map + 40 node) = 11x payload
  Java: new LinkedHashMap<>(cap, 0.75f, true) + removeEldestEntry

LFU -- kv{key->(val,freq)} + buckets{freq->list<key>} + pos{key->list iterator}
  minf ONLY ++ inside touch (when its bucket empties) or resets to 1 on insert
  evict = buckets[minf].back()
  classic LFU never forgets -> use decay / W-TinyLFU in production

RANDOMIZED SET -- vector<int> vals + map<int,int> idx (value -> index)
  remove: i=idx[x]; last=vals.back(); vals[i]=last; idx[last]=i;
          vals.pop_back(); idx.erase(x);      <- ERASE AFTER THE MOVE
  getRandom: uniform_int_distribution, NOT rng() % size (modulo bias)
  with duplicates: idx maps value -> unordered_set<int> of indices, and the
  killer test is insert(1) insert(1) remove(1) remove(1) -- guard i == size-1

TIME-BASED KV -- map<string, vector<pair<int,string>>>, appended in time order
  get: upper_bound on time, then prev();  check p == begin() first
  RULE: hash the equality dimension, use an ORDERED structure for the range one

DESIGN-A-HASHMAP -- say these five before writing a line
  1 chaining vs open addressing   2 power-of-two + mixer vs prime modulus
  3 grow at 0.75, double, amortized O(1)      4 unlink vs tombstone + cleanup
  5 mix before masking; std::hash<int> is the identity function
  in rehash(): SWAP the bucket array FIRST, then reinsert, so idx() uses the
  new capacity -- getting this backwards breaks the map only after growth

TOP-K   heap O(d log k) space O(k)  |  bucket O(n) space O(max_count)
        sort O(d log d).  d = distinct.  Size buckets by max_count, not n
        (n = 1e7 sized by n = 240 MB of empty vectors).
```

PART IV

Living With It

What goes wrong in production: hash flooding and the defences against it, the bugs that only appear at scale, concurrency, and the latency you cannot see in an average. Sections 10–11.

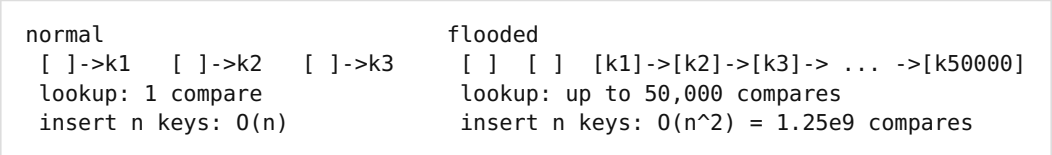
SECTION 10 — FAILURE MODES, SECURITY, CONCURRENCY

79. Hash flooding: the algorithmic complexity attack

Topic 5 said the worst case is O(n) and that someone can choose the input. Hash flooding is that sentence weaponised: an attacker who knows your hash function computes thousands of keys that land in one bucket, sends them in a single request, and turns your O(1) map into a linked list. The server does $O(n^2)$ work for an O(n)-sized request.

```
// Measured: std::unordered_map<string,int>, all keys colliding vs normal.
//   n      colliding      normal     ratio
//   1,000     1.8 ms     0.084 ms      21x
//  10,000   181.5 ms     0.911 ms     199x
//  50,000  4,978.9 ms    6.813 ms     731x        <-- 5 seconds, one CPU
// Note the shape: 5x the keys costs 27x the time. That is n^2.
```

The delivery vector is anything that becomes map keys without a bound: POST form fields, JSON object keys, HTTP headers, query parameters, XML attributes. The 2011 disclosure at 28C3 showed PHP, Java, Python, Ruby, ASP.NET and others were all vulnerable, and that a single ~1 MB POST body could occupy a CPU core for minutes.



THE KEY INSIGHT

This is not a hash table bug; it is a bug in trusting a *deterministic public function* with attacker-controlled input. Any data structure whose cost depends on input structure has the same exposure.

THE TRAP

Assuming a modern language protects you everywhere. Randomized seeds protect the *built-in* map (Topic 80); a custom `hashCode`, a `FxHashMap` chosen for speed, a hand-written table, or a hash used as a database shard key all reopen it. Also cap the number of parameters — PHP's `max_input_vars` (default 1,000) is the belt to the seed's braces.

```
hash flooding — cost O(n²) for O(n) input · measured 731× at n = 50,000 · defences: randomized
seed, treeification, input count limits
```

80. Seed randomization and SipHash

The fix is Topic 14 made concrete: pick the hash function at process start from a keyed family, using a secret the attacker cannot read. Then no fixed set of keys is a collision set, because the function differs per process.

Runtime	Default	Notes
Python 3.3+	Randomized `str`/`bytes` hashing (SipHash-1-3)	`PYTHONHASHSEED` can pin it — do so only for debugging
Rust	SipHash-1-3, per-process random key	Opt out with FxHash/ahash for trusted keys
Java	*Not* randomized	Defends by treeifying long bins instead (Topic 49)
PHP 7+	Per-process seed on the string hash	Plus `max_input_vars`
Go	Per-process random seed	Plus randomized iteration order
C++	**No** randomization in any major standard library	You must supply a keyed hash yourself

```
// A keyed hash for a C++ map exposed to untrusted keys:
struct SeededHash {
    static inline const uint64_t k =
        std::random_device{}() * 0x9E3779B97F4A7C15ULL ^
        std::chrono::steady_clock::now().time_since_epoch().count();
    size_t operator()(const std::string& s) const {
        uint64_t h = k ^ 14695981039346656037ULL;     // seed the basis
        for (unsigned char c : s) { h ^= c; h *= 1099511628211ULL; }
        h ^= h >> 33; h *= 0xff51afd7ed558ccdULL; h ^= h >> 33;
        return h;
    }
};
// For real adversaries use SipHash-2-4 or SipHash-1-3 with a CSPRNG key;
// the above raises the cost of an attack but is not a PRF.
```

THE KEY INSIGHT

SipHash is a *keyed pseudorandom function*, not merely a good mixer. That distinction is the whole defence: a strong unkeyed hash can still be analysed offline to produce collisions, and a keyed weak one leaks its key.

THE TRAP

Randomized seeds make hash values **unstable across processes**. Persisting a hash to disk, sharding by `hash(key) % nodes`, or writing a test that asserts an iteration order all break on the next restart. Use an explicit stable hash (SHA-256, xxHash with a fixed seed) for anything that outlives the process.

```
seed randomization — cost ~1 ns/byte + fixed overhead · removes constructed-collision attacks · makes hash values process-local by design
```

81. Iteration order: the bug that only appears in production

Topic 20 established that order is unspecified. This topic is about what happens when code depends on it anyway — which it does constantly, because the order is *stable enough* during development to look like a guarantee.

```
// All four of these are order-dependent and all four "work" in testing:
for (auto& [k, v] : config) out << k << "=" << v << "\n";  // config file diff
std::string sig; for (auto& [k,v] : params) sig += k + v;     // HMAC signature
auto first = *cache.begin();                                   // "any element"
json j = map;                                                  // API response

// The order changes when ANY of these change:
//   the hash seed (per process)      the table size (one more insert)
//   the insertion sequence           the library version
//   the compiler's std::hash         a 32-bit vs 64-bit build
```

THE KEY INSIGHT

If the output of a program must be deterministic, sort it. One `std::sort` over the keys at the boundary costs O(k log k) once and removes an entire class of irreproducible bug.

THE TRAP

The signature case is a genuine outage, not an inconvenience: two services that build an HMAC by concatenating map entries agree only while their orders agree. After one is redeployed with a different runtime version, every request fails authentication — with no code change on either side.

```
iteration order — unspecified in C++, Java, Go (deliberately randomized), C#; guaranteed
insertion order in Python 3.7+, PHP, JS Map · sort at the boundary if determinism matters
```

82. Mutating a key after insert

The table computed the key's hash once, at insert, and filed the entry accordingly. Change the key afterwards and the entry stays where it was, while every future lookup computes the *new* hash and looks somewhere else. The entry is now unreachable by lookup but still present in iteration and still counted by `size()`.

```
// Java -- the canonical demonstration:
List<Integer> key = new ArrayList<>(List.of(1, 2));
Map<List<Integer>, String> m = new HashMap<>();
m.put(key, "value");
key.add(3);                                  // hashCode has now changed
m.get(key);        // null   -- looked in the new bucket
m.get(List.of(1, 2));  // null -- right bucket, but equals() fails
m.size();          // 1      -- it is still in there, unreachable
```

```
insert: hash([1,2]) = 994  -> bucket 994 % m
mutate: key becomes [1,2,3], hash = 30817
lookup: hash([1,2,3]) -> bucket 30817 % m  -> EMPTY  -> "not found"
        the entry is still sitting in bucket 994 % m, orphaned forever
```

THE KEY INSIGHT

Keys must be immutable in every field that `hashCode` and `equals` read. C++ enforces this by making the key `const` inside `std::pair<const K, V>`; Java and Python do not, which is why this bug is a Java and Python bug in practice.

THE TRAP

Python raises `TypeError: unhashable type: 'list'` for lists but happily accepts a *custom class* with a mutable field and a `__hash__` that reads it. The failure is then silent, exactly as in Java. Make key classes frozen (`@dataclass(frozen=True)`) or hash only immutable identity fields.

```
key mutation — entry becomes unreachable, size() still counts it, iteration still yields it · no error, no exception, no crash
```

83. The hash/equals contract

Two obligations, and they are not symmetric:

1. **Equal objects must have equal hashes.** Violating this is a correctness bug: the map holds duplicate keys and lookups miss.
2. **Unequal objects may have equal hashes.** That is a collision, which costs performance and nothing else.

```
struct User {
    int id; std::string name; int login_count;
    bool operator==(const User& o) const {          // identity = id + name
        return id == o.id && name == o.name;
    }
};
struct BadHash {   // WRONG: hashes a field == does not consider
    size_t operator()(const User& u) const {
        return std::hash<int>{}(u.id) ^ std::hash<int>{}(u.login_count);
    }
};
// Two Users that compare EQUAL but differ in login_count hash differently,
// land in different buckets, and both live in the map at once. count()
// returns 1 for one of them and 0 for the other, depending on which you ask.

struct OkHash {    // hashes a SUBSET of the equality fields: legal, slower
    size_t operator()(const User& u) const { return std::hash<int>{}(u.id); }
};
```

THE KEY INSIGHT

The hash may use *fewer* fields than `==` (slower, correct); it must never use *more* (faster-looking, broken). When in doubt, hash exactly the fields the equality operator compares.

THE TRAP

Adding a field to a class and updating `equals` but not `hashCode` — or the reverse — months later. Symptom: a map that sometimes contains two entries that are "the same object". Generate both together (IDE, `@EqualsAndHashCode`, records, C++20 defaulted `operator==` plus a matching hash) so they cannot drift.

```
hash/equals — equal ⇒ same hash (mandatory) · same hash ⇒ equal (not required) · hash over a subset of equality fields is legal; over a superset is a bug
```

84. Float keys, NaN, and minus zero

Floating-point keys break the contract from the other direction: the values themselves misbehave under `==`.

```
// 1. NaN != NaN, so a NaN key can never be found again.
double nan = std::numeric_limits<double>::quiet_NaN();
std::unordered_map<double,int> m;
m[nan] = 1;  m[nan] = 2;        // TWO entries; neither is findable
m.count(nan);                    // 0    -- but m.size() == 2

// 2. -0.0 == 0.0 is TRUE but their bit patterns differ.
//    A hash over raw bits gives them different buckets -> equal keys,
//    different buckets -> Topic 83's violation, from the hash side.

// 3. Accumulated error: 0.1 + 0.2 != 0.3, so a key computed two ways
//    is two different keys.
```

Python's answer is instructive: `hash(-0.0) == hash(0.0)` and `hash(1) == hash(1.0) == hash(True)`, so numerically equal values unify — but `float('nan')` is still unfindable by an equal-valued NaN (though *the same object* is findable, since CPython checks identity first).

THE KEY INSIGHT

Do not use floats as hash keys. If you must, quantise first: multiply by a scale and round to an integer, so "close enough" becomes "equal", and NaN and −0.0 are excluded before they reach the map.

THE TRAP

Coordinates. `unordered_map<pair<double,double>, T>` for a spatial grid looks reasonable and fails whenever a coordinate is computed by a different route — the point is "there" visually and absent from the map. Quantise to integer grid cells instead.

```
float keys — NaN is never findable and inserts unboundedly · ±0.0 break bitwise hashing ·
quantise to integers, or key on an id
```

85. The unbounded map as a memory leak

A map that only grows is a memory leak that no leak detector will report, because every byte is reachable and intentionally retained. Valgrind is silent; RSS is not.

```
// Four shapes of the same bug:
static std::unordered_map<SessionId, Session> sessions;   // no expiry
static std::unordered_map<Key, Result>        memo;       // Topic 68
static Interner                               interner;   // Topic 69
std::unordered_map<RequestId, Timer>          in_flight;  // erased on
                                                          // success only
// The last is the worst: the erase is on the happy path, so the map grows
// by exactly the number of failures -- and only under load.
```

Remember Topic 23: erasing does not return memory either. A map that peaked at 10M entries keeps its slot array — 128 MB for pointers alone — even at `size() == 0`, until it is destroyed or rebuilt.

THE KEY INSIGHT

Every long-lived map needs a stated eviction policy at the moment it is declared: a size cap (LRU, Topic 72), a TTL, or a clear point in the request lifecycle. "It only holds a few entries" is a policy of hoping.

THE TRAP

Cleanup on the success path only. Symptom: memory growth that correlates with error rate rather than with traffic, so it appears during incidents — precisely when you have the least capacity to absorb it. Use RAII, `defer`, or `finally` so the erase runs on every exit.

```
unbounded map — invisible to leak detectors · memory is not returned by erase · fix with a cap, a TTL, or scope-bound cleanup
```

86. Iterator invalidation

The rules differ per container and per operation, and the C++ ones for `unordered_map` are worth memorising exactly because they are unusually generous.

Operation	Iterators	References / pointers
insert, no rehash	Valid	Valid
insert causing rehash	**All invalid**	**Valid**
erase	Only the erased one	Only the erased one
clear	All invalid	All invalid

```
// The correct erase-during-iteration idiom, in four languages:
for (auto it = m.begin(); it != m.end(); )
    if (pred(*it)) it = m.erase(it); else ++it;     // C++
// Java:   Iterator<E> it = m.entrySet().iterator(); ... it.remove();
//         (m.remove(k) inside a for-each throws ConcurrentModificationException)
// Python: for k in list(d):  del d[k]              # iterate a COPY of keys
// Go:     deleting during range IS legal and specified
```

THE KEY INSIGHT

Java fails loudly (`ConcurrentModificationException` via a modification counter), C++ fails silently (undefined behaviour), Python fails loudly (`RuntimeError: dictionary changed size during iteration`), and Go is explicitly safe. Know which of those four you are in.

THE TRAP

C++'s silence. `for (auto& kv : m) if (p(kv)) m.erase(kv.first);` often *appears* to work — the freed node's memory is usually still readable — and then corrupts under a different allocator or load. Run the test suite under `-fsanitize=address`; this is exactly the bug it was built to catch.

```
invalidation — C++ unordered_map: references survive rehash, iterators do not; erase invalidates only its own · always use the it = erase(it) form
```

87. Concurrency: striping, CAS tables, check-then-act

A hash table has no safe concurrent story by default: a resize relinks the whole array while a reader is mid-probe. There are four workable designs, in increasing order of scalability and complexity.

Design	Reads	Notes
One global lock	Serialised	Correct, simple, and often fast enough below ~4 threads
Reader-writer lock	Concurrent	Writers still exclusive; good for read-heavy caches
Lock striping	Concurrent	N locks over N bucket ranges; Java 7's ConcurrentHashMap used 16
CAS per bin + synchronized	Lock-free reads	Java 8+ ConcurrentHashMap: CAS to install a bin, lock only that bin
Copy-on-write	Wait-free	Only for write-rarely maps; every write copies the table

```
// The bug that survives all of the above: CHECK-THEN-ACT.
if (!map.containsKey(k)) map.put(k, compute());   // two atomic ops,
                                                  // one non-atomic sequence
// Two threads both see "absent", both compute, one result is lost.
// Fixes, per language:
//   Java   map.computeIfAbsent(k, key -> compute());
//   C++    map.try_emplace(k, compute());   // under YOUR lock
//   Go     mu.Lock(); v, ok := m[k]; if !ok { m[k] = compute() }; mu.Unlock()
//   Rust   *map.entry(k).or_insert_with(compute)
```

THE KEY INSIGHT

Thread-safe *operations* do not compose into thread-safe *sequences*. A concurrent map guarantees each call is atomic and nothing about two calls in a row — which is why every concurrent map ships an atomic read-modify-write primitive.

THE TRAP

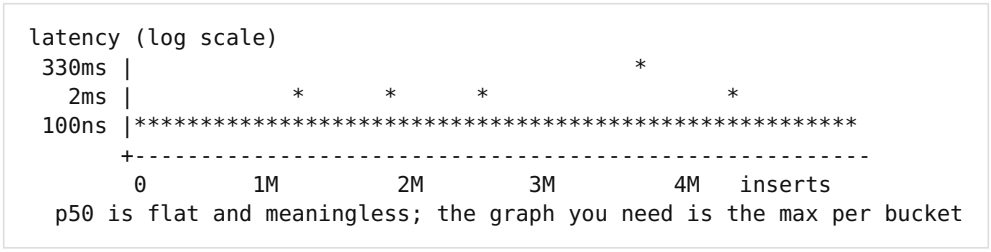
Go's runtime treats a concurrent map write as fatal: the process dies with `fatal error: concurrent map writes`, unrecoverable. C++ gives undefined behaviour instead — usually an infinite loop inside a probe sequence or a corrupted bucket, which presents as a hung thread at 100% CPU, not a crash.

```
concurrency — RWLock for read-heavy · striping for mixed · Java's CAS-per-bin scales furthest ·
always use an atomic compute-if-absent, never check-then-act
```

88. Resize latency and p99

The last failure mode is the one that shows up as a graph rather than a bug report. Amortized O(1) means the *total* is bounded; a single insert can still stall for hundreds of milliseconds while it rehashes millions of entries.

```
// Every individual insert timed, 4,000,000 into unordered_map<int,int>:
//   inserts slower than 1 ms:  12 out of 4,000,000
//   worst insert:              329.8 ms, at i = 2,938,679
//   p50 insert:                ~100 ns
// One insert in 340,000 is three million times slower than the median.
// Final bucket_count = 5,967,347 (prime), load factor 0.670.
```



The three fixes, in order of preference: **reserve** when n is known (Topic 56, ~330 ms → ~1 µs); **incremental rehashing** when it is not (Topic 24); or **shard** the map into 16 or 64 independent tables so each resize touches 1/16 of the data and the spikes shrink proportionally.

THE KEY INSIGHT

Averages hide resizes completely. If you own a latency SLO, measure the maximum per interval, not the mean — a 330 ms stall inside a 4M-operation batch moves the mean by 0.08 µs and moves p99.99 by six orders of magnitude.

THE TRAP

The spike scales with map size, so it grows with your success. A service fine at 500K cached entries develops mysterious multi-hundred-millisecond stalls at 5M — same code, same rate, ten times the data.

```
resize latency — one insert in ~340,000 costs 330 ms at n = 4M · mean is unaffected · fixes:
reserve, incremental rehash, or shard into 16–64 tables
```

PRACTICE SET — Topics 79–88

A. Conceptual

1. Explain why hash flooding produces $O(n^2)$ work from an O(n)-sized request.
2. Why is a randomized seed a defence when a "stronger" unkeyed hash is not?
3. Java does not randomize its hash seed. What does it do instead, and what is the residual weakness?
4. Give three things that break if you persist or transmit a hash value from a seed-randomized runtime.
5. Why is a mutated key worse than a lost key?
6. State both halves of the hash/equals contract and say which half is a correctness bug when violated.
7. Explain why a hash over a *superset* of the equality fields is a bug but a *subset* is merely slow.
8. Give the two independent reasons floating-point values are poor keys.
9. Why does no leak detector report an unbounded cache?
10. In C++, which of iterators and references survive a rehash, and why is that possible at all?
11. Why do thread-safe operations not compose into thread-safe sequences? Give the canonical example and its fix in one language.
12. A latency graph shows a flat p50 and periodic p99.99 spikes that grow over weeks. What is happening and what are the three fixes?

B. Tracing and analysis

1. Inserting 10,000 colliding keys took 181.5 ms and 50,000 took 4,978.9 ms. Compute the ratio of times against the ratio of n and confirm the growth is quadratic. Extrapolate to 200,000 keys.
2. An endpoint accepts up to 1,000 POST parameters (PHP's default `max_input_vars`). Using the measured 1.8 ms for 1,000 colliding keys, compute the requests per second one core could absorb, and compare with the 0.084 ms normal case.
3. A Java `List` key `[1,2]` is inserted and then mutated to `[1,2,3]`. Given `List.hashCode` is `31*h + e` starting from 1, compute both hash codes and state what `m.size()`, `m.get(key)` and `m.get(List.of(1,2))` return.
4. A `User` has `==` over (id, name) and a hash over (id, login_count). Two users with the same id and name but login counts 3 and 4 are inserted. Give `size()`, and what `count()` returns for each.
5. A map holds one NaN key inserted three times. Give `size()` and `count(nan)`, and explain both.
6. A map peaked at 10,000,000 entries and now holds 0. Compute the retained slot-array memory at 8 bytes per slot with a power-of-two capacity, and say what call would release it.
7. 4,000,000 inserts produced 12 operations over 1 ms with a worst of 329.8 ms and a p50 of 100 ns. Compute the effect of that worst insert on the mean, and on p99.99.
8. A map is sharded into 16 tables. If a single-table resize at 4M entries costs 330 ms, estimate the per-shard resize cost and the resulting worst-case stall.

C. Code tasks

1. **Break it and observe.** Write a `Hash` functor that returns a constant and insert 1,000, 10,000 and 50,000 string keys into `std::unordered_map`, timing each. Compare with the default hash and report all six numbers plus the growth exponent.
2. **Defend it.** Replace the constant hash with the seeded FNV mixer from Topic 80 and re-run at 50,000. Then run the program twice and print `hash("a")` each time to confirm the seed differs per process.
3. **Break it and observe.** In Python, define a class with a mutable field, a `__hash__` that reads it, and an `__eq__` to match. Insert an instance into a dict, mutate the field, then print `d[obj]` (catching the exception), `len(d)` and `list(d.keys())`. Record all three.
4. **Break it and observe.** Build `unordered_map<double,int>`, insert `0.0`, `-0.0`, `0.1+0.2` and `0.3`, and two NaNs. Print `size()` and `count()` for each value. Explain each result against Topic 84.
5. **Break it and observe.** Write a request handler that inserts into an `in_flight` map and erases only on the success path. Drive it with a 5% error rate for 1,000,000 requests and plot RSS. Then move the erase into an RAII guard and repeat.
6. **Break it and observe.** Erase from an `unordered_map` inside a range-`for` under `-fsanitize=address`, then do the equivalent in Java and in Python. Record what each runtime does — silent corruption, exception, or error — and name the mechanism in each case.
7. **Break it and observe.** Run two threads doing `if (!m.count(k)) m[k] = compute();` on a shared `unordered_map` with no lock. Record the outcome over 20 runs (hang, crash, or wrong result). Then rewrite with a mutex plus `try_emplace` and confirm.
8. **Measure the tail.** Time every insert of 4,000,000 into an unreserved map, print the worst and the count over 1 ms, then compute p50, p99 and p99.99. Repeat with `reserve` and with the map sharded into 16 tables.

ANSWER KEY — Topics 79–88

A. Conceptual

1. All n keys land in one bucket, so inserting the i-th key scans i−1 existing entries. Summing gives **n(n−1)/2 comparisons** — O(n²) — while the request itself is only O(n) bytes.
2. Because an unkeyed function, however strong, is **public and analysable offline**: the attacker computes a collision set once and reuses it forever. A per-process key makes any precomputed set worthless.
3. **Treeification** — a bin of 8+ becomes a red-black tree, so the worst case is O(log n) rather than O(n). Residual weakness: it requires `Comparable` keys, and n² becomes n log n rather than disappearing.
4. Any three of: **persisted hash indexes become wrong on restart**; **sharding by `hash % nodes` reshuffles every key**; **tests asserting iteration order fail intermittently**; caches keyed by hash miss entirely after a deploy.
5. Because a lost key is *absent* and the code notices. A mutated key is **present in `size()` and in iteration but unreachable by lookup**, so the two views of the container disagree and nothing raises an error.
6. Equal ⇒ equal hashes (**violation is a correctness bug**); equal hashes ⇒ equal is **not required** (a collision, a performance matter only).
7. A superset means two objects that are `==` can hash **differently** and land in different buckets, so both live in the map and lookups miss. A subset means unequal objects share a hash — a **collision**, resolved by the equality check.
8. **(a)** `NaN != NaN`, so a NaN key is never findable and inserts without bound; **(b)** `-0.0 == 0.0` while their bit patterns differ, so bitwise hashing violates the contract. A third, practical reason: values computed by different routes are not bit-equal.
9. Because every byte is **reachable and deliberately retained**. A leak detector reports unreachable allocations; this is a policy failure, not a lost pointer.
10. **References and pointers survive; iterators do not.** It is possible because elements live in **nodes** that are not moved by rehashing — only the bucket pointers are relinked (Topic 41).
11. Because each call is individually atomic and the **sequence between calls is not**. Canonical example: `if (!m.containsKey(k)) m.put(k, compute())`; fix: `m.computeIfAbsent(k, ...)`.
12. A **rehash stall** that grows with the map. Fixes: **reserve**, **incremental rehashing**, or **sharding** into 16–64 tables.

B. Tracing and analysis

1. Time ratio 4,978.9/181.5 = **27.4×** for an n ratio of **5×**. Since $5^2 = 25 \approx 27.4$, growth is **quadratic**. Extrapolating to 200,000 (4× again) gives ≈ 4,979 × 16 = **~80 seconds** on one core.
2. 1.8 ms per request → **~555 requests/s** saturates one core; at 0.084 ms the same core handles **~11,900/s**. A single attacker at **~20 requests/s** consumes 3.6% of a core per request — a few hundred requests per second take a whole machine down.
3. `[1,2]`: ((1×31 + 1)×31 + 2) = 32×31 + 2 = **994**. `[1,2,3]`: 994×31 + 3 = **30,817**. `m.size()` = **1**; `m.get(key)` = **null** (wrong bucket); `m.get(List.of(1,2))` = **null** (right bucket, `equals` now fails).

4. `size()` = **2** — two entries that compare equal. `count()` returns **1 for whichever object you pass** (it hashes to that object's own bucket) and would return 0 for a third equal object with login_count 5.
5. `size()` = **3** (each insert found no match and added a new entry), `count(nan)` = **0**. Both follow from `NaN != NaN`: the lookup reaches the right bucket and the equality test rejects every candidate, including itself.
6. Capacity 2^{24} = 16,777,216 slots × 8 B = **134 MB** retained at `size() == 0`. In libstdc++, `m.rehash(0)` shrinks to fit; otherwise swap with a fresh empty map.
7. Mean effect: 329.8 ms spread over 4,000,000 operations = **+82 ns** on a 100 ns median — it nearly doubles the mean and is invisible if you only look at throughput. p99.99 is the 400th slowest operation, so with 12 operations over 1 ms the p99.99 sits in the **microsecond range**, and the max is **$3.3\times10^{6}\times$ the median**.
8. Each shard holds 250,000 entries, and rehash cost is roughly linear in entries, so **~20 ms per shard resize** — and shards resize at different times, so the worst stall is one shard's 20 ms rather than 330 ms. A **16× improvement** in the tail for no change in total work.

C. Code tasks — what to verify

1. Expect approximately **1.8 / 181.5 / 4,978.9 ms** colliding against **0.084 / 0.911 / 6.813 ms** normal — ratios of **21×, 199×, 731×**. Fitting log(time) against log(n) gives a slope near **2.0**.
2. Expect 50,000 keys to insert in **single-digit milliseconds** again, and `hash("a")` to **differ between the two runs**. If it does not, your seed is being constant-folded — make it non-static or read it from `/dev/urandom`.
3. Expect `d[obj]` to raise `KeyError`, `len(d)` to be **1**, and `list(d.keys())` to **contain the object** — visible, counted, unreachable. Python raises no warning for a mutable custom key, unlike for a list.
4. Expect `0.0` and `-0.0` to behave per your hash: with `std::hash<double>` they typically **unify** (libstdc++ special-cases zero), so `size()` counts one. `0.1+0.2` and `0.3` are **two distinct keys** (0.30000000000000004 ≠ 0.3). The two NaNs add **two entries**, and `count(nan)` is **0**.
5. Expect RSS to grow by **5% of requests × entry size** — 50,000 entries — with growth **proportional to the error rate, not to traffic**. With the RAII guard, RSS is flat.
6. Expect: C++ **silent undefined behaviour**, reported by ASAN as `heap-use-after-free` (no runtime check exists); Java `ConcurrentModificationException` from the `modCount` check; Python `RuntimeError: dictionary changed size during iteration` from a version counter. Go, for contrast, specifies deletion during `range` as legal.
7. Expect a mixture across 20 runs: **lost updates** most often, an **infinite loop at 100% CPU** inside a probe or bucket walk sometimes, and occasionally a crash. The non-determinism is the lesson — the absence of a failure in 20 runs proves nothing.
8. Expect the unreserved run to show **worst ~330 ms, ~12 over 1 ms, p50 ~100 ns**; reserved, **worst ~1 µs**; sharded 16 ways, **worst ~20 ms**. Reserve wins when n is known; sharding is the answer when it is not.

HANDNOTE SUMMARY CARD — Failure modes

```
FAILURE MODES, SECURITY, CONCURRENCY (79-88)
================================================================================
HASH FLOODING -- O(n) input, O(n^2) work.  MEASURED, std::unordered_map<string>:
   n        colliding     normal    ratio        5x the keys -> 27x the time
   1,000       1.8 ms    0.084 ms     21x        => quadratic, confirmed
  10,000     181.5 ms    0.911 ms    199x        200,000 keys ~ 80 SECONDS
  50,000   4,978.9 ms    6.813 ms    731x        of one CPU core
  vectors: POST fields, JSON keys, headers, query params, XML attributes
  DEFENCES: per-process random seed | treeify long bins | CAP THE INPUT COUNT
            (PHP max_input_vars = 1000) | never expose a custom hash to input

SEED RANDOMIZATION BY RUNTIME
  Python 3.3+  randomized (SipHash-1-3), PYTHONHASHSEED pins it -- debug only
  Rust         SipHash-1-3, random key; FxHash/ahash opt out = attack surface
  PHP 7+ / Go  per-process seed (Go also randomizes iteration order)
  Java         NOT randomized -- defends by treeifying at 8 (needs Comparable)
  C++          NO randomization anywhere. You must supply a keyed hash.
  CONSEQUENCE: a hash value is process-local. Never persist, shard or sign
  with it -- use SHA-256 or a fixed-seed xxHash for anything durable.

THE CONTRACT           equal  => same hash    (MANDATORY; violation = bug)
                 same hash  => equal         (NOT required; just a collision)
  hash over a SUBSET of the == fields: legal, slower
  hash over a SUPERSET:                equal keys in different buckets = BROKEN
  mutate a key after insert: entry is counted, iterable, and UNREACHABLE.
    Java [1,2] hash 994 -> mutate to [1,2,3] hash 30817 -> get() null, size() 1
    C++ prevents it (pair<const K,V>); Java and Python do not.

FLOAT KEYS -- don't.  NaN != NaN so a NaN key is never findable and inserts
  without bound (3 inserts -> size 3, count 0).  -0.0 == 0.0 with different
  bits.  0.1+0.2 != 0.3.  FIX: quantise to integers before keying.

UNBOUNDED MAP = leak no detector reports (all bytes reachable, all intentional)
  shapes: session table, memo, interner, in-flight map erased on SUCCESS ONLY
  (that last one grows with the ERROR RATE -- it leaks during incidents)
  erase does not return memory: 10M peak -> 134 MB slot array at size()==0
  FIX: declare an eviction policy at declaration time -- cap, TTL, or RAII

ITERATOR INVALIDATION (C++ unordered_map)
  insert no rehash: all valid | rehash: ITERATORS invalid, REFERENCES valid
  erase: only that element | clear: everything
  idiom:  for (auto it = m.begin(); it != m.end(); )
             if (pred(*it)) it = m.erase(it); else ++it;
  C++ silent UB | Java ConcurrentModificationException | Python RuntimeError
  | Go: deletion during range is legal and specified

CONCURRENCY   global lock < RWLock < striping < CAS-per-bin (Java 8+)
  CHECK-THEN-ACT is the bug that survives every one of them:
    if (!m.count(k)) m[k] = compute();     // two atomic ops, one race
    -> computeIfAbsent / try_emplace / entry().or_insert_with
  Go: concurrent write = FATAL, process dies.  C++: UB, usually a 100% CPU hang.

RESIZE TAIL   4M inserts: 12 over 1 ms, worst 329.8 ms at i=2,938,679, p50 100ns
  the mean moves by 82 ns; the max is 3.3 million x the median
  FIX ranked: reserve (330 ms -> 1 us) > incremental rehash > shard 16x (20 ms)
```

SECTION 11 — SYNTHESIS

89. The master complexity table

Everything in Sections 3–5 in one place, with the constant factors that actually decide, because the asymptotics are identical across the whole table and therefore useless for choosing.

Scheme	Hit	Miss	Delete	Memory/ entry	Max α
Chaining	1 + α/2	1 + α	Unlink, O(1)	32–48 B	> 1 legal
Linear probing	½(1+1/(1−α))	½(1+1/(1−α)²)	Tombstone / shift	16–20 B	0.75
Quadratic	1.99 at 0.75	4.64 at 0.75	Tombstone	16–20 B	0.8
Double hashing	2.02 at 0.75	4.67 at 0.75	Tombstone	16–20 B	0.9
Robin Hood	Same mean as linear	Same, early exit	Backward shift	16–20 B	0.9
Cuckoo	**2, worst case**	**2, worst case**	O(1)	16–24 B	0.5 (2 tables)
Swiss	~1.1 key compares	1 group scan	EMPTY or DELETED	13–27 B	**0.875**
Perfect hash	**1, worst case**	1 + compare	—	2–4 bits index	1.0

```
// The numbers that convert probes into time (Topic 36):
//   L1 1 ns | L2 4 ns | L3 15 ns | DRAM 80-100 ns | hash 10-20 ns
//   chaining at alpha 0.9:  1.45 dependent misses  = ~116 ns
//   linear   at alpha 0.9:  1 miss + 4.3 in-line   = ~84 ns
// Asymptotically identical. In wall-clock terms, a 1.4x difference,
// and at 100,000 entries the measured gap is 2.25x.
```

THE KEY INSIGHT

Every row is O(1) expected and O(n) worst case. The table is therefore not a complexity table at all — it is a *constant-factor* table, and constants are the only thing that distinguishes these designs.

```
use this table with α as the input · convert probes to nanoseconds with the cache latencies
before comparing schemes
```

90. The decision flowchart

```
START: I need to look things up by key.

 1. Do I ever need order, range, min/max, predecessor or prefix?
       YES -> B-tree / std::map / skip list / trie.   STOP. (T6)
       NO  -> continue

 2. Are the keys dense small integers (density > 1/8 of the universe)?
       YES -> plain array. No hash, no collisions, O(1) WORST case. (T3, T71)
       NO  -> continue

 3. Is n below ~30, or is the whole thing under one cache line?
       YES -> vector + linear scan. Beats any hash table by 3-4x. (T6)
       NO  -> continue

 4. Is the key set fixed forever (keywords, headers, opcodes)?
       YES -> minimal perfect hash (gperf / PTHash). 1 probe, worst case. (T45)
       NO  -> continue

 5. Do I only need "have I seen this?", and can I tolerate ~1% false yes?
       YES -> Bloom filter. 1.25 MB per million keys vs 48 MB. (T46)
       NO  -> continue

 6. Do I only need the COUNT of distinct items?
       YES -> HyperLogLog. 12 KB, 0.81% error, any cardinality. (T47)
       NO  -> a real hash table. Continue.

 7. Does any code hold a pointer/reference/iterator into the map across
    an insert?
       YES -> CHAINING (std::unordered_map, node_hash_map), or store
              indices into a vector.                          (T41)
       NO  -> OPEN ADDRESSING, Swiss layout.                  (T35, T38)

 8. Are the keys attacker-controlled?
       YES -> keyed hash with a per-process random seed (SipHash),
              AND cap the number of keys per request.         (T79, T80)

 9. Do I know n in advance?
       YES -> reserve(n). 330 ms tail -> 1 us.                (T56, T88)
       NO  -> incremental rehash, or shard into 16-64 tables. (T24, T88)

10. Will this map live for the process lifetime?
       YES -> state an eviction policy NOW: cap, TTL, or LRU. (T85, T72)
```

THE KEY INSIGHT

Six of the ten questions can end with "do not use a hash table". That proportion is not an accident — it is the most useful thing in this book.

```
work top to bottom · questions 1–6 eliminate the hash table · 7–10 configure it
```

91. The interview drill

What to say, in what order, when a hash-table question arrives. The failure mode in interviews is not ignorance of probing schemes; it is jumping to code before stating the model.

1. **Clarify the key and the operation mix.** "Keys are strings of what length? Roughly how many? Read-heavy or write-heavy? Do we need ordering or just lookup?" This is where you learn whether a hash table is even the answer.

2. **State the structure and its cost model out loud.** "Hash map, O(1) expected, O(n) worst case; the worst case is reachable if keys are attacker-controlled."
3. **Name the space cost with a number.** "About 32–48 bytes per entry for int→int, so 10 million entries is ~400 MB — do we have that?"
4. **Write the code, using the container's atomic primitive.** `try_emplace`, `entry()`, `computeIfAbsent`, `setdefault` — never check-then-act (Topic 87).
5. **Volunteer one trap before being asked.** The strongest candidates say "note that `m[k]` would insert here, so I am using `find`", or "I am seeding the prefix map with {0,1} so subarrays starting at index 0 are counted".
6. **Give the alternative you rejected and why.** "A sorted vector would be smaller and support range queries, but insertion is O(n) and we insert continuously."
7. **Close with the degradation story.** "If keys came from user input I would use a keyed hash and cap the count; if this map lived for the process lifetime I would add an LRU bound."

```
// The four templates to have in muscle memory (Topics 59, 62, 64, 72):
//   complement:  query BEFORE insert
//   prefix sum:  seed the map with {0, 1}
//   window:      guard stale indices with >= start
//   LRU:         map<K, list::iterator> + list, splice on access
// If you can write these four from memory and explain one trap in each,
// you can answer most hash-table interview questions.
```

THE KEY INSIGHT

Steps 1, 3 and 7 are what separate a candidate who has memorised patterns from one who has built systems. They cost about thirty seconds of speaking and they are the part most people skip.

THE TRAP

Saying "O(1)" without qualification. It invites exactly one follow-up — "when is it not?" — and an unprepared answer there undoes a correct solution. The prepared answer is Topics 5, 79 and 88: adversarial keys, a bad hash for this key set, and the amortized-versus-individual distinction.

```
drill — clarify, state the model, quantify space, use the atomic primitive, volunteer a trap,
name the rejected alternative, close with the degradation story
```

FINAL MIXED EXAM — Topics 1–91

Sixty minutes, closed book. Every question draws on at least two sections.

A. Conceptual

1. A lookup is "one hash, one indexed load, one comparison". Name the four mechanisms in this book that exist solely because the comparison sometimes fails.
2. Give the two independent reasons a masked index can be catastrophic, and the one-line fix for each.
3. Chaining needs 1.45 probes at α = 0.9 and linear probing needs 5.33, yet linear probing is faster. Explain in one sentence, with numbers.
4. Name three separate techniques in this book whose purpose is to improve p99 without improving throughput.
5. You are asked to raise a table's load factor from 0.75 to 0.9. State what you need to know before answering.
6. Which single question decides between a flat and a node-based map more often than performance does, and why is it not a performance question?
7. Give three distinct bugs from this book whose symptom is "the entry is in the map but lookups cannot find it".
8. Why does the Swiss table's 7/8 load factor not transfer to a scalar linear-probing table?
9. An interviewer says "so it's O(1)". Give the three qualifications.
10. Name the pattern for each phrase: "within k indices"; "one-to-one"; "subarray sums to k"; "at most two distinct characters".
11. Why is a hash value never a valid database shard key across deployments?
12. State the strongest argument in this book *against* using a hash table.

B. Tracing and analysis

1. A chained map holds 2,000,000 entries with 24-byte keys and 8-byte values at α = 0.75, node overhead 16 B. Compute total memory, then the same figure for a Swiss table at α = 0.7 with one control byte per slot. Give the ratio.
2. A linear-probing table at α = 0.9 accumulates tombstones until (n+tombs)/m = 0.95. Compute the predicted miss probes before and after using Knuth's formula, and the percentage increase.
3. Inserting 1,000,000 keys with no `reserve`, starting from 8 slots and doubling at 0.75: how many rehashes occur, how many total relinks, and how many entries does the final rehash move?
4. Trace `subarrays_summing_to_k([3, 4, 7, 2, -3, 1, 4, 2], 7)`. Give the running prefix, the map contents and the final count.
5. A Bloom filter must hold 20,000,000 keys at a 1% false-positive rate. Compute bits per element, k, and total memory; compare with an exact set at 80 B/entry.
6. A key class compares on (id, email) and hashes on (id, email, last_login). Two objects share id and email but differ in last_login. Give `size()`, the result of `count()` on each, and which half of the contract is broken.
7. 4,000,000 inserts show a p50 of 100 ns and a single 330 ms outlier. Compute the mean with and without the outlier, and state which metric would have caught it.

8. An attacker can send 5,000 colliding keys per request. Using the measured 1.8 ms for 1,000 keys and quadratic growth, estimate the CPU time per request and the requests per second needed to saturate one core.

C. Code tasks

1. **Build.** Implement one hash map with a compile-time switch between chaining and linear probing, sharing the hash and growth policy. Instrument probe counts and report hit/miss averages for both at α = 0.5, 0.75 and 0.9. Confirm against 1.25/1.5, 1.38/2.5 and 1.45/5.33.
2. **Break it and observe.** In the open-addressed mode, replace tombstones with EMPTY on erase. Write a randomised test that inserts, erases and looks up 100,000 times, comparing against `std::map` as an oracle. Report the first divergence and the fraction of lookups that are wrong after 10,000 erases.
3. **Break it and observe.** Add a `reserve` and a per-insert timer. Report worst insert, count over 1 ms, and p99.99 for: no reserve, reserve, and 16-way sharding. Present it as one table.
4. **Attack it.** Expose your map through a function that takes a list of string keys. Write an attacker that produces 20,000 keys colliding under your hash, time the request, then add a per-process random seed and time it again.
5. **Break it and observe.** Implement an LRU cache on top of your map, then hold a reference to a cached value across an insert that triggers eviction and growth. Run under `-fsanitize=address`. Fix it two ways — node-based storage, and indices into a vector — and state which you would ship and why.
6. **Integrate.** Build a word-frequency top-K pipeline over a 100 MB text file: interning (Topic 69), a frequency map (Topic 57), and bucket selection (Topic 78). Report total time, peak RSS, and the time attributable to hashing. Then replace the interner with direct string keys and report the same three numbers.

ANSWER KEY — Final mixed exam

A. Conceptual

1. **Chaining, probe sequences (linear/quadratic/double), tombstones and backward shifting, and the Swiss control byte.** Cuckoo and Robin Hood are refinements of the second.
2. **(a)** The hash has weak low bits (identity on aligned values) — fix by mixing with `fmix64`; **(b)** the capacity is not a power of two, so `m-1` is not a full mask — fix with `assert((m & (m-1)) == 0)`. A prime modulus solves both at 20–40 cycles.
3. Chaining's 1.45 probes are **dependent DRAM misses (~80 ns each, ≈ 116 ns)**; linear probing's 5.33 are one miss plus in-line reads (≈ 84 ns).
4. Any three of: **incremental rehashing** (Topic 24), **Robin Hood** (Topic 32), **reserve** (Topic 56), **sharding** (Topic 88). All redistribute work rather than reduce it.
5. **The probe scheme.** With group scanning (Swiss) 0.9 is normal; with scalar linear probing, miss cost rises from 8.5 to 50.5 probes — a 6× regression. Also whether the workload is miss-heavy, since hits barely move.
6. **"Does any code hold a pointer, reference or iterator into the map across an insert?"** It is an interface question: a flat map cannot provide the guarantee at any speed, so no amount of tuning substitutes.
7. Any three of: **EMPTY written on erase** orphaning later keys (Topic 30); **a mutated key** (Topic 82); **hash over a superset of the equality fields** (Topic 83); **a NaN key** (Topic 84); **no collision handling at all** (Topic 1).
8. Because 7/8 is affordable only when 16 slots are examined in **one SIMD instruction**. Scalar probing at 0.875 costs **32.5 probes per miss**; the number was never the mechanism.
9. **Expected, not worst case**; **amortized, so an individual insert can stall**; **assumes a hash that suits these keys and an input that is not adversarial**.
10. Last-seen index; two-map bijection; prefix sum plus map; sliding window with a frequency map.
11. Because seeds are **randomized per process** and library implementations change between versions, so `hash(key) % nodes` maps the same key to different shards after a restart or upgrade.
12. **It destroys ordering.** Every capability lost — range, predecessor, min/max, sorted iteration, prefix search — follows from that one trade, and retrofitting order is a rewrite while retrofitting speed onto a tree is a tuning change.

B. Tracing and analysis

1. Chained: node = 24 + 8 + 8 (next) = 40, +16 overhead = 56 B × 2M = **112 MB**; slots = 2M/0.75 rounded to 2^{22} = 4,194,304 × 8 = 33.6 MB; total **145.6 MB**. Swiss: m = 2M/0.7 rounded to 2^{22}; slot = 24 + 8 + 1 = 33 B × 4,194,304 = **138 MB**. Ratio **1.05×** — with large keys the memory advantage of flat storage nearly vanishes, which is itself the lesson.
2. At 0.90: ½(1 + 1/0.01) = **50.5**. At 0.95: ½(1 + 1/0.0025) = **200.5**. A **297% increase** with n unchanged.
3. Sizes 8 → 2,097,152: **18 rehashes**. Relinks = 6 + 12 + … + 786,432 ≈ **1,572,858** (about 1.57 per insert). The final rehash moves **786,432** entries in one call.
4. Prefixes: 3, 7, 14, 16, 13, 14, 18, 20. Seed {0:1}. run=3 needs −4 miss; run=7 needs 0 **hit (+1)**; run=14 needs 7 **hit (+1)**; run=16 needs 9 miss; run=13 needs 6 miss; run=14 needs

7 **hit (+1)**; run=18 needs 11 miss; run=20 needs 13 **hit (+1)**. **Count = 4** — namely [3,4], [7], [7,2,-3,1] and [-3,1,4,2] … listing them is the check that your trace is right.

5. 1% needs about **9.6 bits per element** with **k = 7**. Total = 20e6 × 9.6/8 = **24 MB**. Exact set: 20e6 × 80 = **1.6 GB**. Ratio **67×**.
6. `size()` = **2**; `count()` returns **1 for each object when queried with that same object** and 0 for any third equal object. The broken half is **"equal objects must have equal hashes"** — the hash reads a superset of the equality fields.
7. Without the outlier the mean is ~100 ns. With it: 100 + 329,800,000/4,000,000 = **~182 ns**. The metric that catches it is the **maximum per interval** (or p99.99); the mean merely looks 80% worse with no explanation.
8. Quadratic scaling: 5,000 keys is 5× of 1,000, so 25 × 1.8 ms = **~45 ms of CPU per request**. One core is saturated by **~22 requests per second**.

C. Code tasks — what to verify

1. Chaining should land near **1.25/1.5, 1.38/1.75, 1.45/1.9** (hit/miss) and linear probing near **1.50/2.50, 2.49/8.48, 5.33/48.94**. If your chained miss numbers rise sharply with α, you are measuring the slot scan rather than the chain walk.
2. Expect divergence **as soon as a key is inserted after an erase into the same probe run** — often within the first few hundred operations. After 10,000 erases expect on the order of **several percent** of lookups to wrongly report "absent" for present keys. An oracle-based randomised test is the only reliable way to find this class of bug.
3. Expect approximately: no reserve **worst ~330 ms, 12 over 1 ms**; reserve **worst ~1 µs, 0 over 1 ms**; 16-way sharding **worst ~20 ms**. p99.99 should be microseconds in all three cases — which is precisely why p99.99 is not enough on its own.
4. Expect a **several-hundred-fold** slowdown before seeding (20,000 colliding keys ≈ 4× the 10,000-key figure of 181 ms, so **~0.7 s**) and **a few milliseconds** after. Verify the seed changes between runs by printing one hash value.
5. Expect `heap-use-after-free`. Ship the **index-into-a-vector** version if you need flat performance, or the node-based one if the API must hand out references; state that the index version introduces stale-index risk and needs a generation counter.
6. Expect interning to **reduce total time by 30–60%** on a repeated-word corpus (each distinct word is hashed once instead of once per occurrence) while **increasing peak RSS** by the interner's own table. The hashing share should fall from a large fraction of run time to a small one — measure it with `perf` rather than guessing.

HANDNOTE SUMMARY CARD — Synthesis

```
SYNTHESIS (89-91)
================================================================================
MASTER TABLE -- every row is O(1) expected / O(n) worst. Constants decide.
  scheme        hit             miss              mem/entry  max alpha
  chaining      1 + a/2         1 + a               32-48 B   >1 legal
  linear        (1+1/(1-a))/2   (1+1/(1-a)^2)/2     16-20 B   0.75
  quadratic     1.99 @0.75      4.64 @0.75          16-20 B   0.80
  double hash   2.02 @0.75      4.67 @0.75          16-20 B   0.90
  Robin Hood    = linear mean   = linear, early exit 16-20 B   0.90
  cuckoo        2 WORST CASE    2 WORST CASE        16-24 B   0.50
  Swiss         ~1.1 compares   1 group scan        13-27 B   0.875
  perfect hash  1 WORST CASE    1 + compare         2-4 bits  1.00
  convert to time: L1 1ns | L3 15ns | DRAM 80ns | hash 10-20ns
  chaining @0.9 = 1.45 x 80 = 116 ns   linear @0.9 = 80 + 4.3 = 84 ns

THE DECISION FLOWCHART -- six of the ten questions END with "not a hash table"
  1 need order/range/min/max/prefix?      -> tree, skip list, trie
  2 dense small int keys (density > 1/8)? -> plain array, O(1) WORST case
  3 n < ~30?                              -> vector + linear scan (3-4x faster)
  4 key set fixed forever?                -> minimal perfect hash, 1 probe
  5 membership only, ~1% false yes ok?    -> Bloom, 1.25 MB per 1M keys
  6 only need distinct COUNT?             -> HyperLogLog, 12 KB, 0.81%
  7 refs held across inserts?             -> chaining, else Swiss/flat
  8 keys attacker-controlled?             -> keyed seed + cap the input count
  9 know n?                               -> reserve(n): 330 ms tail -> 1 us
 10 lives for the process lifetime?       -> state an eviction policy NOW

THE INTERVIEW DRILL, IN ORDER
  1 clarify key type, n, read/write mix, ordering needs
  2 state the model: "O(1) expected, O(n) worst, worst reachable by input"
  3 quantify space: "~40 B/entry, so 10M entries is ~400 MB -- do we have it?"
  4 use the atomic primitive: try_emplace / entry / computeIfAbsent / setdefault
  5 volunteer ONE trap before being asked ("m[k] would insert here")
  6 name the alternative you rejected and why
  7 close with the degradation story (adversarial keys, growth, eviction)

"SO IT'S O(1)?" -- THE THREE QUALIFICATIONS
  expected, not worst   |   amortized, so one insert can stall 330 ms   |
  assumes a hash that suits THESE keys and an input nobody chose against you

THE FOUR TEMPLATES IN MUSCLE MEMORY
  complement: query BEFORE insert          prefix sum: seed the map with {0,1}
  window: guard with >= start              LRU: map<K,list::iterator> + splice
```

PART I CHEATSHEET — Mechanism (Topics 1–41)

```
PART I -- MECHANISM: everything from Sections 1-5 on one page
================================================================================
THE MODEL      lookup = hash(k) -> index -> compare.  alpha = n/m controls all.
COSTS          hash 10-20 ns | L1 1 | L2 4 | L3 15 | DRAM 80-100 ns | 64 B line
               hot lookup ~16 ns (hash dominates) | cold ~95 ns (miss dominates)

INDEXING       h % prime      20-40 cyc, uses all bits, needs a prime table
               h & (m-1)      1 cyc, uses log2(m) LOW bits -> REQUIRES a mixer
               (h*GOLDEN)>>s  3-4 cyc, uses HIGH bits.  GOLDEN=11400714819323198485
MIXER          fmix64: k^=k>>33; k*=0xff51afd7ed558ccd; k^=k>>33;
                       k*=0xc4ceb9fe1a85ec53; k^=k>>33;     avalanche 32.0/64
STRING HASH    fnv1a  h ^= c; h *= 1099511628211    (basis 1469598103934665603)
COMPOSITE      hash_combine(s,v): s ^= v + 0x9e3779b97f4a7c15 + (s<<6) + (s>>2)
               XOR is WRONG: (3,7)==(7,3), (x,x)->0, measured 3117 ns vs 8 ns

CHAINING       hit 1 + a/2   miss 1 + a   empty m*e^-a   iterate O(m + n)
               insert = find-then-prepend; erase via Node** pp (no head case)
               grow at 0.75 by doubling: 11 rehashes and 1.23 relinks/insert
               for 10k keys; shrink needs a >=4x deadband or it thrashes
               measured 10k keys: m=16384, a=0.610, longest 6, empty 8885
               memory ~77-109 B per string entry = 5.9x payload

OPEN ADDRESS   EMPTY stops a search | TOMB is walked through | FULL compares
               linear    hit (1+1/(1-a))/2      miss (1+1/(1-a)^2)/2
                 a:      0.50   0.75   0.90   0.95
                 hit     1.50   2.49   5.33  10.32
                 miss    2.50   8.48  48.94 195.29   <- the cliff is the MISS
                 worst     22     84    498   1608
               quadratic h + i(i+1)/2 (triangular = all slots); i*i reaches half
               double    h + i*h2, h2 MUST be odd (| 1)
               Robin Hood: same mean, worst 8/16/29/61 -- a p99 technique
               cuckoo: 2 probes WORST CASE, alpha <= 0.5, insert may cycle
               tombstone rot: n pinned, 5 churn cycles -> 12202 tombs, miss +85%

LAYOUT         count CACHE LINES in dependency order, not probes
               chaining @0.9 = 1.45 x 80 ns = 116 ns | linear = 80 + 4.3 = 84 ns
               Swiss: h1 = high 57 bits -> group; h2 = low 7 bits -> ctrl byte
                      16 ctrl bytes, one SIMD compare, ~1.12 key compares
                      ctrl: 0x80 EMPTY, 0xFE DELETED; max load 7/8
               measured flat vs unordered_map: 2.00x @1k, 2.25x @100k,
                      1.08x @4M (both DRAM-bound), 109 MB vs 195 MB

STABILITY      unordered_map: REFERENCES survive rehash, iterators do not
               flat maps: nothing survives growth
               pattern for both: map<K,size_t> index into a vector<V>

DECIDE         order/range needed -> tree | dense small ints -> array |
               n<30 -> vector scan | refs held -> chaining | else Swiss

TOP TRAPS      m not a power of two + mask (4 of 10 slots reachable)
               std::hash<int> is the identity function
               EMPTY on erase orphans every key behind it
               no find-before-insert -> silent multimap
               operator[] on a read path -> phantom entries
               "O(1) so latency is bounded" -> one insert took 330 ms
```

PART II CHEATSHEET — Shapes and tooling (Topics 42–56)

```
PART II -- SHAPES AND TOOLING: Sections 6-7 on one page
================================================================================
SET             map minus the value; ~30% less memory
                if (!s.insert(x).second) {...}   one hash, never count-then-insert
                intersection/union: iterate the SMALLER, probe the larger

MULTI-VALUE     map<K, vector<V>> beats multimap 17x on memory (1 key x 1000 vals:
                4 KB vs 68 KB) and keeps values contiguous. multimap wins only
                when you erase individual (key,value) pairs.

ORDERED         compact two-array layout: sparse int32 indices + dense entries
                iteration O(n) in insertion order, 20-25% smaller
                delete leaves a hole until the next resize

PERFECT HASH    fixed key sets only. 1 hash + 1 read + 1 COMPARE (never skip it).
                gperf for keywords; CHD/PTHash reach 2-4 bits/key.

SKETCHES        Bloom     10 bits/elem, k=7 -> 0.82% FP; 1M keys 1.25 MB vs 48 MB
                          no deletion EVER (clearing bits = false negatives)
                count-min never underestimates; error <= eN, e = e/w
                          never for billing, quotas or hard cutoffs
                HLL       p=14 -> 16384 registers, 12 KB, 0.81% error, any n
                all three MERGE: OR, +, register-max

STANDARD LIBRARIES
  C++ unordered_map  chaining, ~48 B/entry, max_load 1.0, PRIME buckets,
                     refs stable. Slow BECAUSE of the spec, not the implementer.
  Java HashMap       cap 16, load 0.75, treeify 8 / untreeify 6, MIN_TREEIFY 64,
                     spread h ^ (h>>>16), resize splits a bin by ONE bit
  Python dict        compact+ordered, ~36 B/entry, resize at 2/3 to used*3,
                     PYTHONHASHSEED, hash(1)==hash(1.0)==hash(True)
  PHP array          packed ~16 B (8.2+) vs hashed ~36 B; always insertion-order
                     DJBX33A + seed; "10" -> int 10 but "010" stays a string
  JS Object          keys are STRINGS; integer-like keys iterate FIRST ascending;
                     __proto__ pollution.  JS Map: any key type, true order
  Go map (1.24+)     Swiss, 8-slot SWAR groups, load 7/8, RANDOM iteration,
                     &m[k] does not compile, concurrent write = FATAL
  Rust HashMap       hashbrown + SipHash-1-3 keyed per process; entry() API;
                     FxHash/ahash 2-4x faster but attackable
  Redis hash         listpack until 128 entries / 64-byte values, then dict.
                     ONE-WAY. Use SCAN, never KEYS.

ONE-HASH IDIOMS  C++ try_emplace | Rust entry().or_insert | Java computeIfAbsent
                 Python setdefault / defaultdict / Counter | Go v, ok := m[k]

RESERVE -- CHECK THE UNITS
  C++ reserve(n)=ELEMENTS | C++ rehash(b)=BUCKETS | Java ctor=BUCKETS (n/0.75+1)
  Go make(m, n)=elements  | Rust with_capacity(n)=elements
  measured 4M inserts: 22 rehashes, 1.9 s, worst 330 ms  ->  0, 0.9 s, ~1 us
```

PART III CHEATSHEET — Using it (Topics 57–78)

```
PART III -- USING IT: Sections 8-9 on one page
================================================================================
THE FIFTEEN PATTERNS, BY THE PHRASE THAT TRIGGERS THEM
  "how many times"            frequency map          O(n) / O(k)
  "duplicate", "visited"      seen-set, insert().second
  "two numbers that sum to"   complement -- QUERY BEFORE INSERT
  "within k indices"          last-seen index, overwrite (nearest wins)
  "group", "anagram"          canonical key (iff-equal form; sorted or counts)
  "subarray sums to k"        prefix sum + map, SEED {0,1}
  "divisible by k", "equal
   0s and 1s", "even vowels"  prefix state: mod k, +-1 balance, parity bitmask
  "longest substring with"    window + freq map, guard with >= start
  "one-to-one", "isomorphic"  TWO maps (fwd and rev)
  "n <= 40, subsets"          meet in the middle, 2^(n/2), reserve!
  string/pair vertices        map-of-vectors graph; mark visited at ENQUEUE
  overlapping recursion       memoization; pack the key, never XOR
  same heavy key many times   intern to int, then everything is an array
  "repeated substring len L"  rolling hash + set; VERIFY or use M = 2^61-1
  26 letters / 256 bytes      int[26] / int[256] / uint64 bitmask -- 4-8x faster

VARIANT SELECTOR   "exists" -> membership | "count" -> counts |
                   "longest" -> FIRST index, never overwrite

COMPOSITE DESIGNS -- map + a second structure
  LRU     map<K, list<pair<K,V>>::iterator> + list; splice on access; evict back
          works because list nodes never move. vector => O(n) + dead iterators
          ~88 B/key for an int->int payload
  LFU     kv{k->(v,f)} + buckets{f->list<k>} + pos{k->iterator} + minf
          minf only ++ (bucket emptied) or resets to 1 (insert)
          classic LFU never forgets -> decay / W-TinyLFU in production
  RandSet vector<int> vals + map value->index; remove = swap with last,
          then pop_back, then erase. uniform_int_distribution, NOT % size
          duplicates: value -> set<index>; the killer test is
          insert(1) insert(1) remove(1) remove(1)
  TimeMap map<key, sorted vector<(time,val)>> + upper_bound then prev()
          RULE: hash the EQUALITY dimension, order the RANGE dimension
  Top-K   heap O(d log k) | bucket O(n), size buckets by max_count not by n

MEASURED BUGS
  m[x] on a read path        1e6 phantom entries, tens of MB, no error
  insert before query        two_sum([3,2,4],6) returns (0,0)
  missing {0,1} seed         [1,2,3] k=3 -> 1 instead of 2
  missing >= start guard     "abba" -> 3 instead of 2, 5-15% of random inputs
  one map for isomorphism    ("ab","aa") accepted; 10-25% of fuzz cases
  hash match not verified    M=1e9+7, 1e6 windows -> ~500 false duplicates
```

PART IV CHEATSHEET — Living with it (Topics 79–91)

```
PART IV -- LIVING WITH IT: Sections 10-11 on one page
================================================================================
HASH FLOODING   O(n) input -> O(n^2) work.  MEASURED (unordered_map<string,int>)
   n = 1,000     1.8 ms vs 0.084 ms   21x
   n = 10,000  181.5 ms vs 0.911 ms  199x     5x the keys -> 27x the time
   n = 50,000 4978.9 ms vs 6.813 ms  731x     200k keys ~ 80 s of one core
  DEFENCES  per-process keyed hash (SipHash) + CAP the key count per request
            + treeify long bins (Java).  C++ randomizes NOTHING by default.
  CONSEQUENCE  hash values are process-local: never persist, shard or sign them

THE CONTRACT    equal => same hash (MANDATORY) | same hash => equal (not needed)
  hash over a SUBSET of the == fields = legal, slower
  hash over a SUPERSET = equal keys in different buckets = BROKEN
  mutate a key after insert -> counted, iterable, UNREACHABLE, no error
  float keys: NaN != NaN (never findable, inserts forever), -0.0 == 0.0 with
  different bits, 0.1+0.2 != 0.3.  Quantise to integers instead.

"ENTRY IS THERE BUT LOOKUPS MISS" -- the five causes
  EMPTY written on erase | key mutated | hash uses more fields than == |
  NaN key | no collision handling at all

MEMORY          an unbounded map is a leak no detector reports
  shapes: sessions, memo tables, interners, in-flight maps erased on SUCCESS
  ONLY (that one grows with the ERROR RATE, i.e. during incidents)
  erase does not return memory: 10M peak -> 134 MB slot array at size()==0
  every long-lived map needs a cap, a TTL or an LRU bound AT DECLARATION

ITERATOR RULES (C++ unordered_map)
  insert+rehash: iterators invalid, REFERENCES VALID | erase: only that one
  idiom: for (auto it=m.begin(); it!=m.end(); ) if (p(*it)) it=m.erase(it);
                                                else ++it;
  C++ silent UB | Java ConcurrentModificationException | Python RuntimeError |
  Go: delete during range is legal

CONCURRENCY     global lock < RWLock < striping < CAS-per-bin (Java 8+)
  CHECK-THEN-ACT survives all of them: if (!m.count(k)) m[k] = f();
  -> computeIfAbsent / try_emplace / entry().or_insert_with
  Go: concurrent write = fatal, process dies. C++: UB, usually a 100% CPU hang.

TAIL LATENCY    4M inserts: p50 100 ns, 12 over 1 ms, worst 329.8 ms
  the mean moves 82 ns; the MAX is 3.3 million x the median
  fixes ranked: reserve (-> 1 us) > incremental rehash > shard 16x (-> 20 ms)
  measure MAX PER INTERVAL, not the mean

THE THREE QUALIFICATIONS ON "O(1)"
  expected not worst | amortized, so one op can stall | assumes a hash that
  suits THESE keys and an input nobody chose against you
```

MASTER CHEATSHEET — Every topic

```
MASTER CHEATSHEET  1/3 -- THE MODEL AND THE MECHANISM
================================================================================
THE THREE QUESTIONS, ASKED IN THIS ORDER, FOR EVERY HASH-TABLE DECISION
  1. Do I need ORDER?          yes -> not a hash table (tree, trie, skip list)
  2. What is my ALPHA and my probe scheme?    they decide every cost in the book
  3. Who CHOOSES the keys?     an attacker -> keyed seed + input caps

TOPIC MAP
   1-6   foundation: array-index trick, dictionary ADT, direct address, alpha,
         expected vs worst, hash vs tree vs sorted array vs trie
   7-15  hash functions: division, multiplicative, masking, djb2/fnv/sdbm,
         rolling, avalanche, universal families, composite keys
  16-25  chaining: anatomy, insert, lookup, erase, iterate, alpha, resize,
         shrink hysteresis, incremental rehash, memory arithmetic
  26-35  open addressing: probe sequences, tombstones, backward shift,
         Robin Hood, cuckoo, hopscotch, the decision table
  36-41  cache: hierarchy, AoS/SoA, Swiss tables, 7/8 load, measured deltas,
         reference stability
  42-47  variants: sets, multimaps, ordered maps, perfect hashing, Bloom,
         count-min, HyperLogLog
  48-56  standard libraries: C++, Java, Python, PHP, JS, Go, Rust, Redis, reserve
  57-71  fifteen problem patterns
  72-78  composite designs: LRU, LFU, RandomizedSet, TimeMap, from scratch, top-K
  79-88  failure modes: flooding, seeds, iteration order, key mutation, the
         hash/equals contract, float keys, unbounded growth, invalidation,
         concurrency, tail latency
  89-91  master table, decision flowchart, interview drill

THE NUMBERS TO KNOW COLD
  alpha            0.50   0.75   0.90   0.95
  chaining hit     1.25   1.38   1.45   1.48        = 1 + a/2
  chaining miss    1.50   1.75   1.90   1.95        = 1 + a
  linear hit       1.50   2.49   5.33  10.32
  linear miss      2.50   8.48  48.94 195.29        <- squared term
  linear worst       22     84    498   1608
  Robin Hood worst    8     16     29     61
  empty fraction   e^-a:  0.61   0.47   0.41   0.39
  L1 1 ns | L2 4 | L3 15 | DRAM 80-100 | hash 10-20 | 64-byte line
  chaining 32-48 B/entry | flat 16-20 | Swiss uint64->int32 13 B
  growth 8->16384 over 10k inserts: 11 rehashes, 1.23 relinks per insert
  4M inserts unreserved: worst single insert 330 ms, 12 over 1 ms, p50 100 ns
  50k colliding keys: 4.98 s vs 6.8 ms = 731x
```

```
MASTER CHEATSHEET  2/3 -- THE FOUR TEMPLATES WORTH MEMORISING
================================================================================
1. CHAINED MAP -- the structure itself (Topics 16-22)
   size_t idx(K k){ return mix(hash(k)) & (m-1); }        // m = power of two
   V* find(K k){ for (Node* c=b[idx(k)]; c; c=c->next)
                   if (c->key==k) return &c->val;
                return nullptr; }
   void insert(K k, V v){ if (V* p=find(k)) { *p=v; return; }
                          b[i] = new Node{k,v,b[i]};
                          if (++n > m*3/4) rehash(2*m); }
   void erase(K k){ for (Node** pp=&b[idx(k)]; *pp; pp=&(*pp)->next)
                       if ((*pp)->key==k){ Node* d=*pp; *pp=d->next;
                                           delete d; --n; return; } }
   rehash: SWAP the array first, then reinsert, so idx() uses the new capacity

2. COMPLEMENT LOOKUP -- Two Sum and its family (Topic 59)
   for (int i=0;i<n;i++){
       auto it = pos.find(target - a[i]);     // QUERY FIRST -- no self-pairing
       if (it != pos.end()) return {it->second, i};
       pos[a[i]] = i;
   }

3. PREFIX SUM + MAP -- every "subarray sums to k" (Topics 62-63)
   unordered_map<long long,int> seen{{0,1}};       // SEED = the empty prefix
   long long run = 0; int count = 0;
   for (int x : a){ run += x;
                    if (auto it=seen.find(run-k); it!=seen.end())
                        count += it->second;
                    ++seen[run]; }
   "longest" variant: store the FIRST index of each state and never overwrite
   negatives: normalise with ((run % k) + k) % k

4. WINDOW + FREQUENCY MAP -- every "longest/at most k distinct" (Topic 64)
   int start = 0, best = 0;
   for (int i=0;i<n;i++){
       if (auto it=last.find(s[i]); it!=last.end() && it->second >= start)
           start = it->second + 1;             // >= start GUARDS stale indices
       last[s[i]] = i;
       best = max(best, i - start + 1);
   }
   counting variant: ERASE keys whose count hits zero, or size() lies

BONUS -- LRU, because it composes two structures (Topic 72)
   map<K, list<pair<K,V>>::iterator> pos; list<pair<K,V>> order;
   get:  order.splice(order.begin(), order, it->second);
   put:  if full { pos.erase(order.back().first); order.pop_back(); }
         order.push_front({k,v}); pos[k] = order.begin();
```

```
MASTER CHEATSHEET  3/3 -- THE UNIVERSAL TEST MATRIX
================================================================================
Run every hash-table implementation and every hash-table-based solution against
these. Each row exists because it caught a real defect in this book.

  #  CASE                              EXPECTED / WHAT IT CATCHES
  1  empty container                   find -> not found; iterate -> 0 visits
  2  single element                    find hit, find miss, erase, size 0 after
  3  duplicate insert                  size unchanged, value UPDATED (not 2 rows)
  4  erase then re-insert same key      found again; tombstone reused
  5  erase the ONLY element             size 0, iteration empty, no dangling head
  6  erase mid-chain / mid-probe-run   keys AFTER it still findable  <- T30
  7  erase the LAST array slot         swap-with-last guard          <- T75
  8  insert exactly at the threshold   grows once, all keys still findable
  9  insert 2x threshold               all keys findable after 2 rehashes
 10  all keys collide (constant hash)  correct, and quadratically slow <- T79
 11  keys differing only in low bits   mask + weak hash -> clustering  <- T10
 12  keys 64 bytes apart (pointers)    identity hash -> 16 of 1024 slots
 13  negative / signed keys            no negative index; ((h%m)+m)%m
 14  key mutated after insert          documents the unreachable-entry bug
 15  NaN, -0.0, 0.1+0.2 as keys        NaN never found; size grows anyway
 16  equal keys, different hash        the map holds two "equal" entries <- T83
 17  erase during iteration            it = m.erase(it); ASAN clean
 18  reference held across a rehash    valid for node maps, UB for flat
 19  reserve(n) then insert n          zero rehashes; check bucket_count
 20  1e6 lookups of ABSENT keys        map size must NOT grow (no operator[])
 21  iteration order across 2 runs     must not be asserted anywhere
 22  concurrent read + write           locked, or documented as unsafe
 23  churn at constant size            tombstones and latency must not drift
 24  peak then drain to zero           memory retained? state the policy

MEASURE, DON'T ASSERT
  probe counts (hit and miss), longest chain, empty slots, tombstone count,
  (n+tombs)/m, rehash count, MAX single-operation latency, bytes per entry.
  If your monitoring exports only size() and mean latency, Topics 30 and 88
  are invisible to you.
```

PATTERN-RECOGNITION TABLE

Read the left column as the words that appear in the problem statement. This is the fastest path from a question to the right two paragraphs of this book.

The problem says…	Use	Topic
"count occurrences", "most common", "how many times"	Frequency map, or `int[26]`	57, 71
"contains duplicate", "unique", "already visited"	Seen-set, `insert().second`	58
"two numbers that sum to", "pair with difference k"	Complement lookup; query before insert	59
"within k indices", "nearest previous occurrence"	Last-seen index map	60
"group anagrams", "same shape", "normalise"	Canonical key	61
"subarray sums to k", "count subarrays with sum"	Prefix sum + map, seeded {0,1}	62
"divisible by k", "equal number of 0s and 1s", "even count of"	Prefix state: remainder, balance, parity mask	63
"longest substring without repeating", "at most k distinct"	Sliding window + frequency map	64
"isomorphic", "follows the pattern", "one-to-one"	Two maps, both directions	65
"n ≤ 40", "choose a subset summing to"	Meet in the middle	66
Vertices are strings, pairs or arbitrary ids	Map-of-vectors graph + hash set visited	67
"count the ways", overlapping recursive calls	Memoization with a packed key	68
The same expensive key is hashed repeatedly	Intern to an integer id	69
"repeated substring of length L", "any duplicate window"	Rolling hash + set (verify matches)	70
Keys are letters, bytes, months, small enums	`int[26]`, `int[256]`, or a bitmask	3, 71
"least recently used", "evict", "capacity"	Map + doubly linked list	72
"least frequently used"	Map + per-frequency lists + minf	73
"getRandom in O(1)", "uniform random element"	Dense vector + index map, swap with last	74, 75
"value at time t", "as of", versioned lookup	Map + sorted vector + binary search	76
"design a hash map"	State five decisions, then ~25 lines	77
"top K frequent"	Frequency map + heap, or frequency buckets	78
"is it in the set" and memory is tight	Bloom filter	46

"how many distinct users" at scale	HyperLogLog	47
Keys are fixed forever (keywords, headers)	Minimal perfect hash	45
"range", "min", "predecessor", "sorted output", "prefix"	**Not a hash table** — tree, skip list, trie	6, 90

REVISION TRACKER

Spaced repetition, filled in by hand. The intervals are Day 0 (first read), +1 day, then practice, then +7, +30 and +90 days. Write the date in each cell. A section is "done" when you can reproduce its summary card from memory and explain one trap in your own words.

Section	First	Rev 1 +1d	Practice	Rev 2 +7d	Rev 3 +30d	Rev 4 +90d
1. Foundation (1–6)						
2. Hash functions (7–15)						
3. Chaining (16–25)						
4. Open addressing (26–35)						
5. Cache and layout (36–41)						
6. Variants (42–47)						
7. Standard libraries (48–56)						
8. Problem patterns (57–71)						
9. Composite designs (72–78)						
10. Failure modes (79–88)						
11. Synthesis (89–91)						
Final mixed exam (1–91)						

How to use this book a second time

Read a section, then close the book and write its summary card from memory before turning to it. The gap between what you produce and what is printed is your actual revision list — everything else is re-reading, which feels like learning and is not.

For the practice sets, group C is the part that matters. Every one of those tasks was chosen because the failure it produces is invisible in ordinary testing: a map that reports the wrong size, a lookup that misses a key it contains, an insert that stalls for a third of a second, a filter that denies an entry it has seen. Reading about them changes nothing. Running them once means you will recognise the symptom the next time it appears in production, which is the only reason any of this is worth memorising.

Everything About Hash Tables · Core Computer Science, Volume 2 · 91 topics, 11 sections, 4 parts. All measurements in this book were produced by compiling and running the programs described, on x86-64 Linux with GCC at -O2; your constants will differ, your ratios should not.